

# From CLIP to Vision-Language Models

Following our CLIP lecture · ES 667 · Nipun Batra · IIT Gandhinagar

## How Images Become Context for Language Generation

Last time: compare an image with descriptions.

Today: use the image to generate an answer.



| How does visual information become context for next-token prediction?

# Recall: two encoders, one comparison space



CLIP compares an image vector with a supplied text vector.

# Recall: paired examples teach the match



Update both encoders, both projections and the score scale.

The matching image-caption pair is the target in both directions.

# Recall: reuse the vectors for different tasks

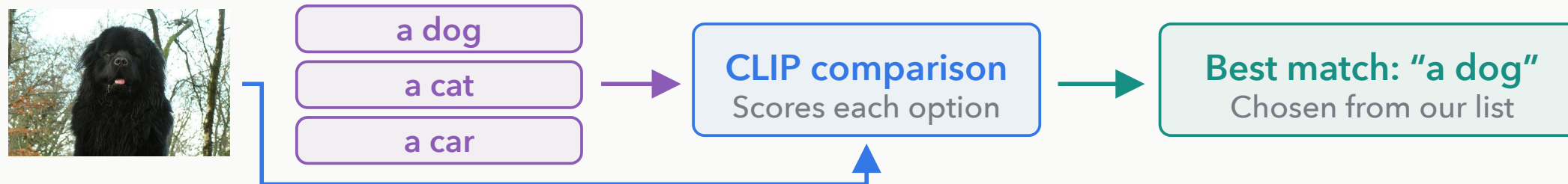


Reuse the frozen encoders. Choose which vectors to compare.

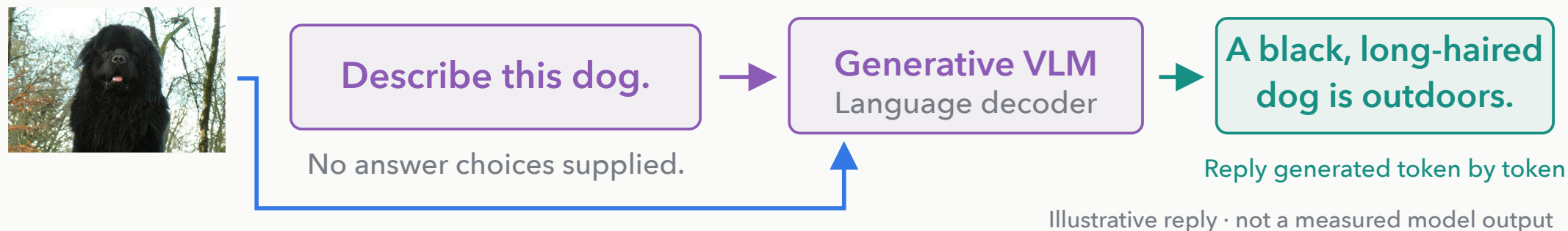
**New descriptions give us new choices without retraining the encoders.**

# Next task: generate an answer from the image

## CLIP · WE SUPPLY THE POSSIBLE DESCRIPTIONS



## GENERATIVE VLM · WE SUPPLY AN IMAGE + A QUESTION



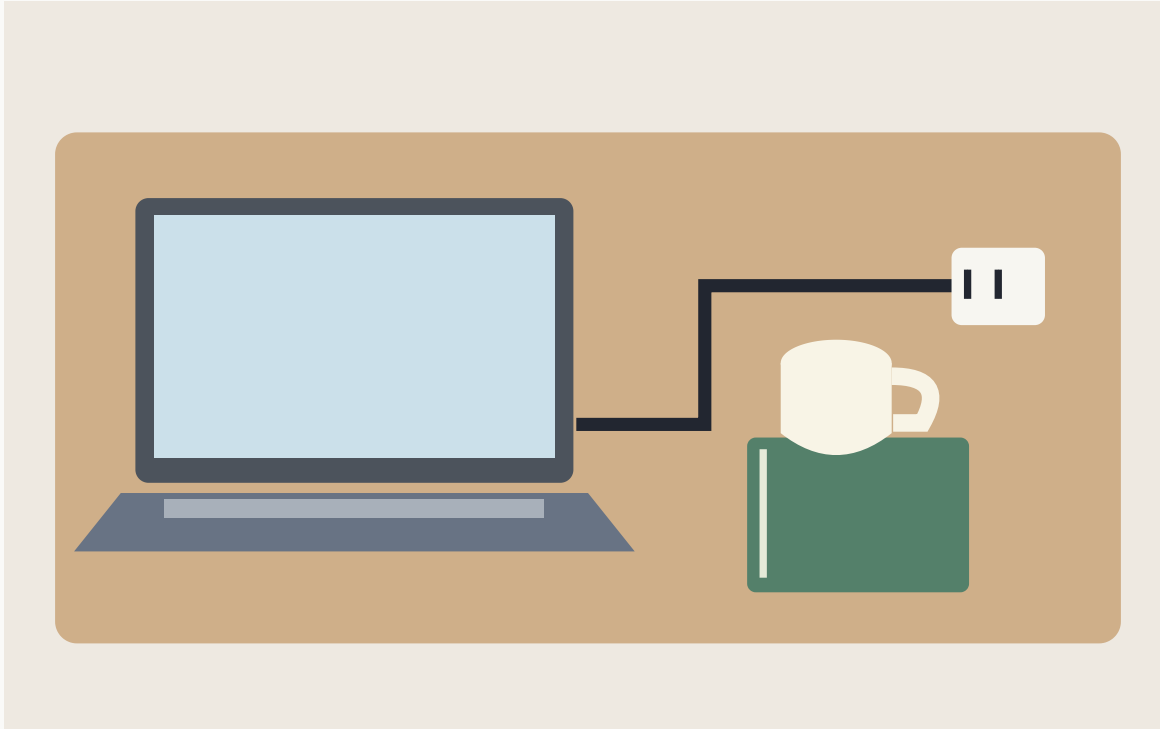
A language decoder writes the reply, using the image as context.

## SECTION 01

# What can a VLM do?

Start with the evidence an answer needs.

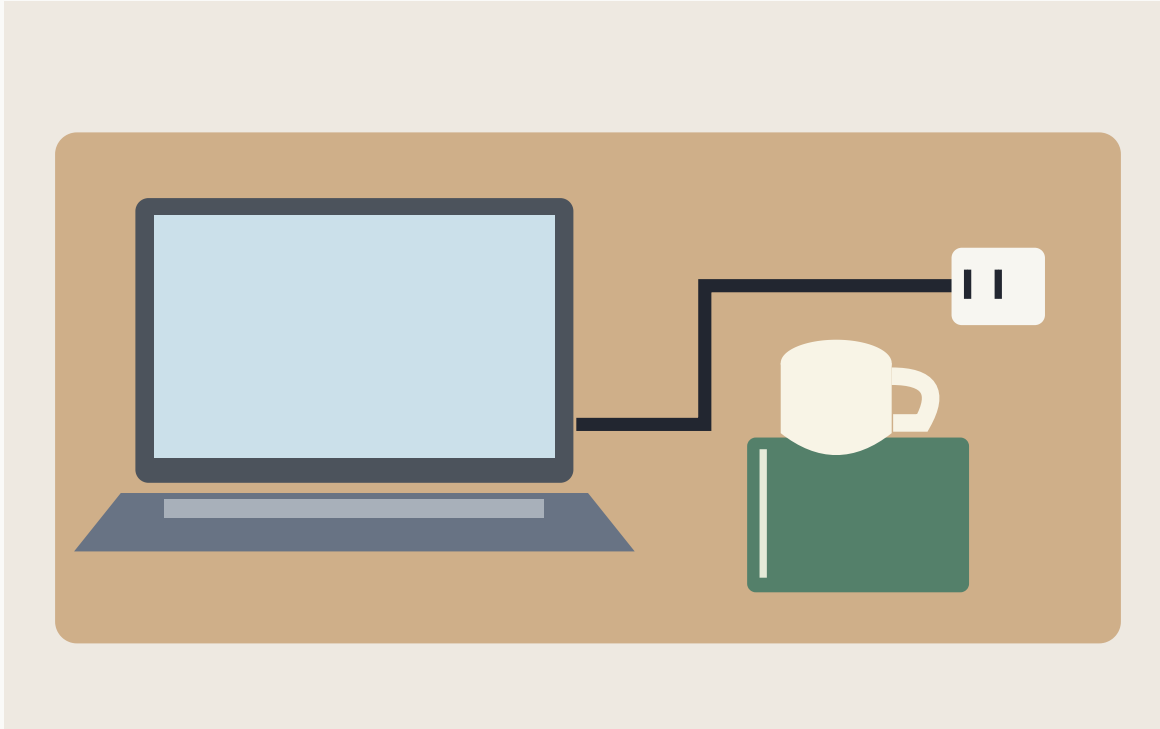
# Can the answer use visible evidence?



QUESTION

Is the laptop plugged in?

# Can the answer use visible evidence?



REFERENCE ANSWER

Is the laptop plugged in?

---

Yes. A cable connects the laptop to the socket.

# Can the answer use visible evidence?



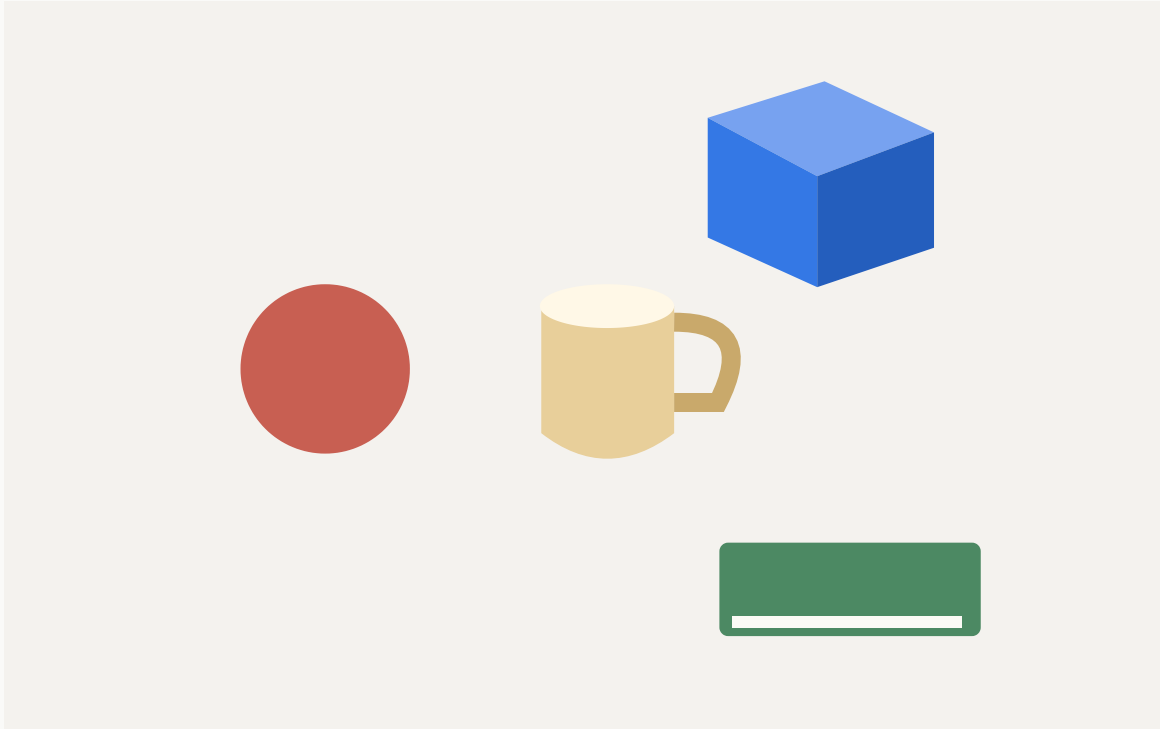
ARCHITECTURAL REQUIREMENT

Is the laptop plugged in?

Yes. A cable connects the laptop to the socket.

| Trace a connection between objects.

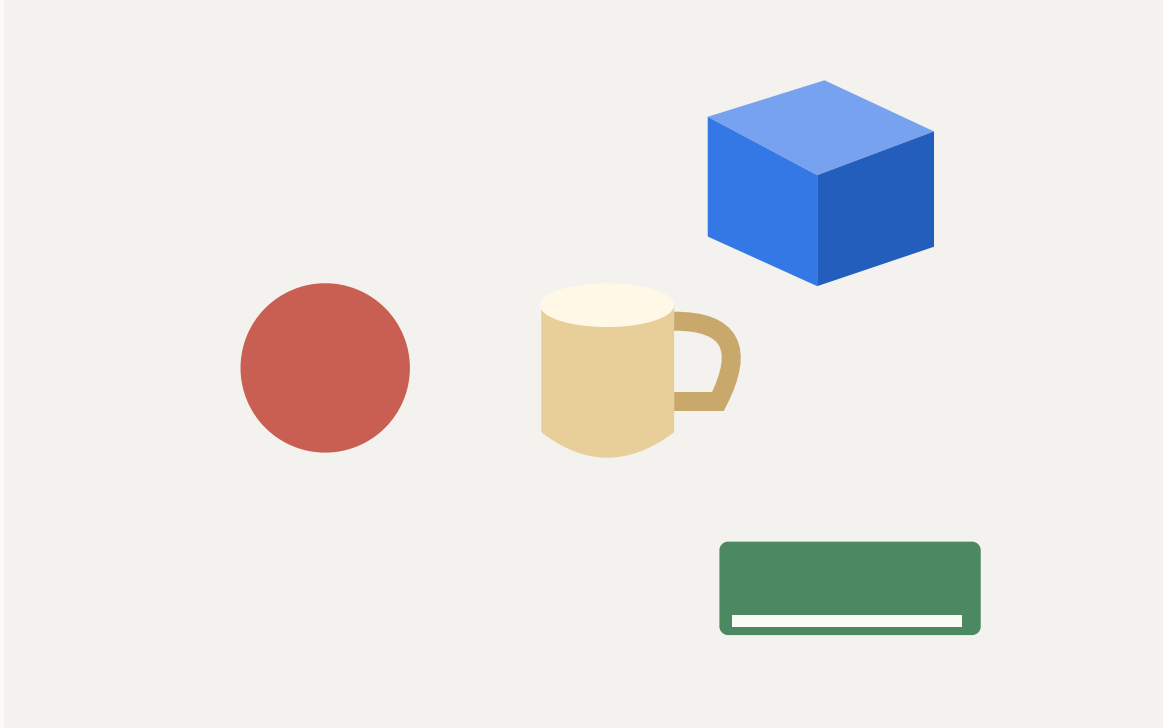
# Identity is only part of the question



QUESTION

What is immediately left of the mug?

# Identity is only part of the question



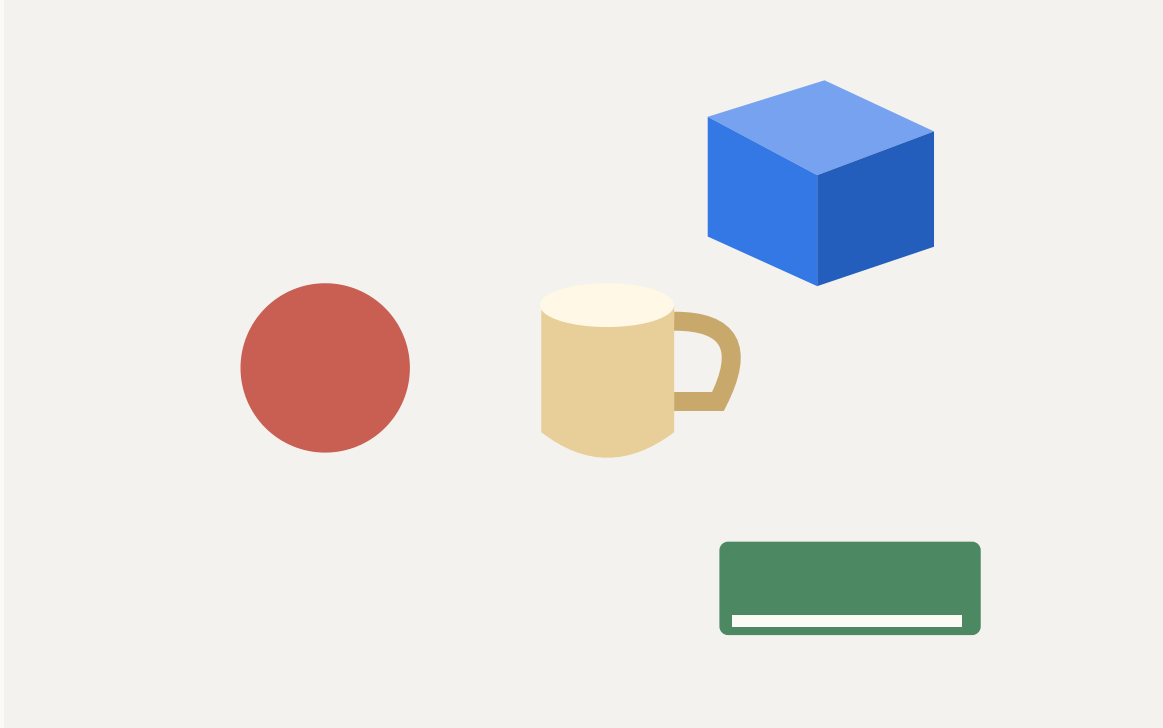
REFERENCE ANSWER

What is immediately left of the mug?

---

The red ball.

# Identity is only part of the question



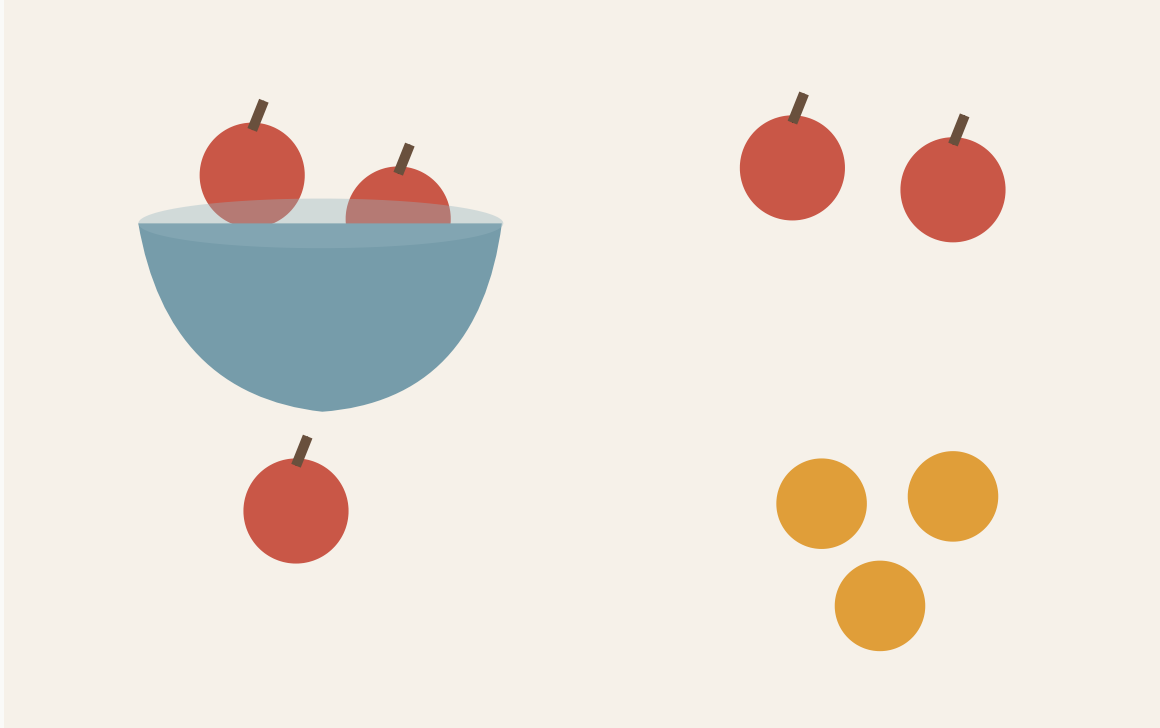
ARCHITECTURAL REQUIREMENT

What is immediately left of the mug?

The red ball.

| Keep object identity and spatial relations.

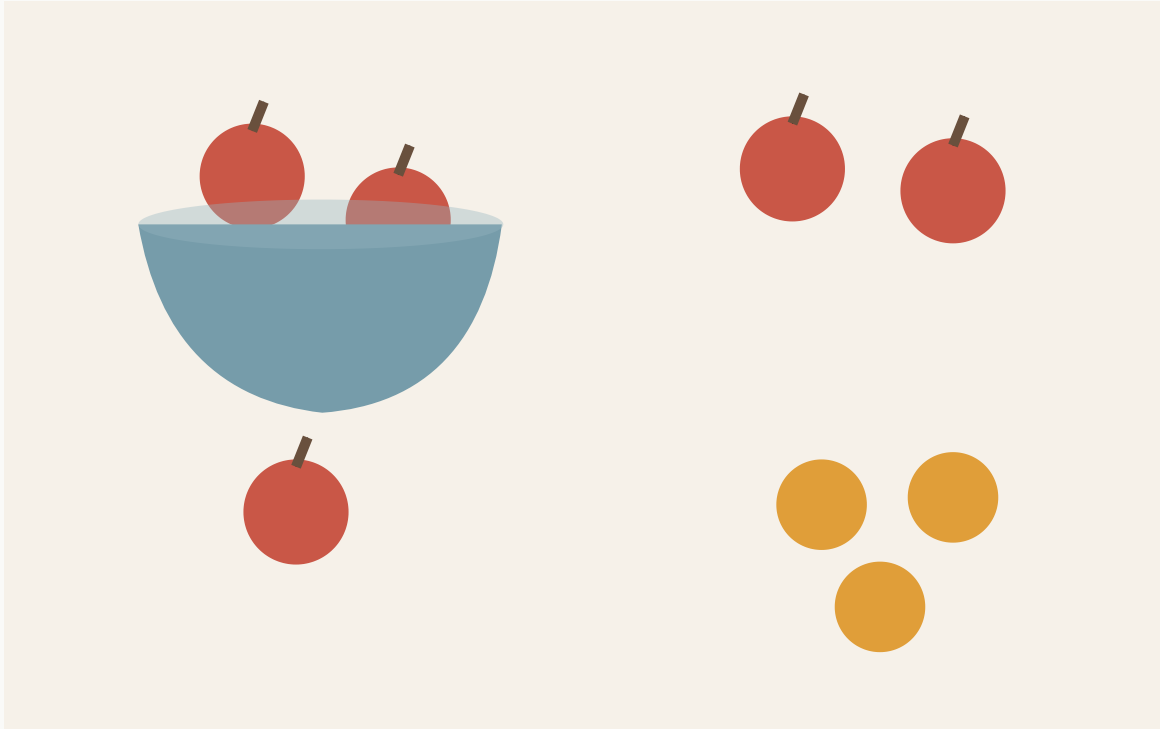
# The question selects a subset



QUESTION

How many apples are outside the bowl?

# The question selects a subset



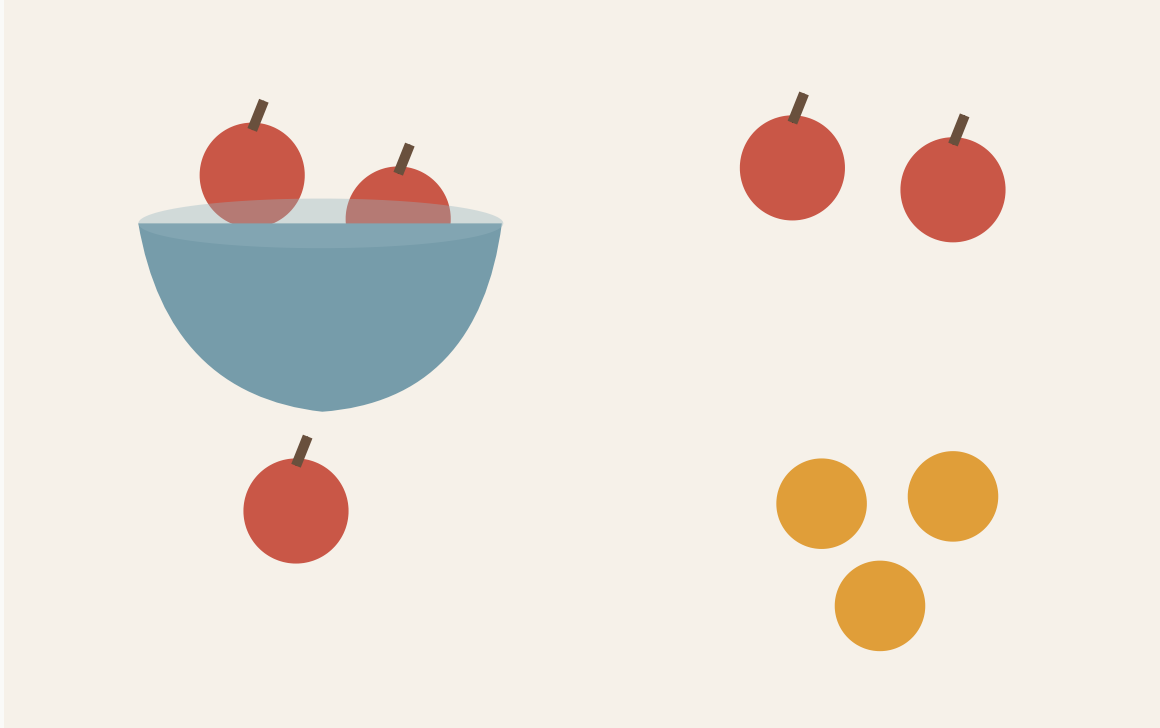
REFERENCE ANSWER

How many apples are outside the bowl?

---

Three.

# The question selects a subset



ARCHITECTURAL REQUIREMENT

How many apples are outside the bowl?

Three.

Represent multiple objects and the region that selects them.

# Read first, then calculate

## CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

---

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

---

Subtotal ₹1,000

GST ₹180

---

**TOTAL ₹1,180**

### QUESTION

What is the invoice ID?

What percentage of the subtotal is GST?

# Read first, then calculate

## CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

---

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

---

Subtotal ₹1,000

GST ₹180

---

**TOTAL ₹1,180**

## REFERENCE ANSWER

What is the invoice ID?

What percentage of the subtotal is GST?

---

ES667-042. GST is  $180 / 1000 \times 100 = 18\%$ .

# Read first, then calculate

## CAMPUS STATIONERY

Invoice: ES667-042

Date: 01 October 2026

---

Notebooks (4 × ₹200) ₹800

Pens (10 × ₹20) ₹200

---

Subtotal ₹1,000

GST ₹180

---

**TOTAL ₹1,180**

## ARCHITECTURAL REQUIREMENT

What is the invoice ID?

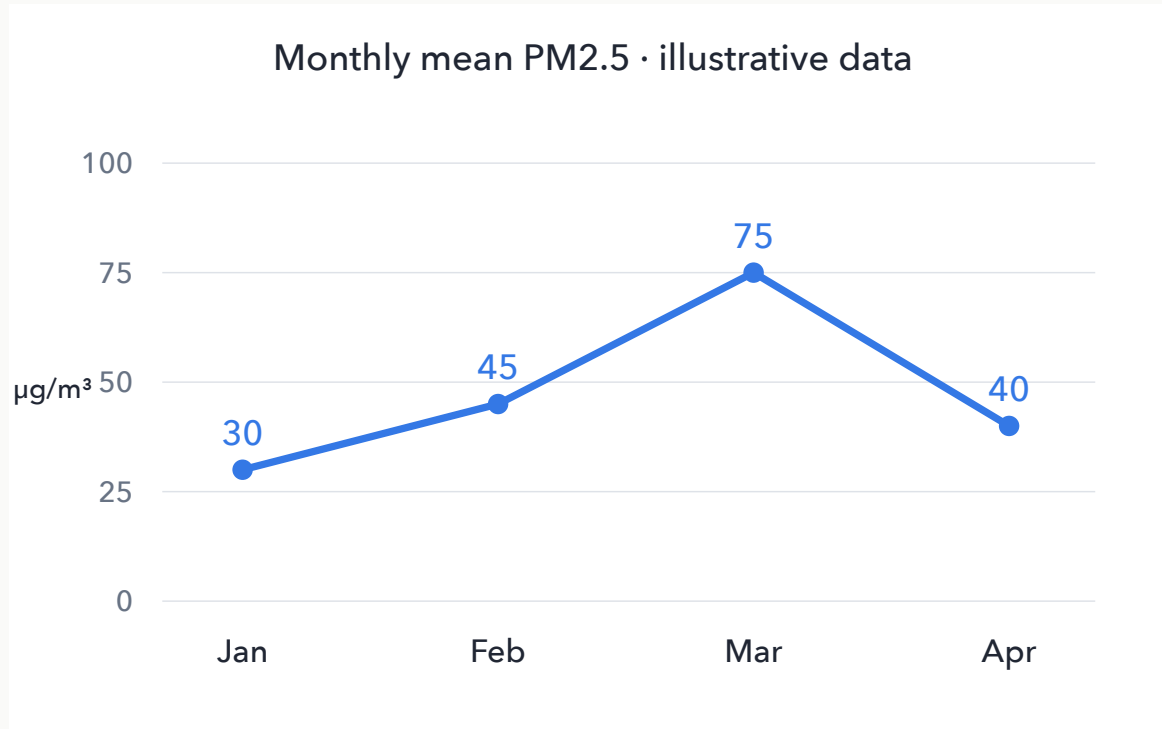
What percentage of the subtotal is GST?

---

ES667-042. GST is  $180 / 1000 \times 100 = 18\%$ .

| Preserve fine text, numbers and their roles.

# The labels change the answer



QUESTION

Which month is highest?

How much higher is March than January?

# The labels change the answer



REFERENCE ANSWER

Which month is highest?  
How much higher is March than January?

March. The difference is 45  $\mu\text{g}/\text{m}^3$ .

# The labels change the answer



ARCHITECTURAL REQUIREMENT

Which month is highest?

How much higher is March than January?

March. The difference is 45 μg/m³.

Bind chart labels to values and units.

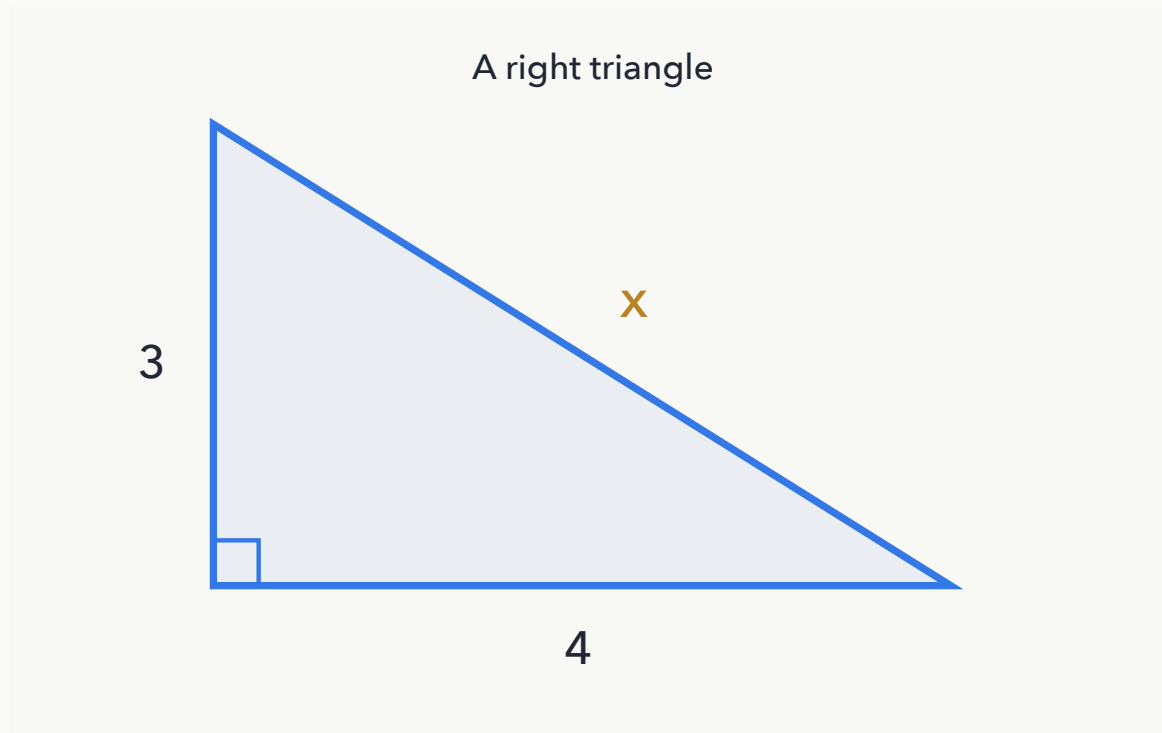
# A diagram supplies conditions



QUESTION

What is x?

# A diagram supplies conditions



REFERENCE ANSWER

What is x?

---

$x = 5$ , using the marked right angle and sides 3 and 4.

# A diagram supplies conditions



ARCHITECTURAL REQUIREMENT

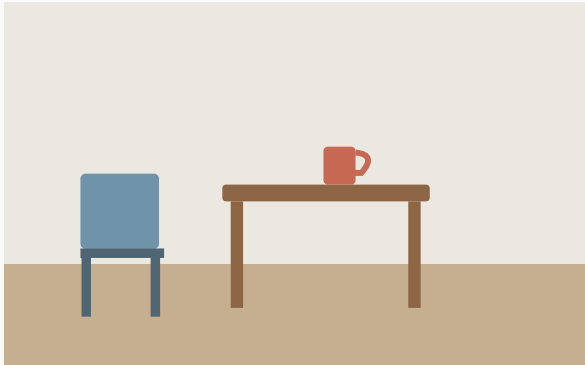
What is  $x$ ?

$x = 5$ , using the marked right angle and sides 3 and 4.

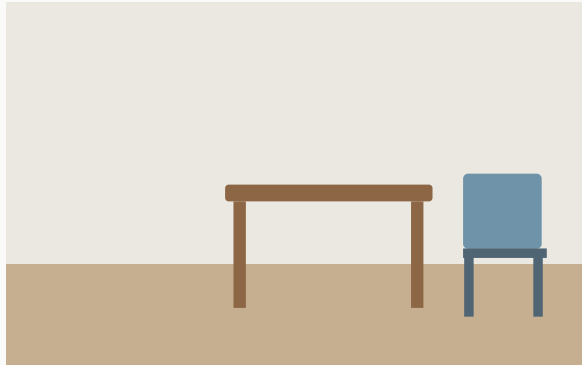
Combine diagram evidence with a mathematical rule.

# Compare presence and position

A · BEFORE



B · AFTER



QUESTION

What changed from A to B?

# Compare presence and position

A · BEFORE



B · AFTER



REFERENCE ANSWER

What changed from A to B?

---

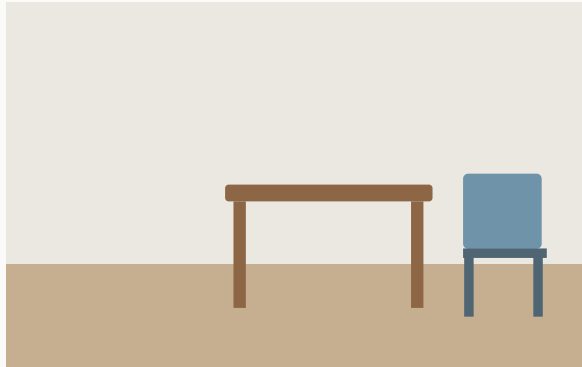
The chair moved to the right; the cup disappeared.

# Compare presence and position

A · BEFORE



B · AFTER



ARCHITECTURAL REQUIREMENT

What changed from A to B?

The chair moved to the right; the cup disappeared.

| Compare evidence across images.

# A screenshot is a spatial interface



QUESTION

Where should I click to disable notifications?

# A screenshot is a spatial interface

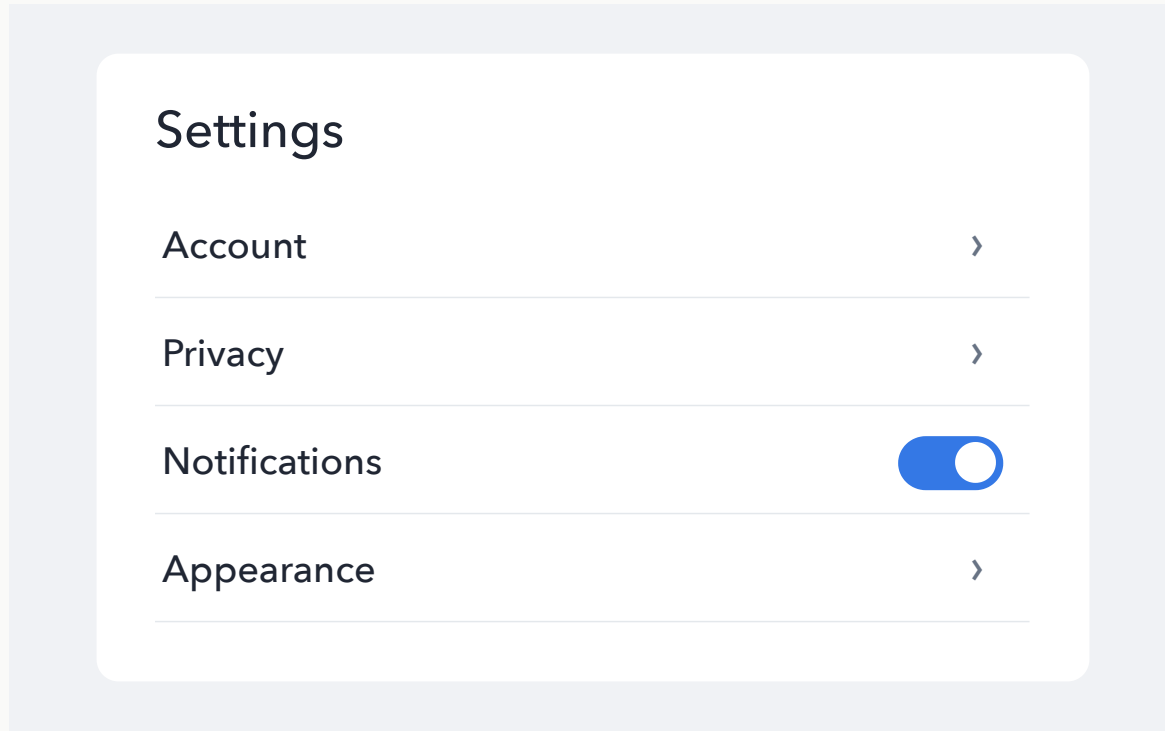


REFERENCE ANSWER

Where should I click to disable notifications?

The blue toggle on the Notifications row.

# A screenshot is a spatial interface



## ARCHITECTURAL REQUIREMENT

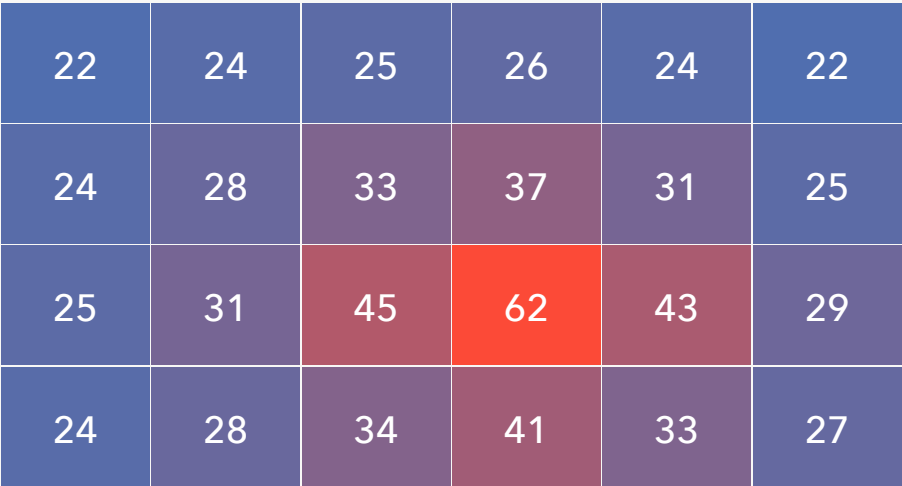
Where should I click to disable notifications?

The blue toggle on the Notifications row.

Locate a control relative to its label.

# The image can be a scientific figure

Thermal map of a test plate (°C)



Synthetic measurements; no physical experiment claimed.

QUESTION

What is the hottest measured region?

# The image can be a scientific figure

Thermal map of a test plate (°C)

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 22 | 24 | 25 | 26 | 24 | 22 |
| 24 | 28 | 33 | 37 | 31 | 25 |
| 25 | 31 | 45 | 62 | 43 | 29 |
| 24 | 28 | 34 | 41 | 33 | 27 |

Synthetic measurements; no physical experiment claimed.

REFERENCE ANSWER

What is the hottest measured region?

Row 3, column 4: 62 °C.

# The image can be a scientific figure

Thermal map of a test plate (°C)

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 22 | 24 | 25 | 26 | 24 | 22 |
| 24 | 28 | 33 | 37 | 31 | 25 |
| 25 | 31 | 45 | 62 | 43 | 29 |
| 24 | 28 | 34 | 41 | 33 | 27 |

Synthetic measurements; no physical experiment claimed.

ARCHITECTURAL REQUIREMENT

What is the hottest measured region?

Row 3, column 4: 62 °C.

| Read a scale and local measurements.

# The question can be wrong



QUESTION

What color is the bicycle?

# The question can be wrong



REFERENCE ANSWER

What color is the bicycle?

---

No bicycle is visible.

# The question can be wrong



ARCHITECTURAL REQUIREMENT

What color is the bicycle?

No bicycle is visible.

Let visual evidence override the wording of the question.

# What must survive the visual representation?

| QUESTION FAMILY      | INFORMATION NEEDED                        |
|----------------------|-------------------------------------------|
| Counting / spatial   | Multiple objects and their arrangement    |
| OCR / charts         | Fine detail, labels, values and units     |
| UI / multiple images | Locations and correspondence across views |
| False premise        | Evidence that can contradict the prompt   |

| Would one global CLIP vector always be enough?

# Compare two small vision-language models



| A reference answer is not a model prediction.

Predict → compare → inspect both answers.

[Open the VLM lab ↗](#)

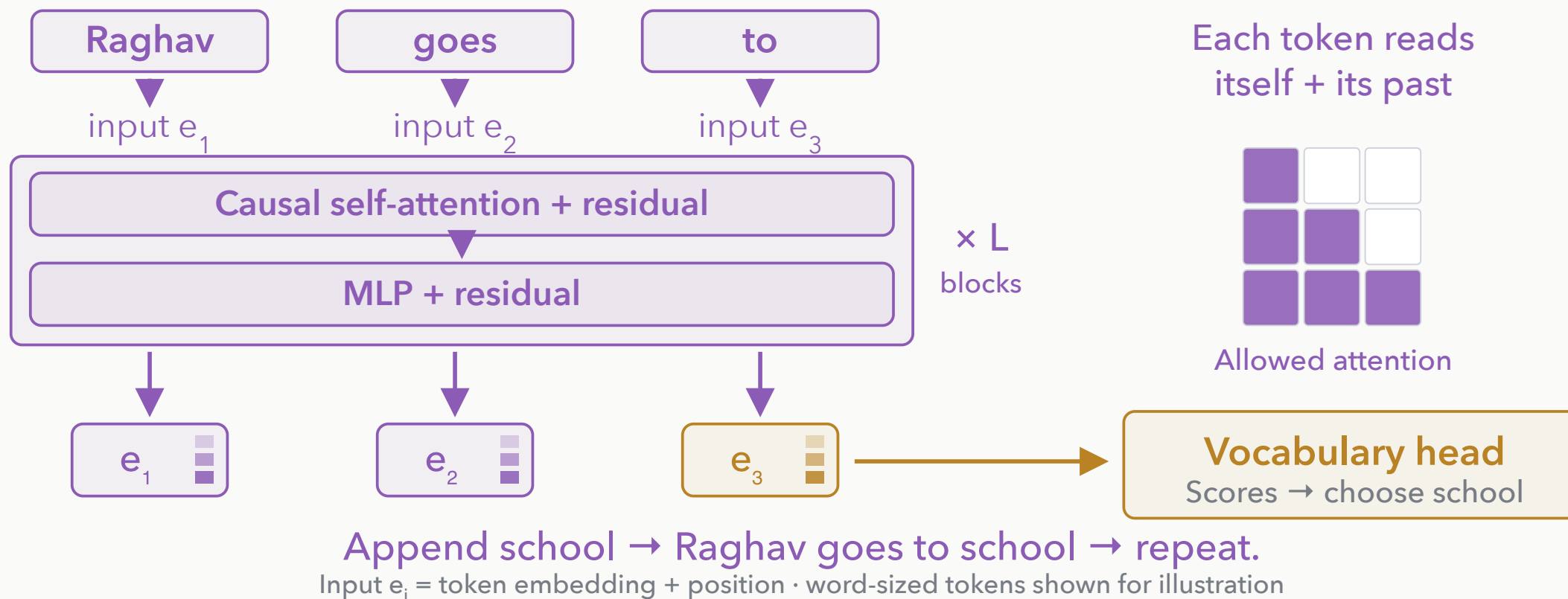
256M and 500M · same image and question.  
Live or recorded answers; image-swap experiment.

## SECTION 02

# From matching to generation

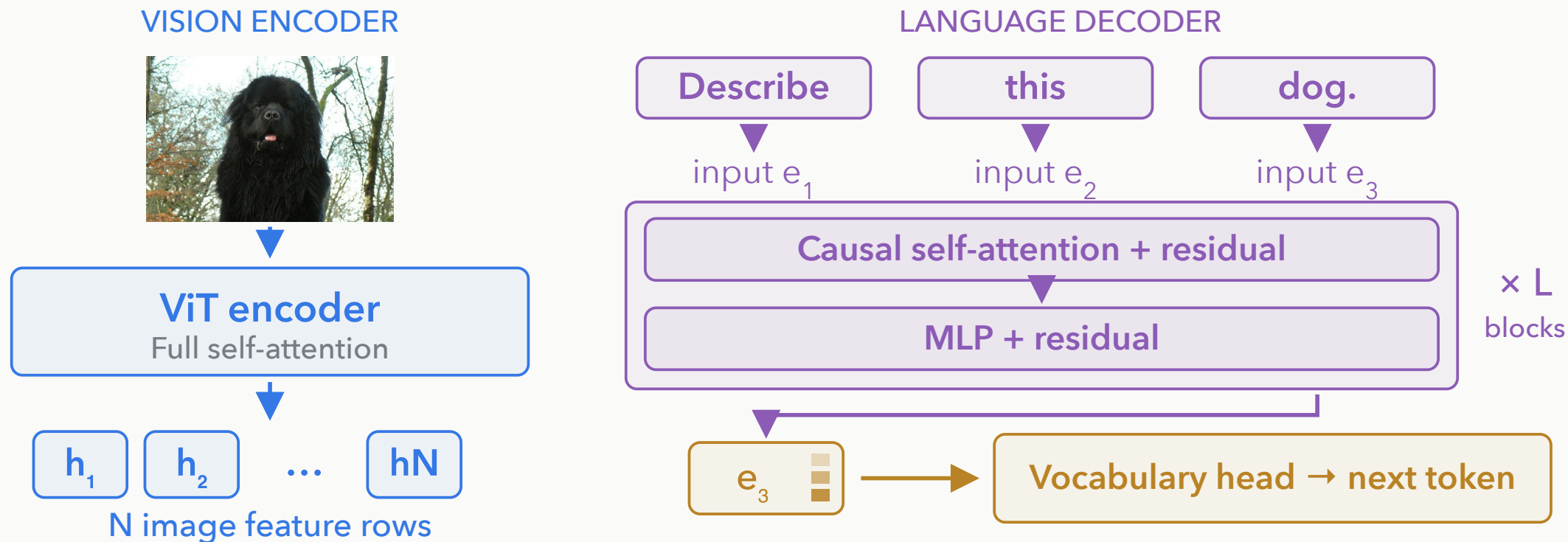
We have an image encoder. We also know a language decoder.

# We already know this decoder from Beyond Attention



Read the last updated state  $\rightarrow$  score the vocabulary  $\rightarrow$  append a token  $\rightarrow$  repeat.

# Keep this decoder; give it access to the image



How can these image rows become context for this decoder?

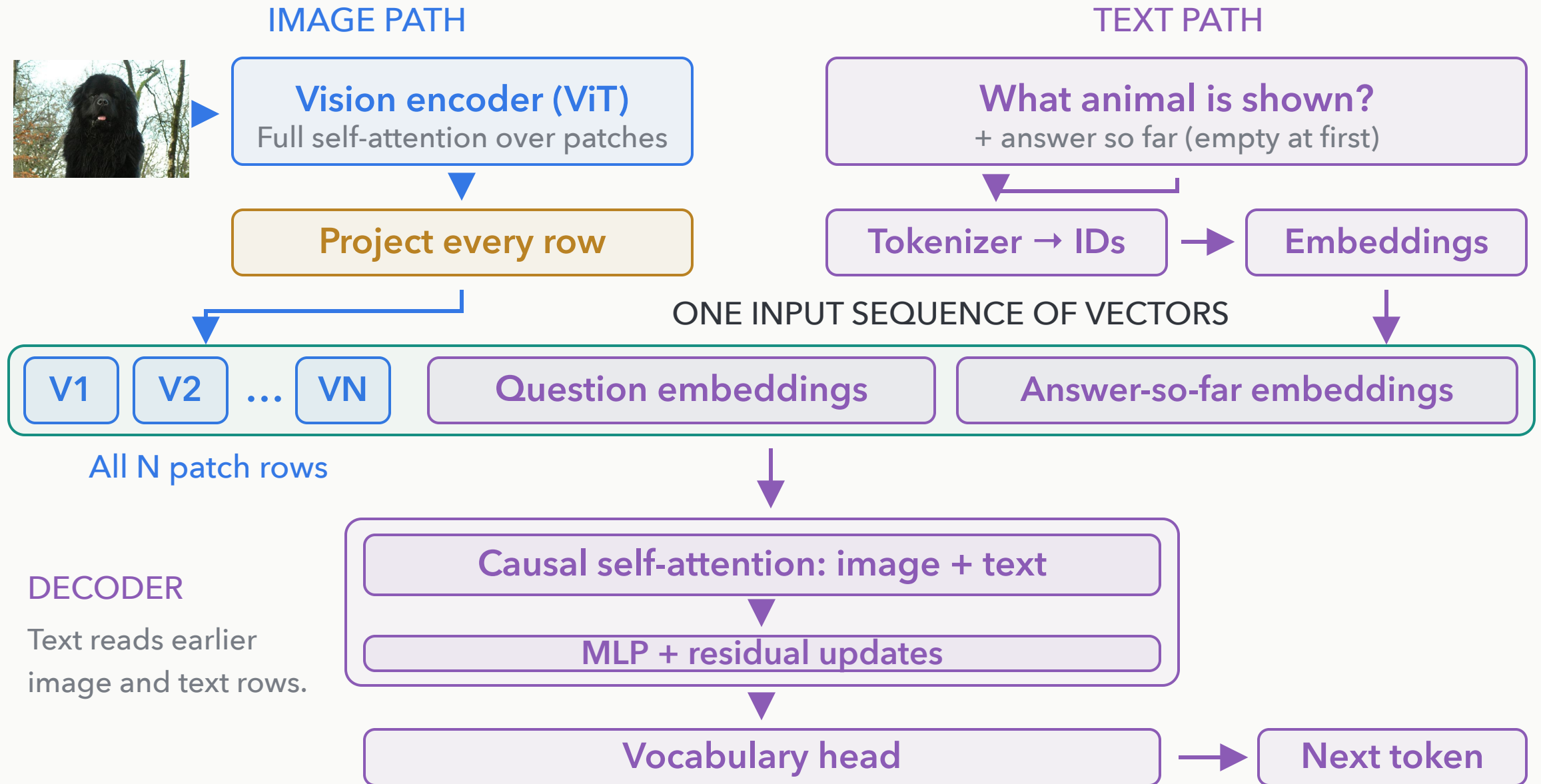
| We have both components. Next, choose how the decoder accesses the visual rows.

### SECTION 03

# How can vision enter a language model?

First, see the three ideas. Then build each connection.

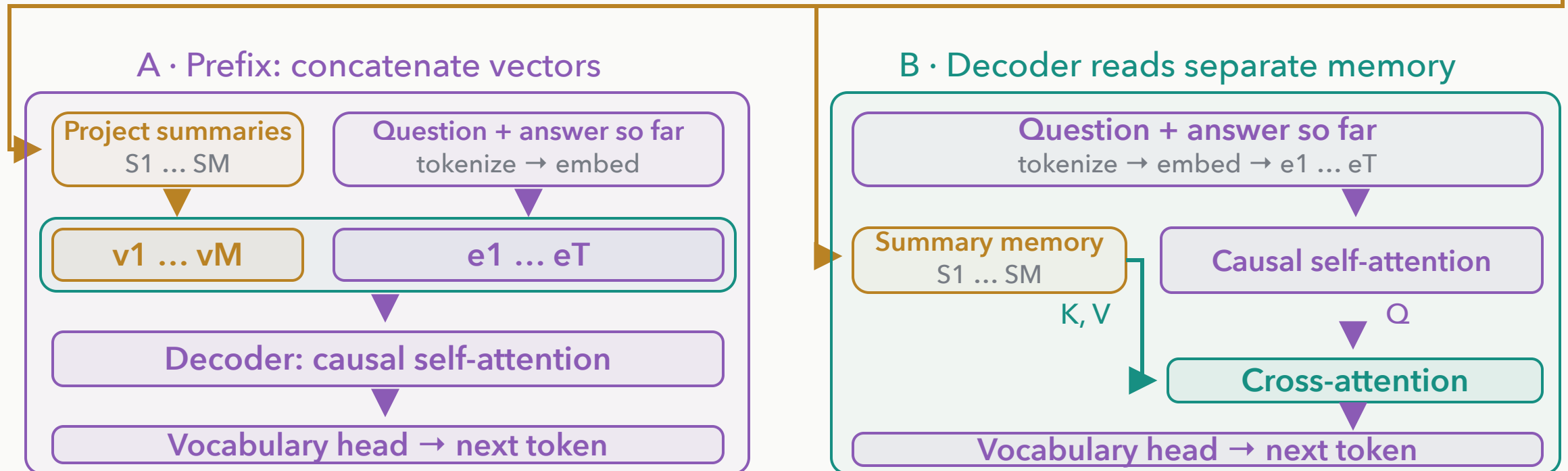
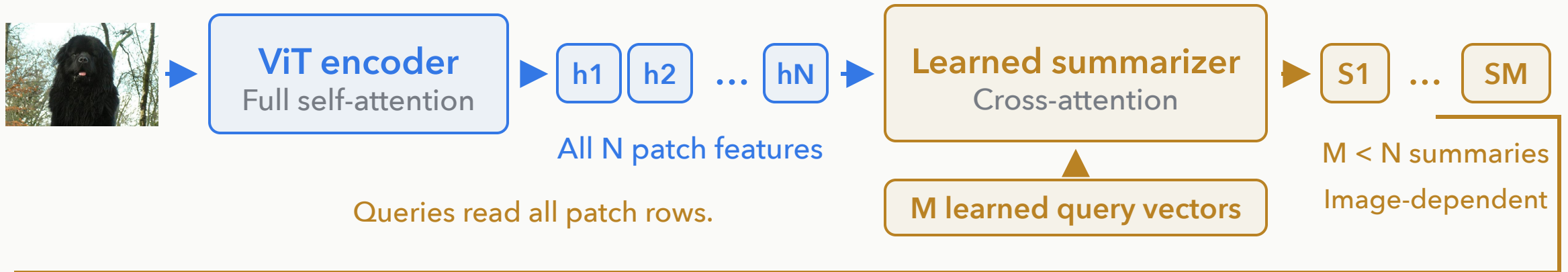
# Idea 1 • prepend the image as visual tokens



## Idea 2 • let the decoder look at a visual memory



# Idea 3 · summarize the image, then use either connection



# The map: two access routes, one compression choice

| IDEA                | HIGH-LEVEL DECISION             | THEN WE WILL EXPLAIN...                        |
|---------------------|---------------------------------|------------------------------------------------|
| 1 · Visual prefix   | Put visual vectors before text. | How to make them fit the decoder input.        |
| 2 · Separate memory | Let language read the image.    | How a text state retrieves visual information. |
| 3 · Shorter summary | Use fewer visual vectors.       | How learned readers make the summaries.        |

**Ideas 1 and 2 choose the access route. Idea 3 can be used with either.**

# Idea 1 in detail · build the visual prefix



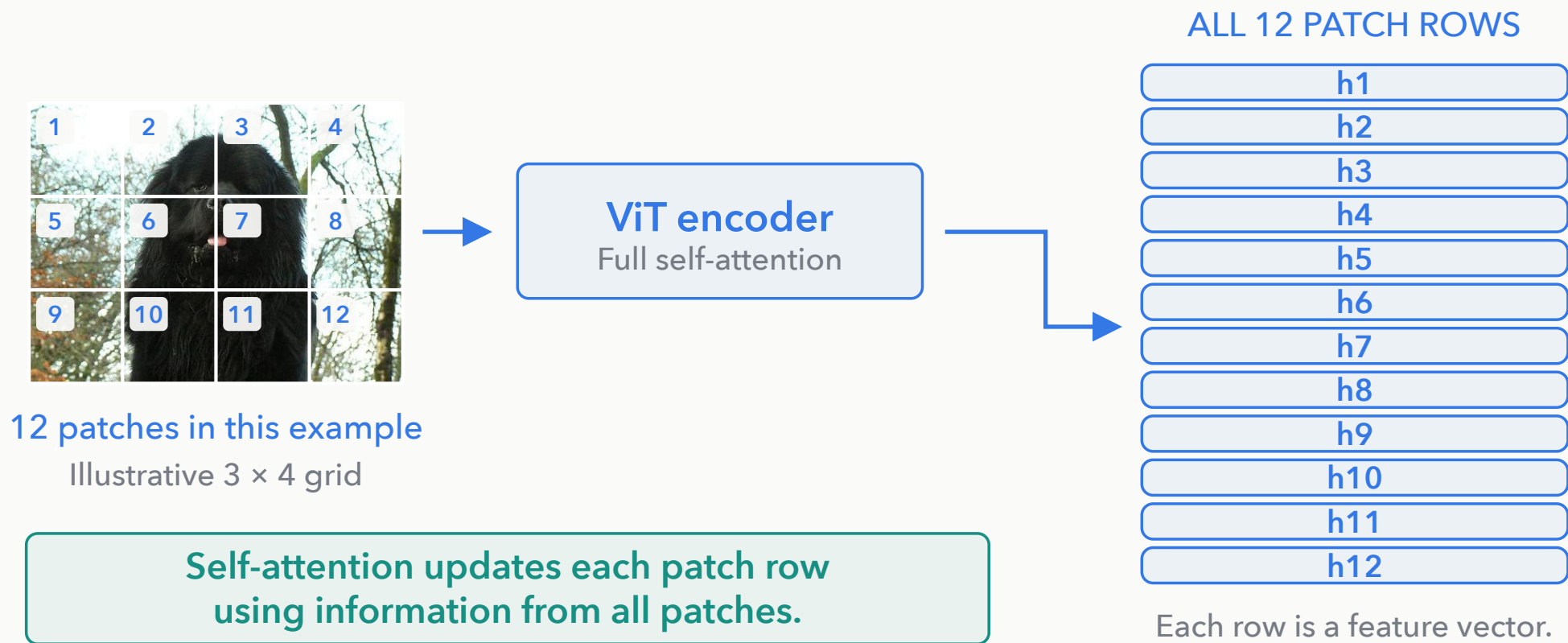
One sequence; the same causal self-attention can use both kinds of context.

# What must we build to make this work?

| JOB                               | WHY IT IS NEEDED                                         |
|-----------------------------------|----------------------------------------------------------|
| 1. Keep image feature vectors     | The decoder needs access to visual evidence.             |
| 2. Match the decoder input width  | All input rows must have the same number of coordinates. |
| 3. Place them before the question | Causal attention can read earlier context.               |

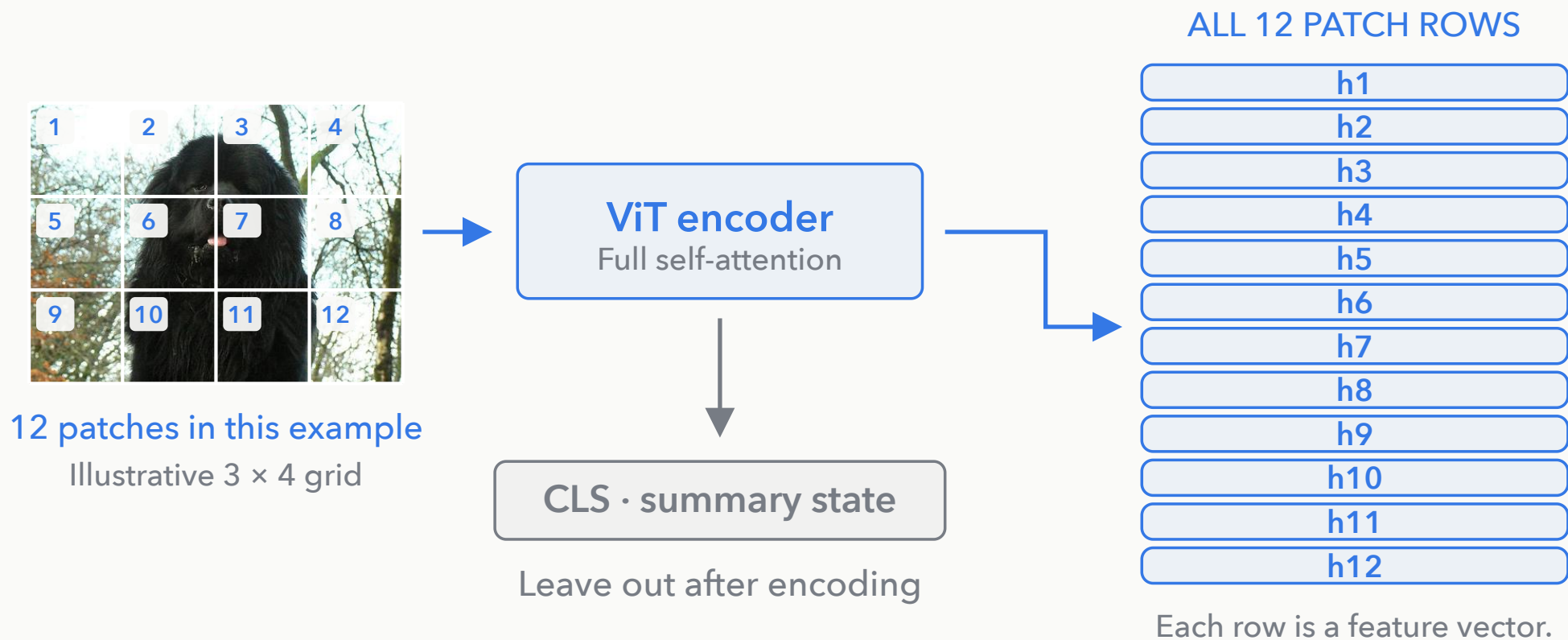
**| We already have the decoder. Now build its visual input.**

## Recall ViT: every patch already has an output row



| Patch 1 has row h1, patch 2 has row h2, and so on through patch 12.

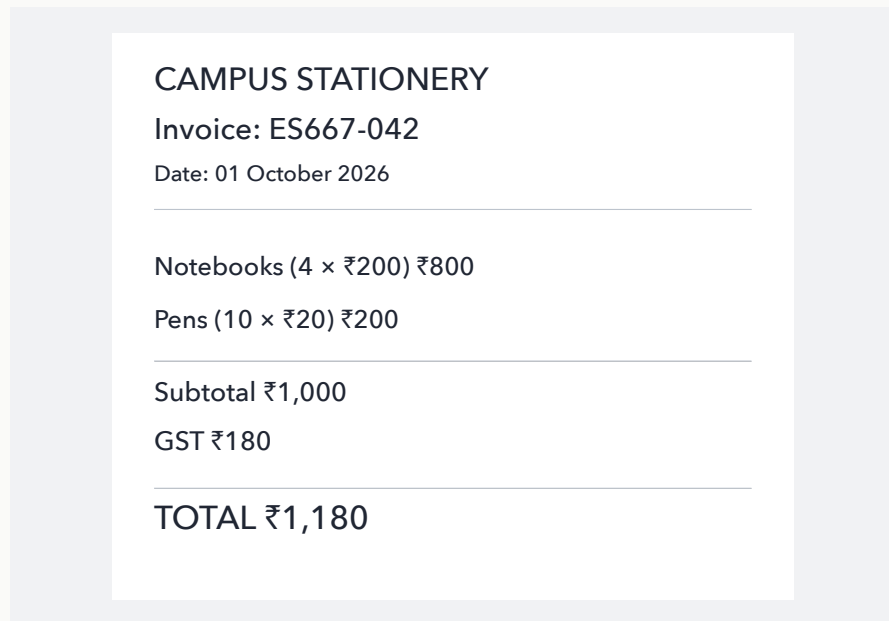
# ViT outputs a CLS state and one row per patch



Forward all 12 patch rows to the projector; leave out the CLS output.

# Pass the patch rows onward; leave out CLS

The question needs image details



Invoice ID? GST percentage?

SELECT FROM THE ViT OUTPUTS

CLS output

Leave out after encoding

12 patch rows

h1  
h2  
h3  
h4  
h5  
h6  
h7  
h8  
h9  
h10  
h11  
h12

KEEP EVERY PATCH ROW

Project  
each row

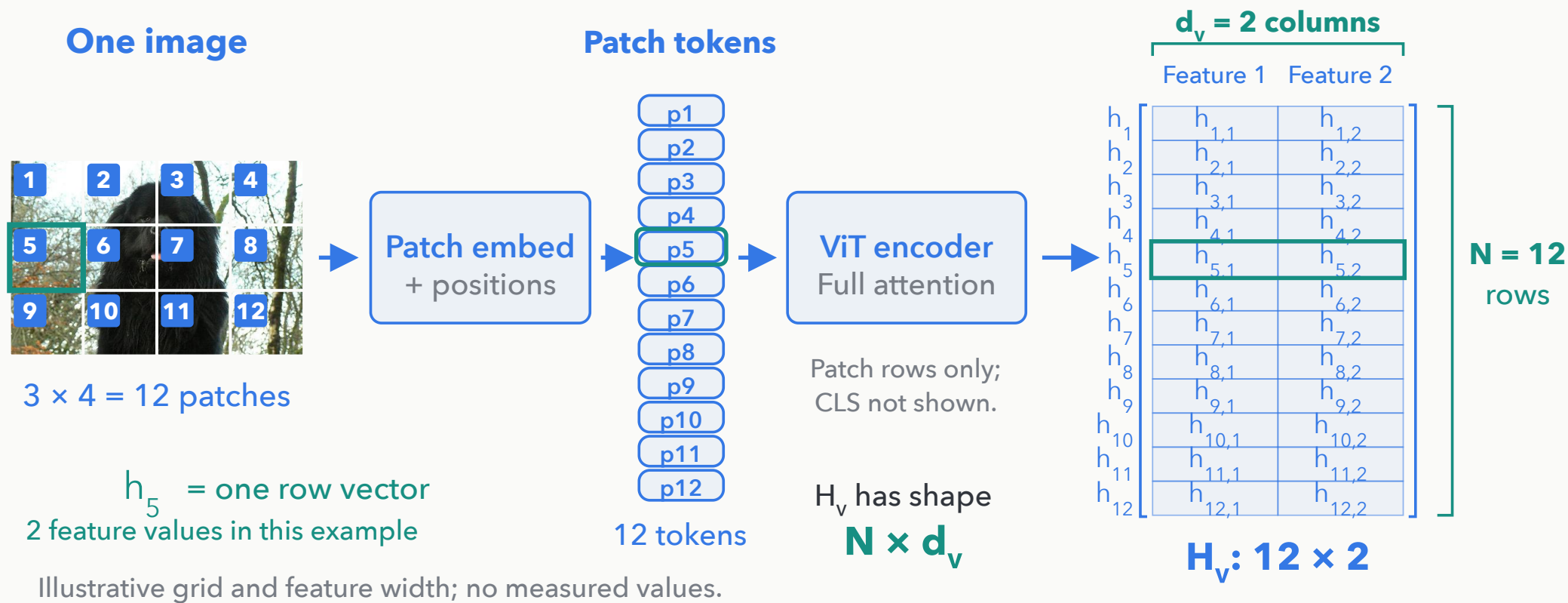
Visual prefix  
12 vectors

Combine with text embeddings  
for the language decoder.

Illustration: 12 patch rows; CLS left out.

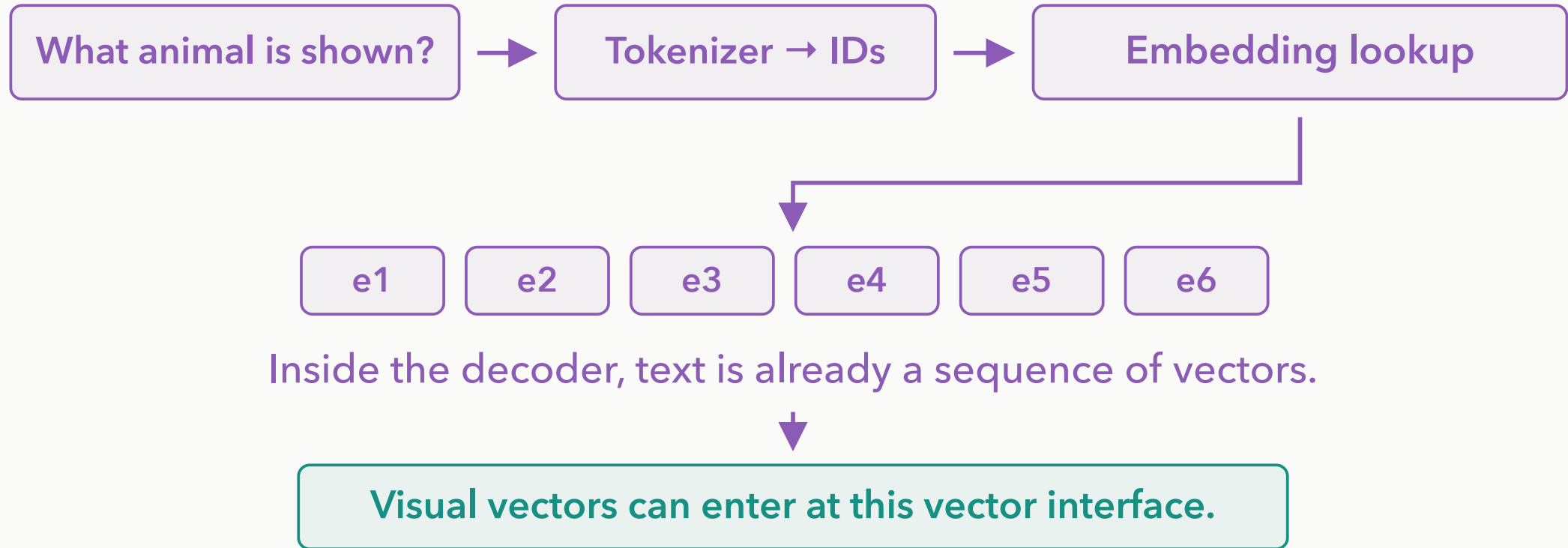
This design sends all N patch rows to the projector and omits the CLS output.

# A visual state is one row of the feature matrix



One patch position  $\rightarrow$  one output row. Each row has  $d_v$  feature values.

## Recall: the decoder already consumes vectors



| Text IDs become embeddings before reaching the Transformer blocks.

# The image rows have 2 coordinates; text rows have 3

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

Can these rows enter the same decoder?

$h_1$

|   |   |
|---|---|
| 2 | 1 |
|---|---|

≠ width

$e_1$

|   |   |   |
|---|---|---|
| 1 | 0 | 2 |
|---|---|---|

2 coordinates per image row

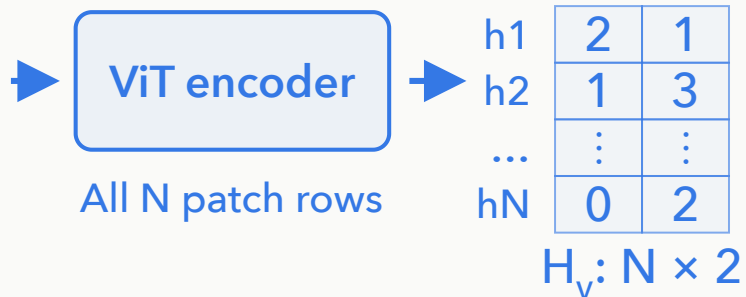
3 coordinates per text row

We need to turn each image row into a 3-coordinate vector.

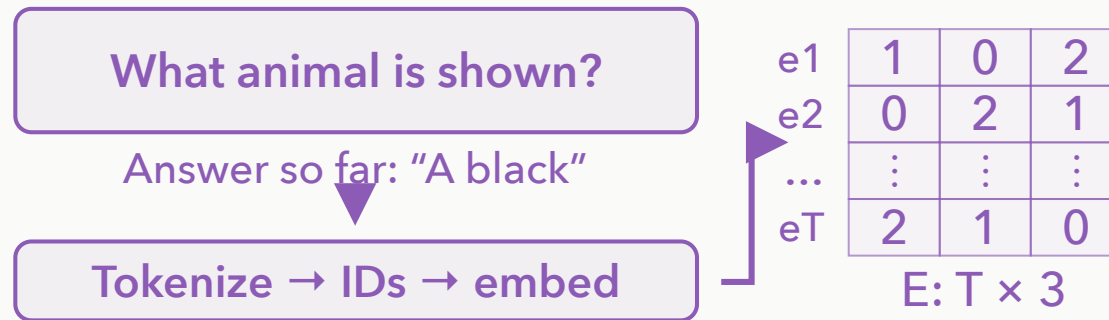
To form one input sequence, every vector must have the decoder's input width.

# Add a projector to the image path

## IMAGE → FEATURE VECTORS

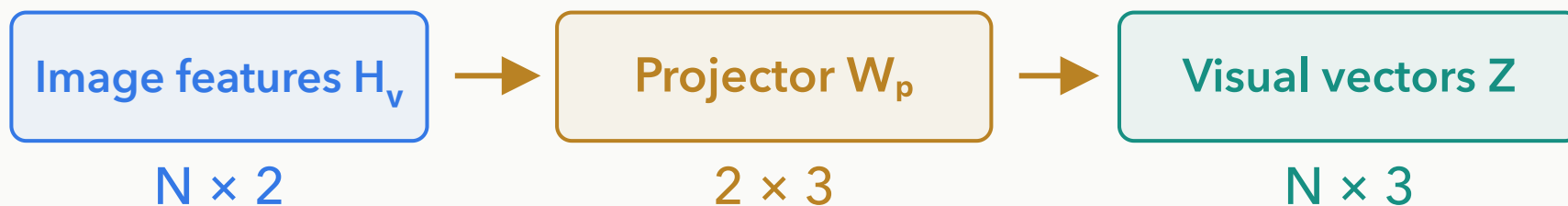


## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

## Insert a shared projector on the image path



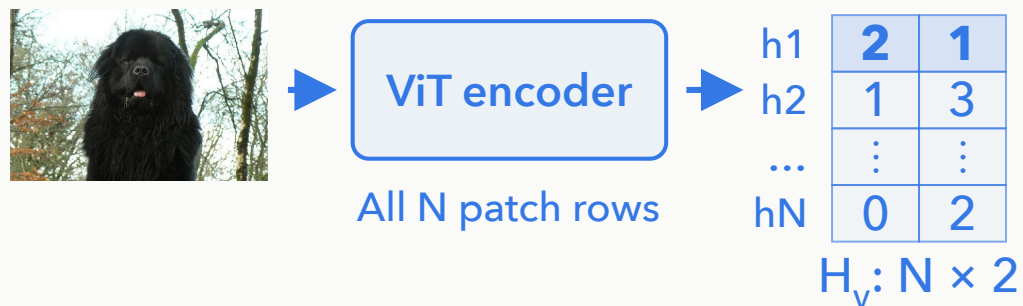
Text stays  $E$   
 $T \times 3$

$Z = H_V W_p$  · Keep N rows. Change 2 coordinates to 3.

The projector changes  $N \times 2$  image features into  $N \times 3$  visual vectors.

# Zoom into one row from the image

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

## Zoom into $h_1$



$h_1$  is one of the image rows above. Its two coordinates are 2 and 1.

Take one 2-coordinate row from the image feature sequence.

# Use a $2 \times 3$ matrix to change the width

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

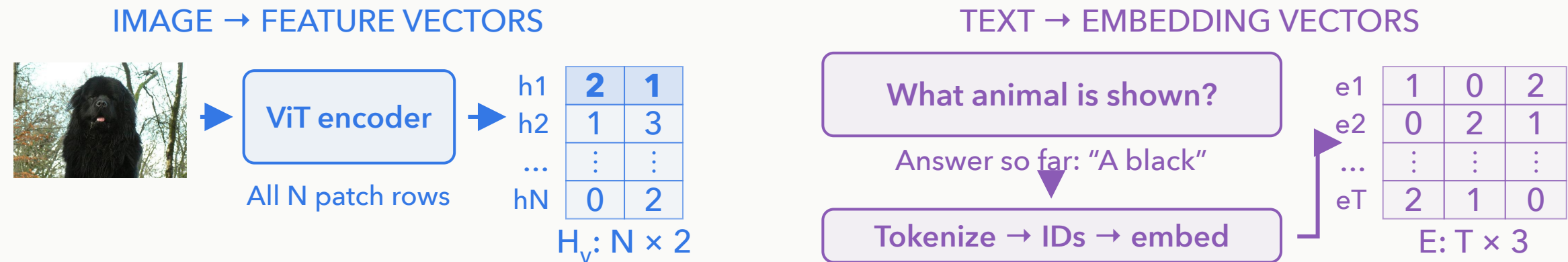
## Zoom into h1

$$\begin{array}{c} \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\ 1 \times 2 \end{array} \times \begin{array}{c} W_p \\ \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\ 2 \times 3 \end{array} = \begin{array}{c} z1 \\ \begin{array}{|c|c|c|} \hline ? & ? & ? \\ \hline \end{array} \\ 1 \times 3: \text{ same width as } e1 \end{array}$$

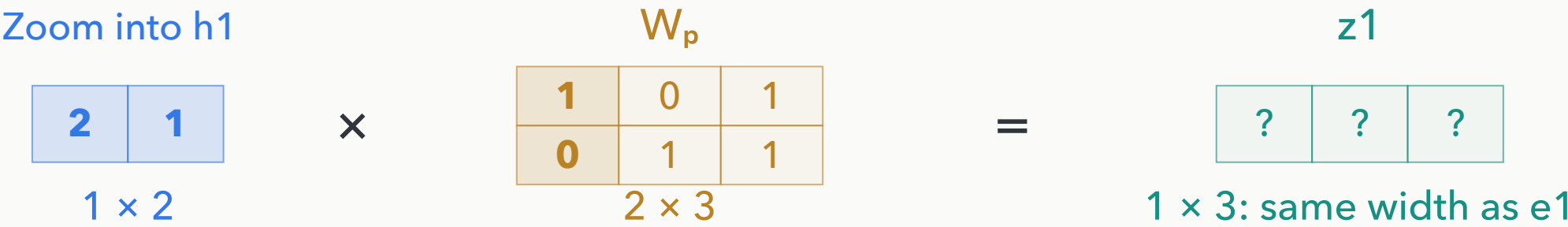
Each of the three columns of  $W_p$  will make one output coordinate.

Each column makes one of the 3 decoder-input coordinates.

# The first column makes the first coordinate



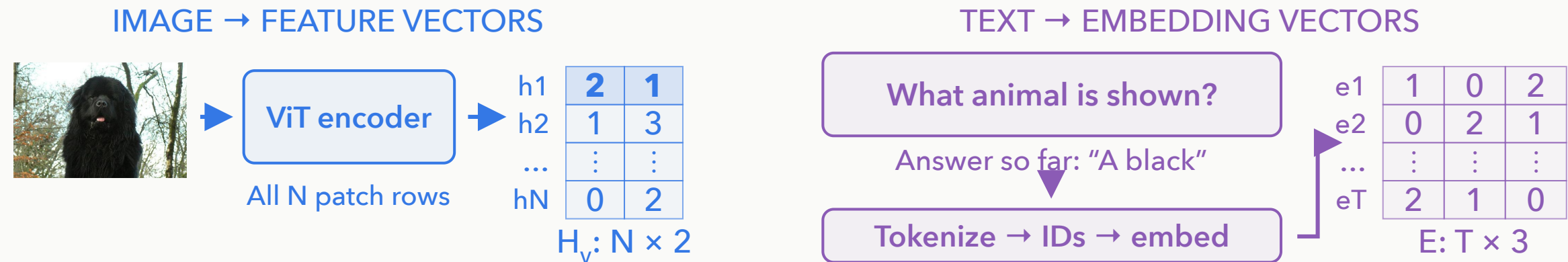
Toy widths and values; not measured from this image or text. Middle rows omitted.



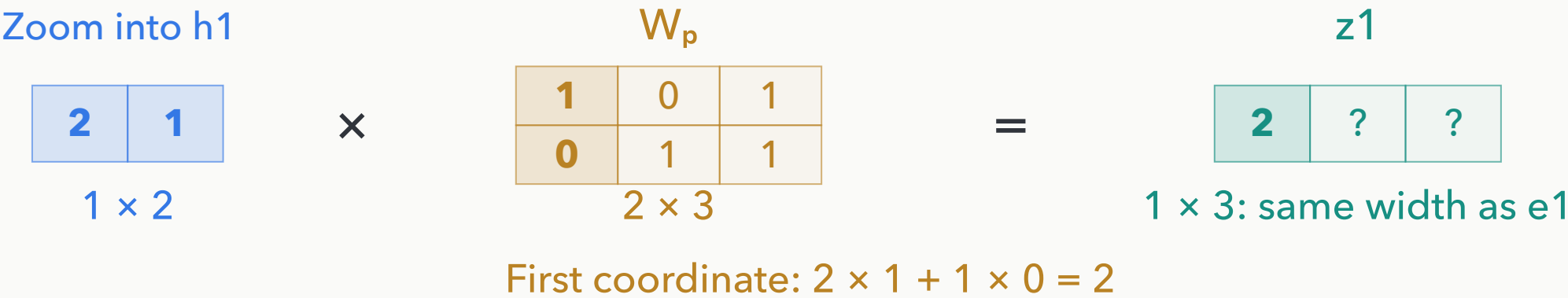
First column: how much of 2, and how much of 1, should we use?

| Which input coordinates does this column combine?

# The first output coordinate is 2



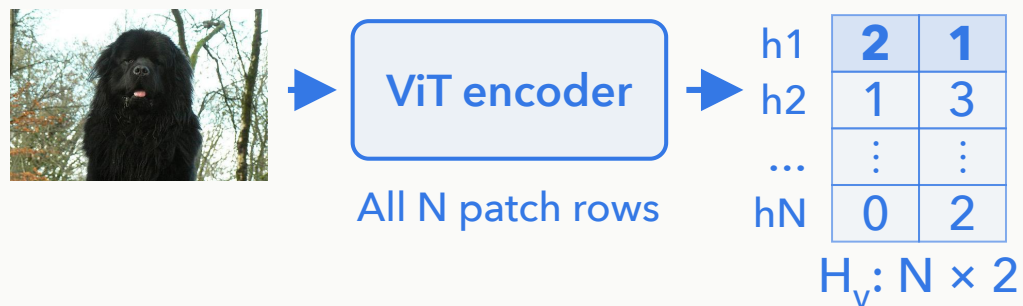
Toy widths and values; not measured from this image or text. Middle rows omitted.



The first output coordinate copies the first input in this toy map.

# The second column reuses the same image row

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

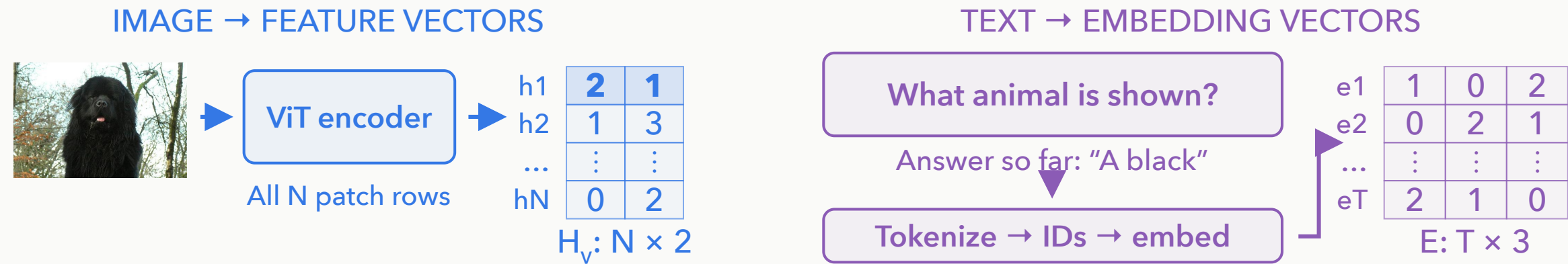
## Zoom into h1

$$\begin{array}{c}
 \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\
 1 \times 2
 \end{array}
 \times
 \begin{array}{c}
 W_p \\
 \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\
 2 \times 3
 \end{array}
 =
 \begin{array}{c}
 z1 \\
 \begin{array}{|c|c|c|} \hline 2 & ? & ? \\ \hline \end{array} \\
 1 \times 3: \text{ same width as } e1
 \end{array}$$

Second column: use the same input [2, 1] again.

Now use the same input row with the second column.

# The second output coordinate is 1



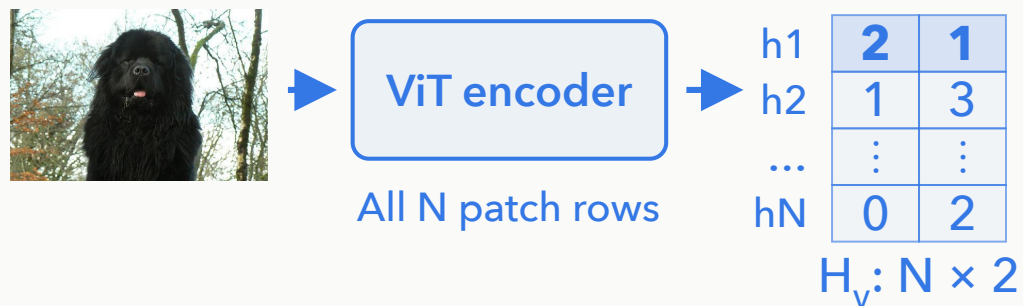
Toy widths and values; not measured from this image or text. Middle rows omitted.



The second output coordinate copies the second input in this toy map.

# The third column combines both input coordinates

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

## Zoom into $h_1$

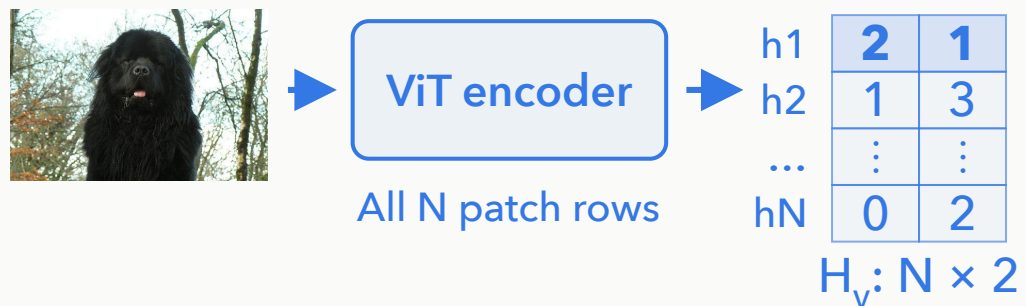
$$\begin{array}{c} \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\ 1 \times 2 \end{array} \times \begin{array}{c} W_p \\ \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\ 2 \times 3 \end{array} = \begin{array}{c} z_1 \\ \begin{array}{|c|c|c|} \hline 2 & 1 & 3 \\ \hline \end{array} \\ 1 \times 3: \text{ same width as } e_1 \end{array}$$

$$\text{Third coordinate: } 2 \times 1 + 1 \times 1 = 3$$

The third output coordinate adds the two inputs in this toy map.

# Now this image vector has the required width

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

## Zoom into $h_1$

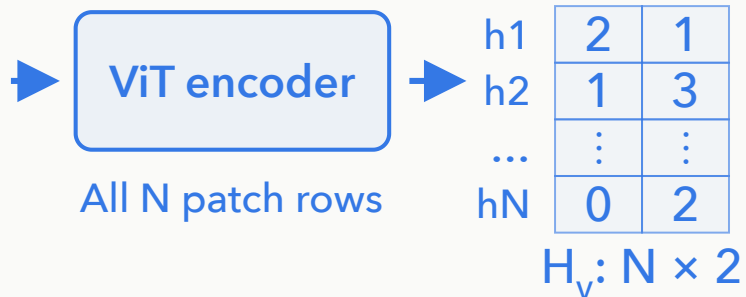
$$\begin{array}{c}
 \begin{array}{|c|c|} \hline 2 & 1 \\ \hline \end{array} \\
 1 \times 2
 \end{array}
 \times
 \begin{array}{c}
 W_p \\
 \begin{array}{|c|c|c|} \hline 1 & 0 & 1 \\ \hline 0 & 1 & 1 \\ \hline \end{array} \\
 2 \times 3
 \end{array}
 =
 \begin{array}{c}
 z_1 \\
 \begin{array}{|c|c|c|} \hline 2 & 1 & 3 \\ \hline \end{array} \\
 1 \times 3: \text{ same width as } e_1
 \end{array}$$

The image row  $[2, 1]$  has become  $[2, 1, 3]$ . The text rows stay as they are.

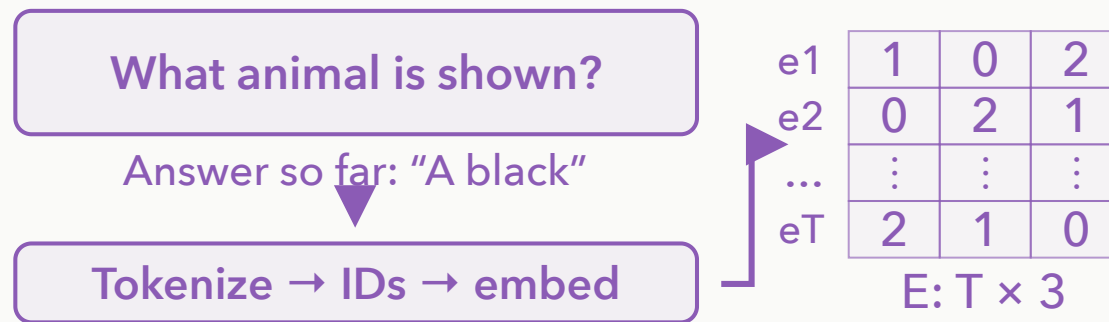
| The row  $[2, 1, 3]$  fits beside the 3-coordinate text embeddings.

# Use the same matrix for all N image rows

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

## Every image row

h1 

|   |   |
|---|---|
| 2 | 1 |
|---|---|

  
 h2 

|   |   |
|---|---|
| 1 | 3 |
|---|---|

  
 ... 

|   |   |
|---|---|
| ⋮ | ⋮ |
|---|---|

  
 hN 

|   |   |
|---|---|
| 0 | 2 |
|---|---|

$N \times 2$

## Same $W_p$

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
|---|---|---|

|   |   |   |
|---|---|---|
| 0 | 1 | 1 |
|---|---|---|

$2 \times 3$

=

## All N projected rows

z1 

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
|---|---|---|

  
 z2 

|   |   |   |
|---|---|---|
| 1 | 3 | 4 |
|---|---|---|

  
 ... 

|   |   |   |
|---|---|---|
| ⋮ | ⋮ | ⋮ |
|---|---|---|

  
 zN 

|   |   |   |
|---|---|---|
| 0 | 2 | 2 |
|---|---|---|

$N \times 3$

Width now matches E

N is unchanged.

N rows stay N rows; every projected row now has 3 coordinates.

# Check: did the projector create more visual tokens?

## Same image grid



$N = 3 \times 4 = 12$  patch rows

Image features  $H_v$

$12 \times 2$

## From our toy ViT

Example row  $h_1$

|   |   |
|---|---|
| 2 | 1 |
|---|---|

$d_v = 2$  features per row

Illustrative width, as before

## From the text embeddings

Example row  $e_1$

|   |   |   |
|---|---|---|
| 1 | 0 | 2 |
|---|---|---|

$d_l = 3$  features per row

Illustrative width, as before

Shared projector  $W_p$

$2 \times 3$

Projected features  $Z$

$? \times 3$

**How many rows will  $Z$  have?**

Predict the output shape before advancing.

# Still 12 rows; each row now has 3 features

## Same image grid



$N = 3 \times 4 = 12$  patch rows

Image features  $H_v$

$12 \times 2$

## From our toy ViT

Example row  $h_1$

|   |   |
|---|---|
| 2 | 1 |
|---|---|

$d_v = 2$  features per row

Illustrative width, as before

## From the text embeddings

Example row  $e_1$

|   |   |   |
|---|---|---|
| 1 | 0 | 2 |
|---|---|---|

$d_l = 3$  features per row

Illustrative width, as before

Shared projector  $W_p$

$2 \times 3$

Projected features  $Z$

$12 \times 3$

$\times$

$=$

**12 rows stay 12 rows. Each row changes from 2 features to 3.**

| The projector changes each row's features and keeps every patch position.

# Now the two sequences fit one decoder input

## IMAGE → FEATURE VECTORS



## TEXT → EMBEDDING VECTORS



Toy widths and values; not measured from this image or text. Middle rows omitted.

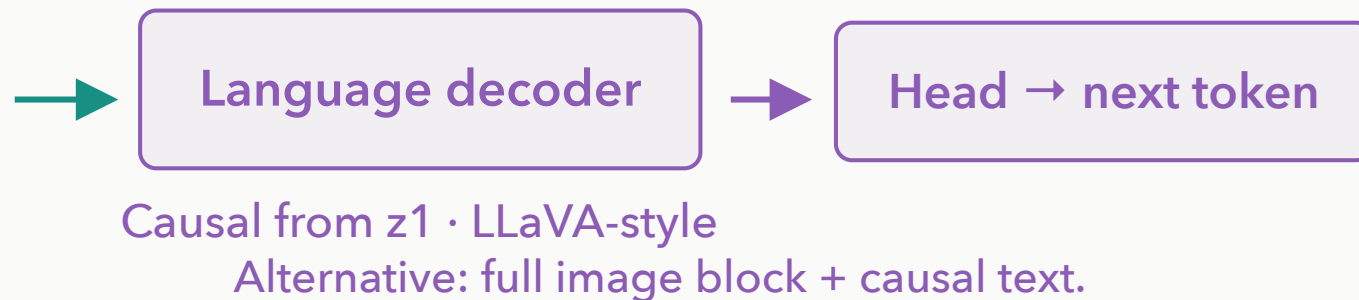
Image → shared  $W_p \rightarrow Z$

N projected rows

+ T text rows

$(N + T) \times 3$

|         |          |          |          |
|---------|----------|----------|----------|
| $z_1$   | 2        | 1        | 3        |
| $\dots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $z_N$   | 0        | 2        | 2        |
| $e_1$   | 1        | 0        | 2        |
| $\dots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $e_T$   | 2        | 1        | 0        |

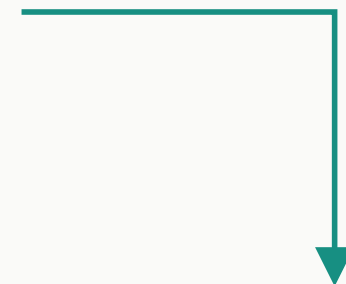
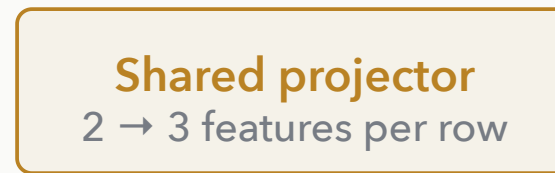
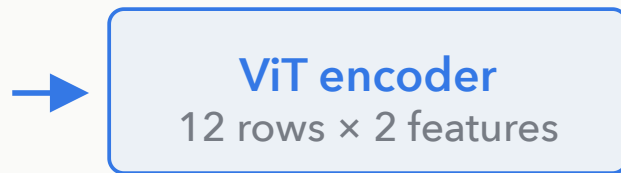


Our LLaVA-style example uses causal attention from the first image position.

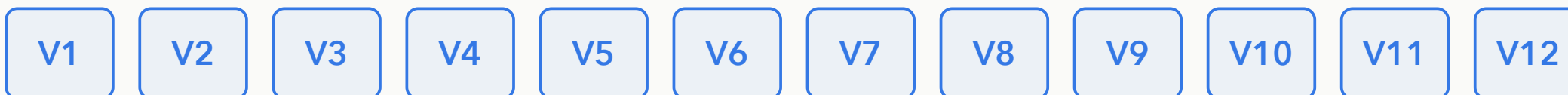
## Start with all the projected image vectors



Same 12 patches



### THE VISUAL PREFIX · ALL 12 ROWS



Each  $V_i$  is a vector with 3 features in our running example.

Next, put the formatted conversation after these vectors.

| Keep all 12 patch rows; each row has the decoder's input width.

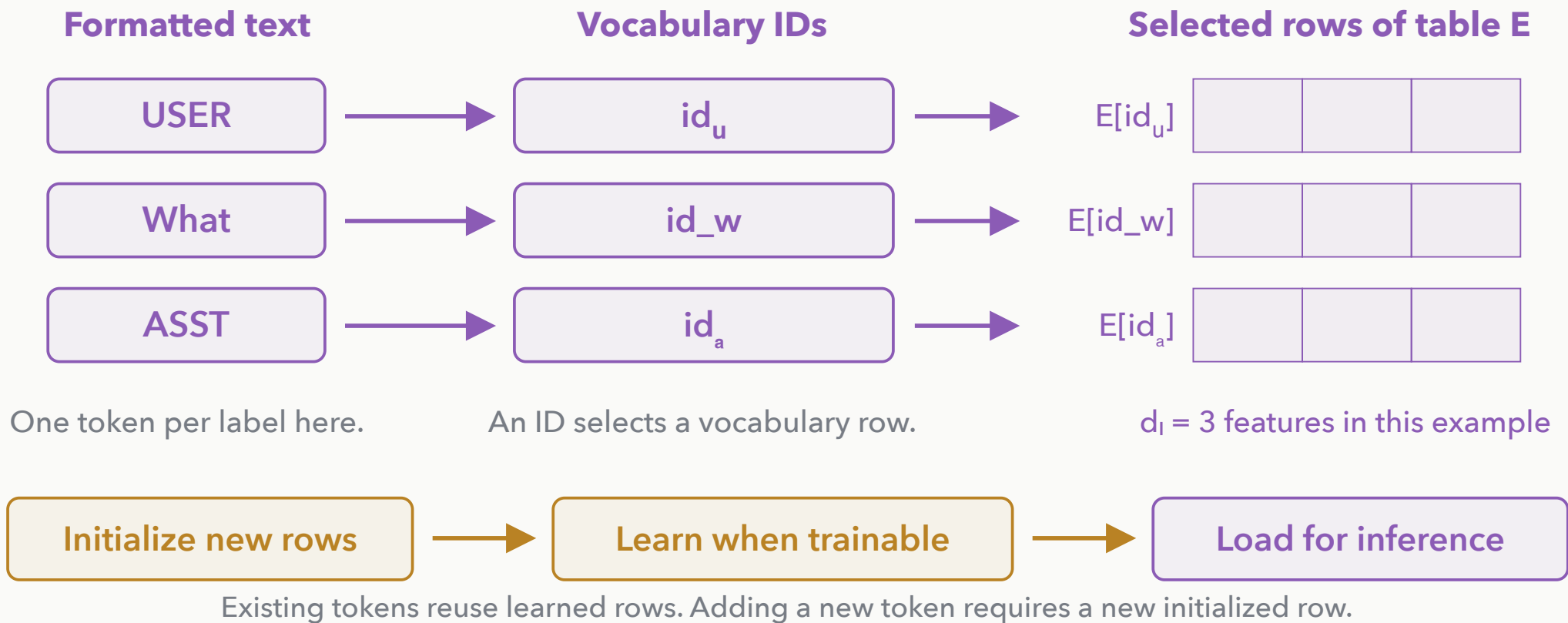
## USER and ASST mark who is speaking



USER / ASST are teaching labels; a real role marker can span several tokens.

| The template supplies the role markers; generation continues after ASST.

# Role tokens use the same embedding table



| A special role token is a vocabulary entry with its own learned embedding row.

# Finish the question; start the assistant's answer

## ONE SEQUENCE OF VECTORS



Illustration: 12 visual rows + 1 USER + 4 question tokens + 1 ASST.

0 ... 11

12

13 14 15 16

17

Position

USER is at 12; ASST is at 17. Positions do not restart at a role boundary.



Position-aware causal self-attention

LM head →

It

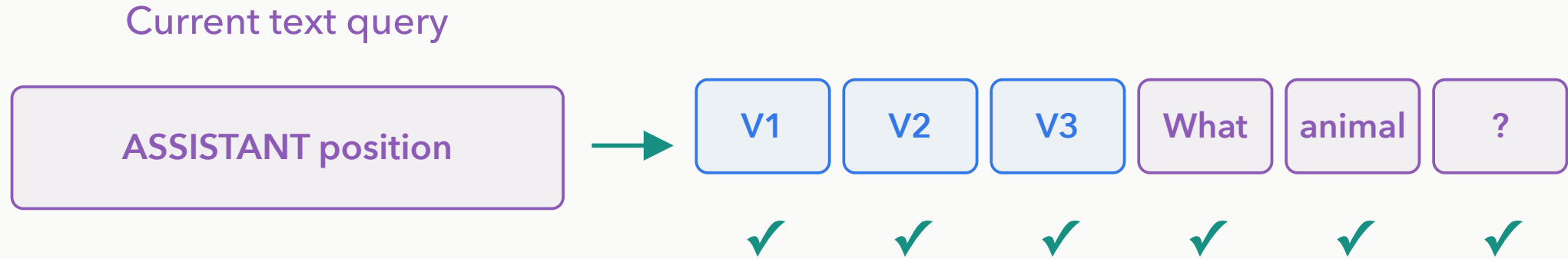
Additive scheme:  $x_i = E[\text{token ID}_i] + P_i$ , including role tokens.

LLaMA uses RoPE: position  $i$  rotates  $Q_i$  and  $K_i$  inside attention.

Next input  
position: 18

Role tokens have sequence positions and participate in the same causal attention.

# What may the current text position read?



All six are earlier context.

No future text position is available.

| The assistant's state can use both the picture and the question.

# The decoder mask is a model choice

## OUR DEFAULT · LLaVA-STYLE

Causal across image + text

| Query ↓ | Keys →  |         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|---------|---------|
|         | z1      | z2      | z3      | T1      | T2      | T3      | A       |
| z1      | Colored | Grey    | Grey    | Grey    | Grey    | Grey    | Grey    |
| z2      | Colored | Colored | Grey    | Grey    | Grey    | Grey    | Grey    |
| z3      | Colored | Colored | Colored | Grey    | Grey    | Grey    | Grey    |
| T1      | Grey    | Grey    | Grey    | Colored | Grey    | Grey    | Grey    |
| T2      | Grey    | Grey    | Grey    | Colored | Colored | Grey    | Grey    |
| T3      | Grey    | Grey    | Grey    | Colored | Colored | Colored | Grey    |
| A       | Grey    | Grey    | Grey    | Colored | Colored | Colored | Colored |

z1 reads itself; z2 reads z1 + z2.

Colored:  
allowed

Grey:  
blocked

Amber:  
extra links

## ALTERNATIVE · e.g. GEMMA 3

Full image block; causal text

| Query ↓ | Keys →  |         |         |         |         |         |         |
|---------|---------|---------|---------|---------|---------|---------|---------|
|         | z1      | z2      | z3      | T1      | T2      | T3      | A       |
| z1      | Colored | Amber   | Amber   | Grey    | Grey    | Grey    | Grey    |
| z2      | Colored | Colored | Amber   | Grey    | Grey    | Grey    | Grey    |
| z3      | Colored | Colored | Colored | Grey    | Grey    | Grey    | Grey    |
| T1      | Grey    | Grey    | Grey    | Colored | Grey    | Grey    | Grey    |
| T2      | Grey    | Grey    | Grey    | Colored | Colored | Grey    | Grey    |
| T3      | Grey    | Grey    | Grey    | Colored | Colored | Colored | Grey    |
| A       | Grey    | Grey    | Grey    | Colored | Colored | Colored | Colored |

z1 can read z1, z2 and z3.

z: image · T: prompt · A: assistant start. ViT mixed the image features earlier.

Both keep text causal. The difference is which image rows can read each other.

# This is the basic LLaVA-style connection

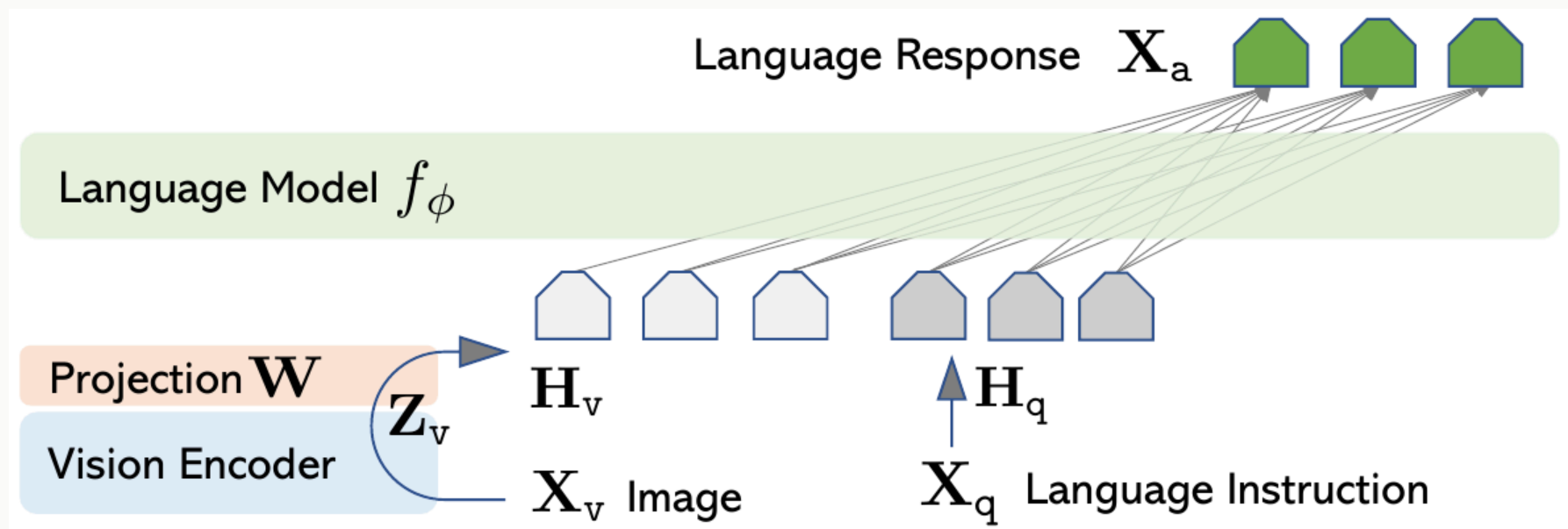


Projected patch vectors occupy earlier positions; they do not become word IDs.

# Visual Instruction Tuning

Haotian Liu • Chunyuan Li • Qingyang Wu • Yong Jae Lee

[Read the paper ↗](#)



Original Figure 1 • Image features and instruction embeddings enter the same language model.

## 1 • Vision encoder

CLIP ViT-L/14 supplies image features  $Z_v$ .

## 2 • Projection $W$

The original paper uses a linear map to match the LM embedding width.

## 3 • Language model

Vicuna reads the visual vectors and text, then predicts the response.

# Idea 1, end to end in one function



**image**

One image

**question** = "What animal is shown?"

**answer\_so\_far** = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

|                                                                 |                         |
|-----------------------------------------------------------------|-------------------------|
| <code>def next_token(image, question, answer_so_far):</code>    |                         |
| <code>    patches = vit(image).patch_states</code>              | $N \times d_V$          |
| <code>    visual = projector(patches)</code>                    | $N \times d_{LM}$       |
| <code>    ids = tokenize(question, answer_so_far)</code>        | T token IDs             |
| <code>    text = embed(ids)</code>                              | $T \times d_{LM}$       |
| <code>    inputs = concat([visual, text], axis="tokens")</code> | $(N + T) \times d_{LM}$ |
| <code>    hidden = causal_decoder(inputs)</code>                | $(N + T) \times d_{LM}$ |
| <code>    scores = lm_head(hidden[-1])</code>                   | Vocabulary scores       |
| <code>    return argmax(scores)</code>                          | Next text token ID      |

**causal\_decoder** stacks causal self-attention + MLP blocks over *all* input positions.

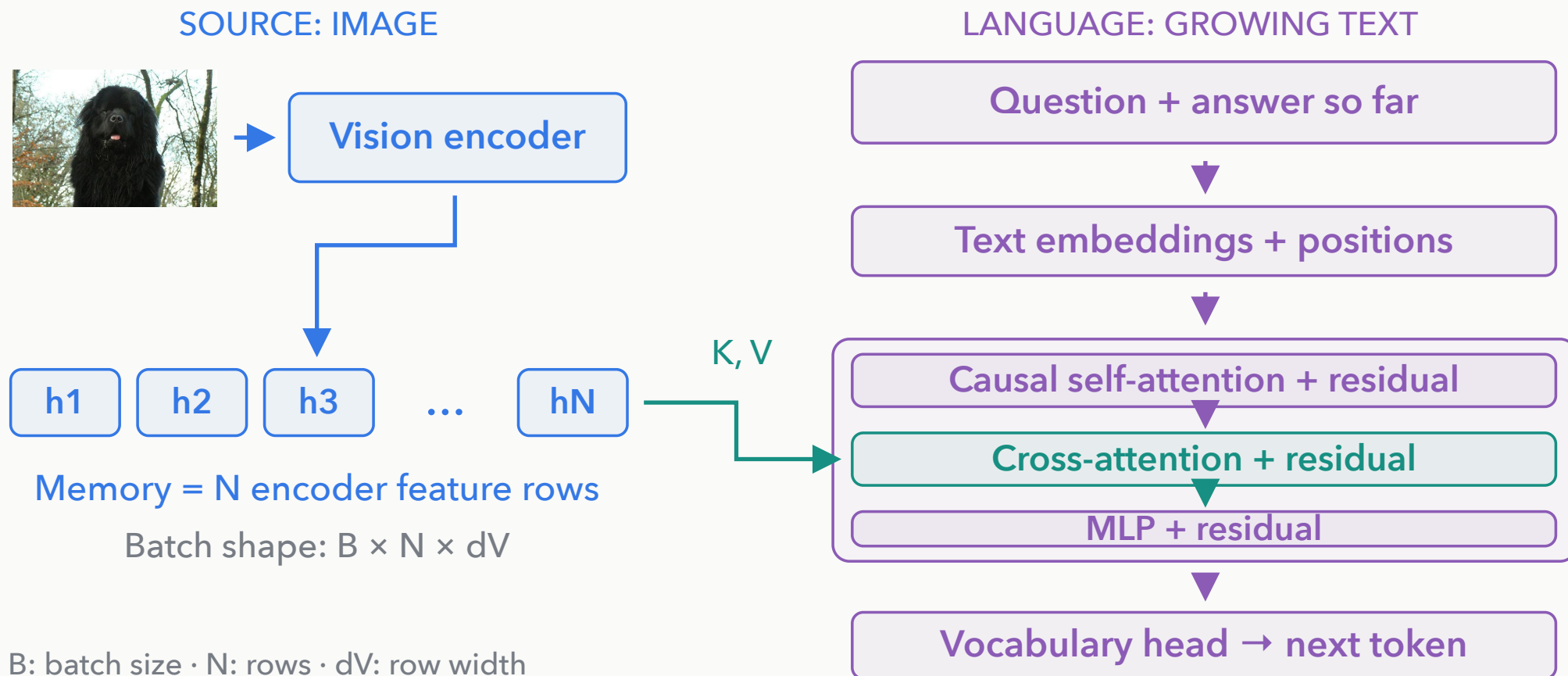
Display the returned token with the tokenizer; append its ID to the answer and predict again.

## SECTION 04

# Could vision remain separate?

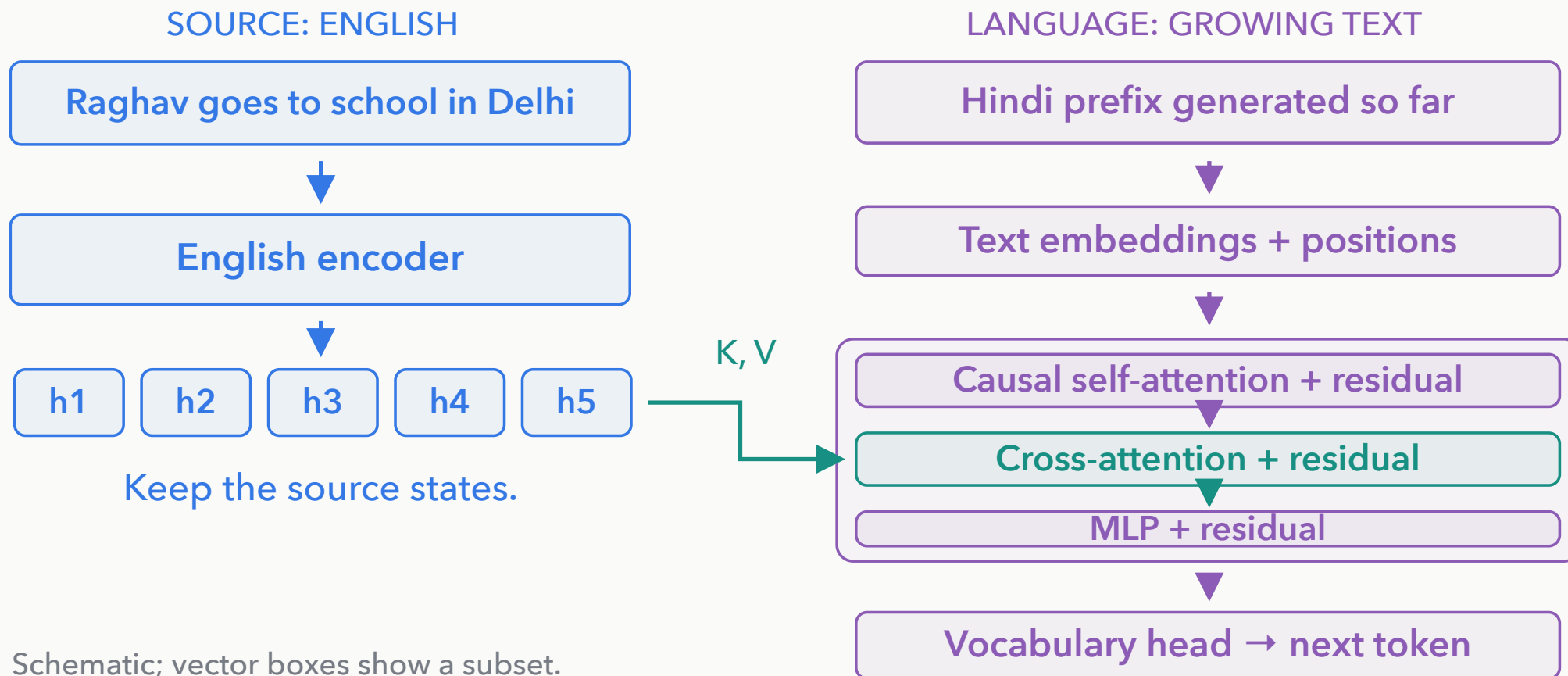
Return to Idea 2: how can language read separate visual memory?

# Idea 2 in detail · read a separate visual memory



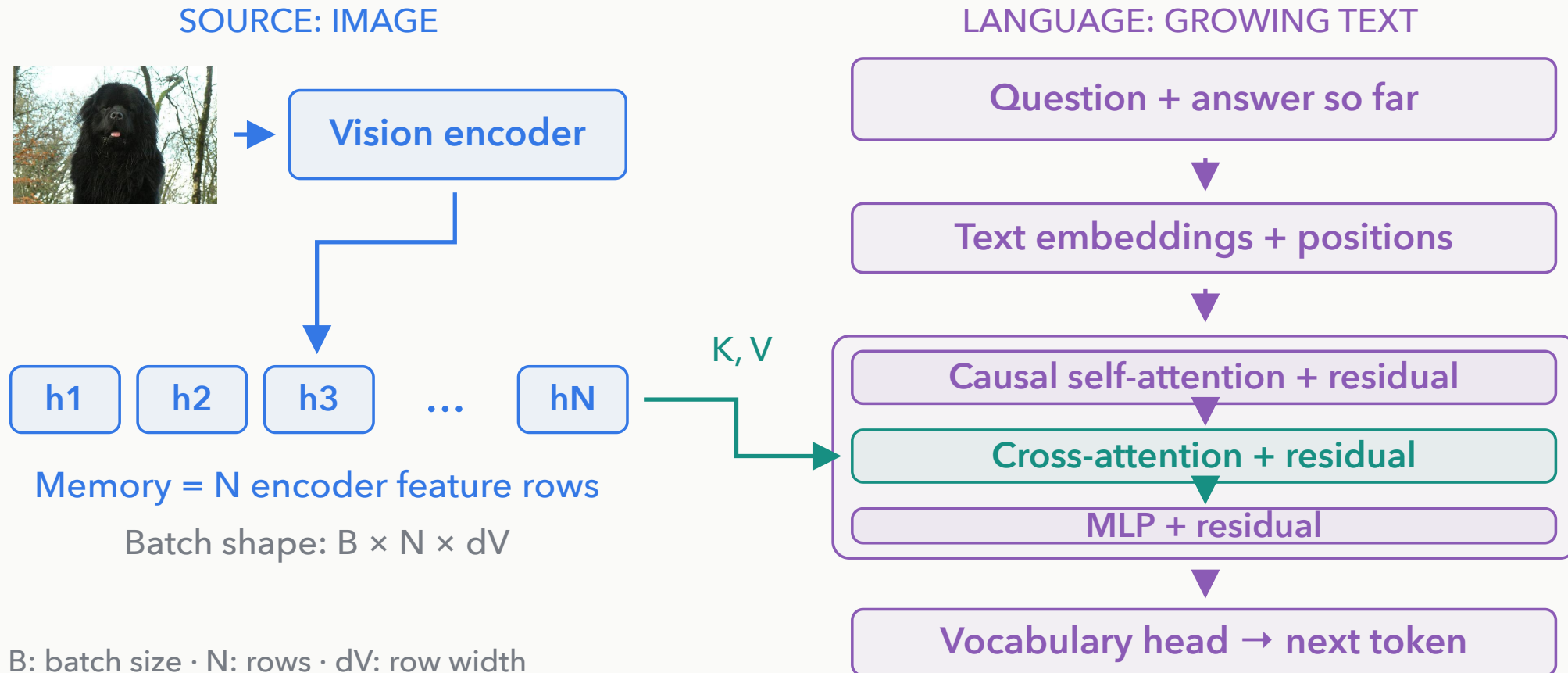
**Causal self-attention reads text; cross-attention reads image memory.**

# We already did this with English-to-Hindi translation



English source states stayed available while the Hindi prefix grew.

# Keep the decoder idea; change English to an image



| The source changes. The decoder still reads a separate memory.

# Same image, two questions: what should we read?

## ONE IMAGE



A dog, with trees behind it.

Authored examples;  
not a recorded model run.

## QUESTION A

What animal is shown?

Answer so far: "It is a"

**Reference continuation: dog**

## QUESTION B

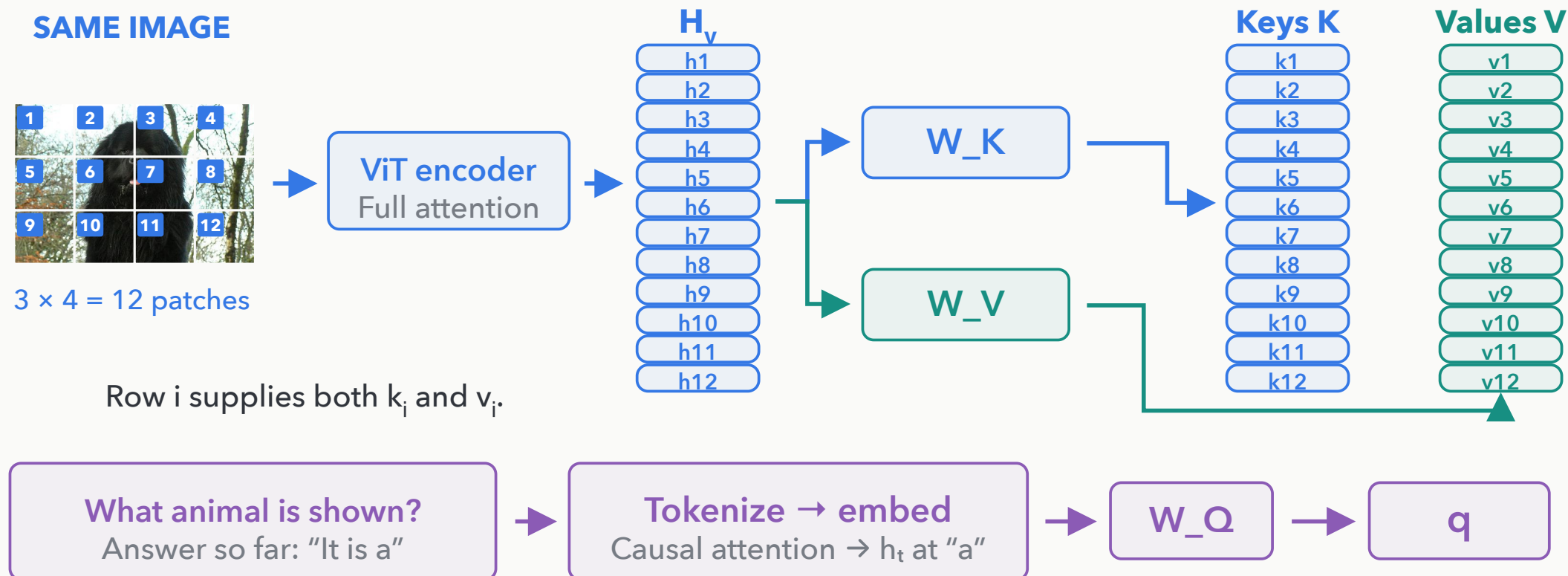
What is behind the dog?

Answer so far: "Behind it are"

**Reference continuation: trees**

Different questions need different visual evidence from the same image.

# The text supplies Q; the image supplies K and V



Each image row supplies a key and value; the contextual state at "a" supplies  $q$ .

# Use the keys to weight the values, then continue the answer



What animal is shown?

Answer so far:

It

is

a

State at "a"  $\rightarrow W_Q \rightarrow q$   
After causal self-attention

Includes the question + "It is".

## 1 · Compare q with every key

k1

k2

k3

k4

k5

k6

k7

k8

k9

k10

k11

k12

## 2 · Softmax gives weights $\alpha_1 \dots \alpha_{12}$ , which sum to 1

$\alpha_1$

$\alpha_2$

$\alpha_3$

$\alpha_4$

$\alpha_5$

$\alpha_6$

$\alpha_7$

$\alpha_8$

$\alpha_9$

$\alpha_{10}$

$\alpha_{11}$

$\alpha_{12}$

x

x

x

x

x

x

x

x

x

x

x

x

v1

v2

v3

v4

v5

v6

v7

v8

v9

v10

v11

v12

$$c = \sum_i \alpha_i v_i$$

Update "a" state  $\rightarrow$  LM head

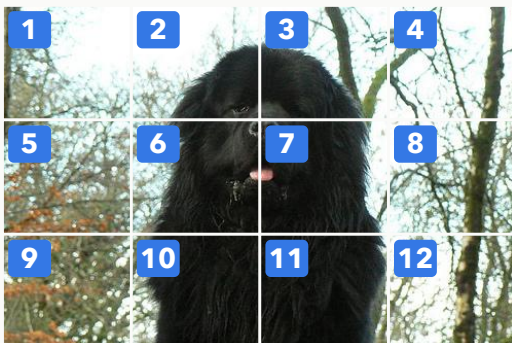
dog

Reference continuation

Update the state at "a"; the language head predicts the next token after it.

# The state at "a" supplies the query

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

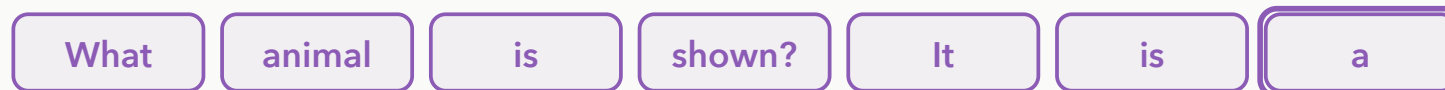
ANSWER SO FAR



Read the state at "a".

Toy numbers · one head  
Not measured from this photo

QUESTION + ANSWER: READABLE TOKEN LABELS



Tokenizer → IDs → embeddings + position handling



Causal self-attention → current text state  $h_t$

$$\begin{array}{c} h_t (1 \times 2) \\ \begin{array}{|c|c|} \hline 1 & 2 \\ \hline \end{array} \end{array} \times \begin{array}{c} W_Q (2 \times 2) \\ \begin{array}{|c|c|} \hline 0 & 1 \\ \hline 0.5 & -0.5 \\ \hline \end{array} \end{array} = \begin{array}{c} q (1 \times 2) \\ \begin{array}{|c|c|} \hline 1 & 0 \\ \hline \end{array} \end{array}$$

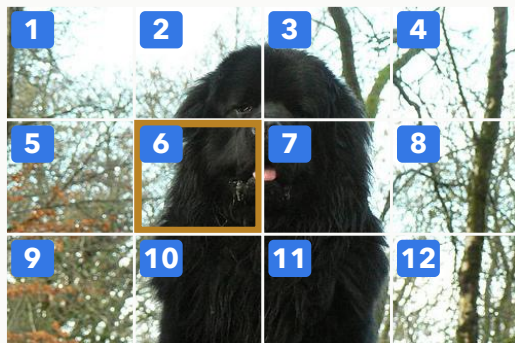
$$q = [1, 2] W_Q = [1, 0]$$

Toy  $h_t$  is assumed after the text sublayer;  $W_Q$  stays fixed.

Use the final supplied text state, after causal self-attention, to make  $q$ .

# The same twelve image rows become twelve keys

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head  
Not measured from this photo

ViT output  $H_v$ 

12 rows × 2 features

|    |   |   |
|----|---|---|
| 1  | 3 | 0 |
| 2  | 2 | 1 |
| 3  | 2 | 1 |
| 4  | 3 | 0 |
| 5  | 1 | 1 |
| 6  | 0 | 3 |
| 7  | 1 | 3 |
| 8  | 2 | 1 |
| 9  | 1 | 0 |
| 10 | 0 | 2 |
| 11 | 0 | 2 |
| 12 | 2 | 0 |

 $W_K$ 

|   |   |
|---|---|
| 0 | 1 |
| 1 | 0 |

2 × 2

$$H_v W_K = K$$

Keys  $K$ 

12 rows × 2 features

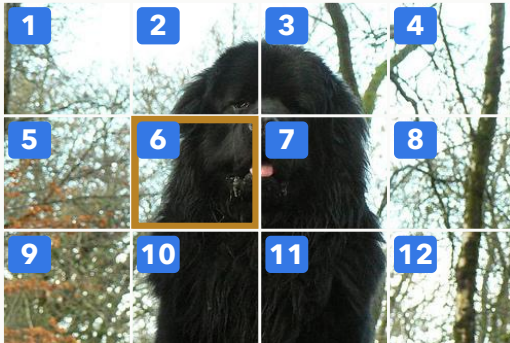
|    |   |   |
|----|---|---|
| 1  | 0 | 3 |
| 2  | 1 | 2 |
| 3  | 1 | 2 |
| 4  | 0 | 3 |
| 5  | 1 | 1 |
| 6  | 3 | 0 |
| 7  | 3 | 1 |
| 8  | 1 | 2 |
| 9  | 0 | 1 |
| 10 | 2 | 0 |
| 11 | 2 | 0 |
| 12 | 0 | 2 |

Row 6:  $[0, 3] W_K = [3, 0]$

Apply  $W_K$  to every encoder row; inspect patch 6 without dropping the other rows.

# Each image row also supplies a paired value

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head  
Not measured from this photo

ViT output  $H_v$ 

12 rows × 2 features

|    |   |   |
|----|---|---|
| 1  | 3 | 0 |
| 2  | 2 | 1 |
| 3  | 2 | 1 |
| 4  | 3 | 0 |
| 5  | 1 | 1 |
| 6  | 0 | 3 |
| 7  | 1 | 3 |
| 8  | 2 | 1 |
| 9  | 1 | 0 |
| 10 | 0 | 2 |
| 11 | 0 | 2 |
| 12 | 2 | 0 |

 $W_v$ 

|   |   |
|---|---|
| 1 | 0 |
| 1 | 2 |

2 × 2

$$H_v W_v = V$$

Values  $V$ 

12 rows × 2 features

|    |   |   |
|----|---|---|
| 1  | 3 | 0 |
| 2  | 3 | 2 |
| 3  | 3 | 2 |
| 4  | 3 | 0 |
| 5  | 2 | 2 |
| 6  | 3 | 6 |
| 7  | 4 | 6 |
| 8  | 3 | 2 |
| 9  | 1 | 0 |
| 10 | 2 | 4 |
| 11 | 2 | 4 |
| 12 | 2 | 0 |

Row 6:  $[0, 3] W_v = [3, 6]$

The key determines the match; its paired value supplies the features to mix.

# Score every image row for this text position

## SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head  
Not measured from this photo

$q = [1, 0]$  · one head has  $d_k = 2$

| Patch $i$ | Key $k_i$ | $q \cdot k_i$ | Score $s_i = (q \cdot k_i) / \sqrt{2}$ |
|-----------|-----------|---------------|----------------------------------------|
| 1         | [0, 3]    | 0             | 0.000                                  |
| 2         | [1, 2]    | 1             | 0.707                                  |
| 3         | [1, 2]    | 1             | 0.707                                  |
| 4         | [0, 3]    | 0             | 0.000                                  |
| 5         | [1, 1]    | 1             | 0.707                                  |
| 6         | [3, 0]    | 3             | 2.121                                  |
| 7         | [3, 1]    | 3             | 2.121                                  |
| 8         | [1, 2]    | 1             | 0.707                                  |
| 9         | [0, 1]    | 0             | 0.000                                  |
| 10        | [2, 0]    | 2             | 1.414                                  |
| 11        | [2, 0]    | 2             | 1.414                                  |
| 12        | [0, 2]    | 0             | 0.000                                  |

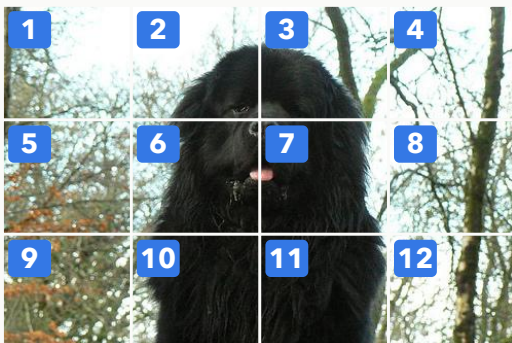
Score scale · before softmax

Patch 6:  $(1 \times 3 + 0 \times 0) / \sqrt{2} = 2.121$

One query × twelve keys gives twelve scores; every patch stays in the computation.

# Normalize across all twelve image positions

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head  
Not measured from this photo

$$\alpha_i = \exp(s_i) / \sum_j \exp(s_j)$$

| Patch $i$ | Score $s_i$ | $\exp(s_i)$ | Weight $\alpha_i$ |
|-----------|-------------|-------------|-------------------|
| 1         | 0.000       | 1.000       | 0.027             |
| 2         | 0.707       | 2.028       | 0.055             |
| 3         | 0.707       | 2.028       | 0.055             |
| 4         | 0.000       | 1.000       | 0.027             |
| 5         | 0.707       | 2.028       | 0.055             |
| 6         | 2.121       | 8.342       | 0.225             |
| 7         | 2.121       | 8.342       | 0.225             |
| 8         | 0.707       | 2.028       | 0.055             |
| 9         | 0.000       | 1.000       | 0.027             |
| 10        | 1.414       | 4.113       | 0.111             |
| 11        | 1.414       | 4.113       | 0.111             |
| 12        | 0.000       | 1.000       | 0.027             |

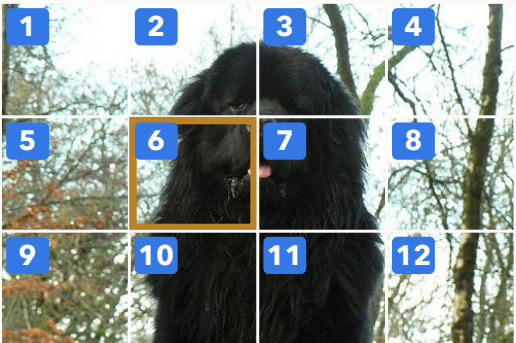
**Patch 6: 8.342 / 37.023 = 0.225**

$\sum_i \alpha_i = 1$  exactly before rounding; these are not word probabilities.

Divide each exponential by the sum of all twelve exponentials.

# Weight the paired values and add the twelve rows

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head

Not measured from this photo

$$c = aV = \sum_i a_i v_i$$

| Patch $i$ | Weight $a_i$ | Value $v_i$    | Contribution $a_i v_i$ |
|-----------|--------------|----------------|------------------------|
| 1         | 0.027        | [3.000, 0.000] | [0.081, 0.000]         |
| 2         | 0.055        | [3.000, 2.000] | [0.164, 0.110]         |
| 3         | 0.055        | [3.000, 2.000] | [0.164, 0.110]         |
| 4         | 0.027        | [3.000, 0.000] | [0.081, 0.000]         |
| 5         | 0.055        | [2.000, 2.000] | [0.110, 0.110]         |
| 6         | 0.225        | [3.000, 6.000] | [0.676, 1.352]         |
| 7         | 0.225        | [4.000, 6.000] | [0.901, 1.352]         |
| 8         | 0.055        | [3.000, 2.000] | [0.164, 0.110]         |
| 9         | 0.027        | [1.000, 0.000] | [0.027, 0.000]         |
| 10        | 0.111        | [2.000, 4.000] | [0.222, 0.444]         |
| 11        | 0.111        | [2.000, 4.000] | [0.222, 0.444]         |
| 12        | 0.027        | [2.000, 0.000] | [0.054, 0.000]         |

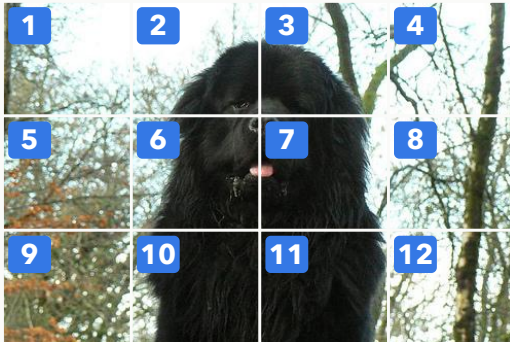
Add all 12 rows →  $c = [2.867, 4.031]$

Shown to 3 decimal places; products and sums use full precision.

All twelve contributions add up to one visual message for the current text position.

# Change the question; read the state at "are"

SAME IMAGE · ALL 12 PATCHES



What is behind the dog?

ANSWER SO FAR

Behind it are

Read the state at "are".

Toy numbers · one head  
Not measured from this photo

QUESTION + ANSWER: READABLE TOKEN LABELS

What is behind the dog? Behind it are

Tokenizer → IDs → embeddings + position handling



Causal self-attention → current text state  $h_t$

$$\begin{array}{c} h_t (1 \times 2) \\ \begin{array}{|c|c|} \hline 3 & 0 \\ \hline \end{array} \end{array} \times \begin{array}{c} W_Q (2 \times 2) \\ \begin{array}{|c|c|} \hline 0 & 1 \\ \hline 0.5 & -0.5 \\ \hline \end{array} \end{array} = \begin{array}{c} q (1 \times 2) \\ \begin{array}{|c|c|} \hline 0 & 3 \\ \hline \end{array} \end{array}$$

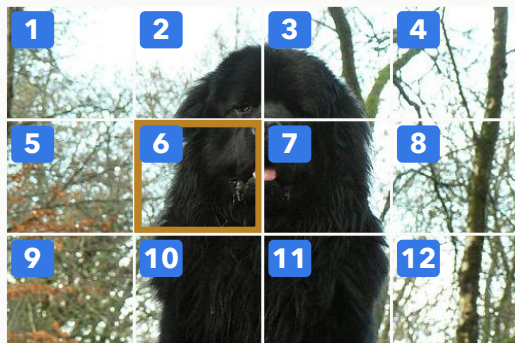
$$q = [3, 0] \quad W_Q = [0, 3]$$

Toy  $h_t$  is assumed after the text sublayer;  $W_Q$  stays fixed.

The image, keys, values and  $W_Q$  stay fixed; the text state changes.

# Same patch 6, two different query scores

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR



Read the state at "a".

Toy numbers · one head

Not measured from this photo

ZOOM INTO PATCH 6 · SAME KEY IN BOTH LOOKUPS

$$h_6 = [0, 3] \rightarrow k_6 = [3, 0]$$

ANIMAL QUESTION

$$[1, 0] \cdot [3, 0] = 3$$

$$\text{Scaled score: } 3 / \sqrt{2} = 2.121$$

BACKGROUND QUESTION

$$[0, 3] \cdot [3, 0] = 0$$

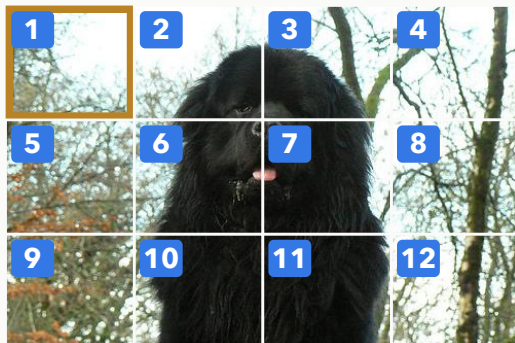
$$\text{Scaled score: } 0 / \sqrt{2} = 0.000$$

This is one row zoom. Both lookups still use all 12.

A fixed image key can match different language queries differently.

# Now inspect patch 1 with those same queries

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR



Read the state at "a".

Toy numbers · one head

Not measured from this photo

ZOOM INTO PATCH 1 · SAME KEY IN BOTH LOOKUPS

$$h1 = [3, 0] \rightarrow k1 = [0, 3]$$

ANIMAL QUESTION

$$[1, 0] \cdot [0, 3] = 0$$

Scaled score:  $0 / \sqrt{2} = 0.000$

BACKGROUND QUESTION

$$[0, 3] \cdot [0, 3] = 9$$

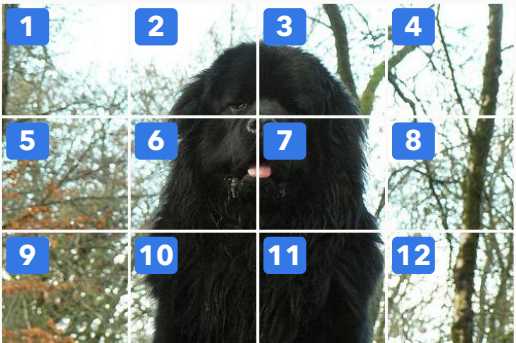
Scaled score:  $9 / \sqrt{2} = 6.364$

This is one row zoom. Both lookups still use all 12.

The query changes which keys receive large scores.

# Recompute the whole score row, then softmax

SAME IMAGE · ALL 12 PATCHES



What is behind the dog?

ANSWER SO FAR

Behind it are

Read the state at "are".

Toy numbers · one head  
Not measured from this photo

SAME 12 KEYS · DIFFERENT LANGUAGE QUERIES

| Patch | Key $k_i$ | $s_i: q = [1, 0]$ | Weight | $s_i: q = [0, 3]$ | Weight |
|-------|-----------|-------------------|--------|-------------------|--------|
| 1     | [0, 3]    | 0.000             | 0.027  | 6.364             | 0.396  |
| 2     | [1, 2]    | 0.707             | 0.055  | 4.243             | 0.047  |
| 3     | [1, 2]    | 0.707             | 0.055  | 4.243             | 0.047  |
| 4     | [0, 3]    | 0.000             | 0.027  | 6.364             | 0.396  |
| 5     | [1, 1]    | 0.707             | 0.055  | 2.121             | 0.006  |
| 6     | [3, 0]    | 2.121             | 0.225  | 0.000             | 0.001  |
| 7     | [3, 1]    | 2.121             | 0.225  | 2.121             | 0.006  |
| 8     | [1, 2]    | 0.707             | 0.055  | 4.243             | 0.047  |
| 9     | [0, 1]    | 0.000             | 0.027  | 2.121             | 0.006  |
| 10    | [2, 0]    | 1.414             | 0.111  | 0.000             | 0.001  |
| 11    | [2, 0]    | 1.414             | 0.111  | 0.000             | 0.001  |
| 12    | [0, 2]    | 0.000             | 0.027  | 4.243             | 0.047  |

Softmax runs separately for each query.

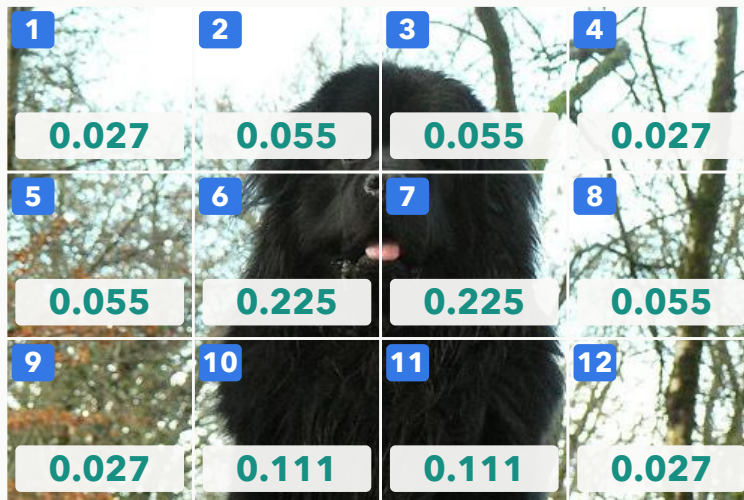
The keys did not change. The comparison with those keys did.

Each query gets its own twelve weights; the image memory is reused.

# Put the calculated weights back on the patch grid

What animal is shown?

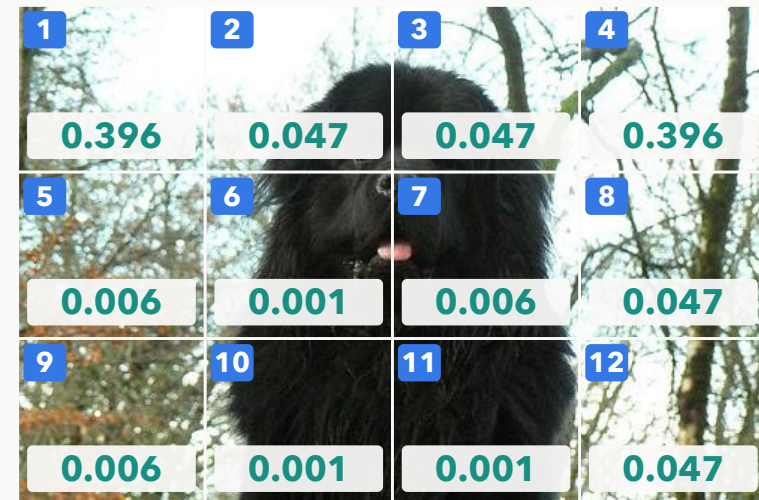
It is a  $\rightarrow q = [1, 0]$



$\Sigma \text{ weights} = 1$

What is behind the dog?

Behind it are  $\rightarrow q = [0, 3]$



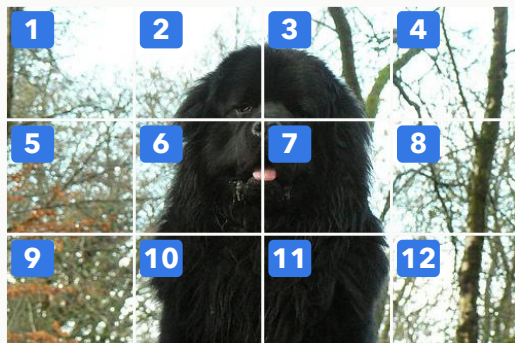
$\Sigma \text{ weights} = 1$

Calculated toy weights on patch positions · not a measured model attention map

| Same image and keys; different queries give different mixtures of image values.

# Two questions, two messages from the same values

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR



Read the state at "a".

Toy numbers · one head  
Not measured from this photo

V STAYS FIXED; EACH QUERY CHOOSES ITS MIXTURE

Animal question ·  $q = [1, 0]$

$$c = \sum_{i=1}^{12} a_i v_i = [2.867, 4.031]$$

All 12 rows; weights from the first softmax.

Background question ·  $q = [0, 3]$

$$c = \sum_{i=1}^{12} a_i v_i = [2.94, 0.34]$$

All 12 rows; weights from the second softmax.

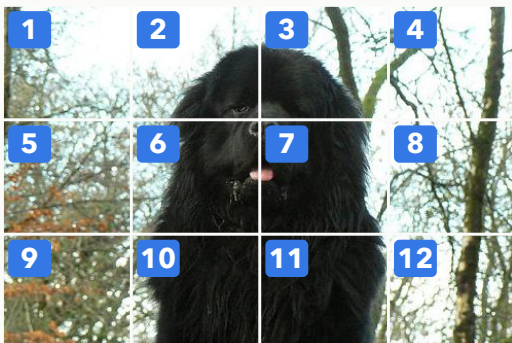
**Different messages can update the language state differently.**

Neither 2-number vector is the word "dog" or the word "trees".

Changing  $q$  can change the visual message without recomputing  $K$  or  $V$ .

# Project the visual message and update the text state

SAME IMAGE · ALL 12 PATCHES



What animal is shown?

ANSWER SO FAR

It is a

Read the state at "a".

Toy numbers · one head

Not measured from this photo

RETURN THE VISUAL MESSAGE TO THE TEXT PATH

$$c = [2.867, 4.031] (1 \times 2)$$

$$c \times \begin{matrix} \begin{matrix} 1 & 0 \\ 0.5 & 1 \end{matrix} \\ W_O (2 \times 2) \end{matrix} = [4.883, 4.031]$$

$$h'_t = h_t + cW_O$$

$$= [1, 2] + [4.883, 4.031] = [5.883, 6.031]$$

Remaining decoder → vocabulary head → next token

The visual message changes the state at "a"; a later vocabulary head scores words.

# Check: has cross-attention already answered “dog”?

## Lookup result

$$c \approx [2.867, 4.031]$$

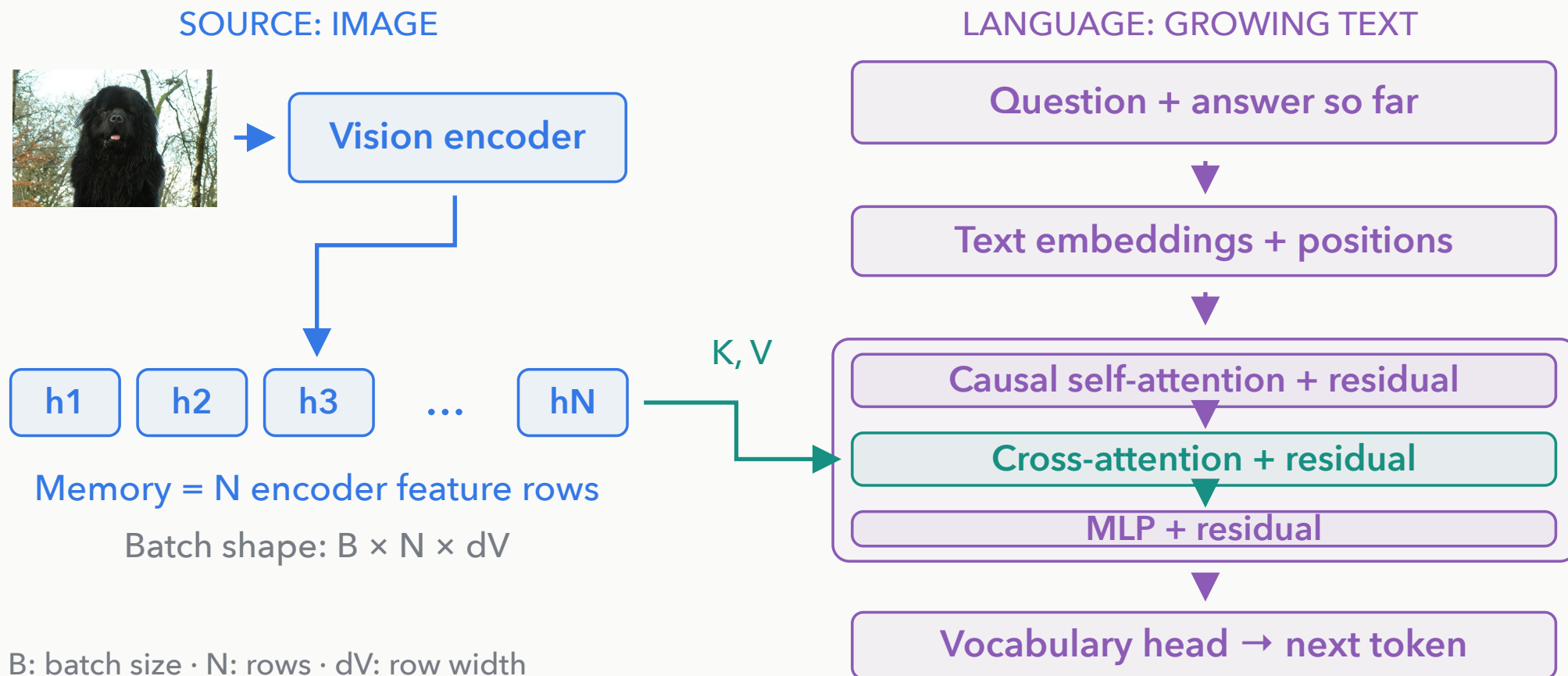
## What happens next?

Does  $c$  name a word?

Or does it update the decoder’s current state?

| Predict before advancing: where does a vocabulary token actually appear?

# The vocabulary head still produces the next-token scores



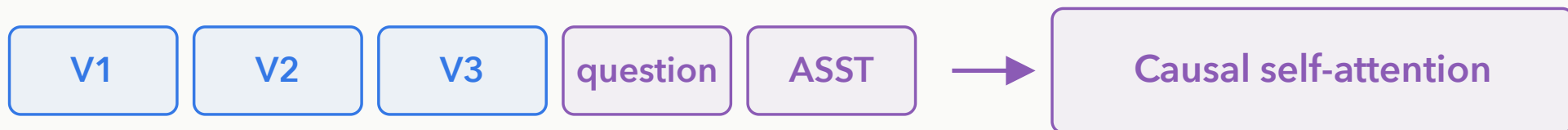
Visual lookup changes the language state; the language head chooses words.

# Two ways for language to access vision

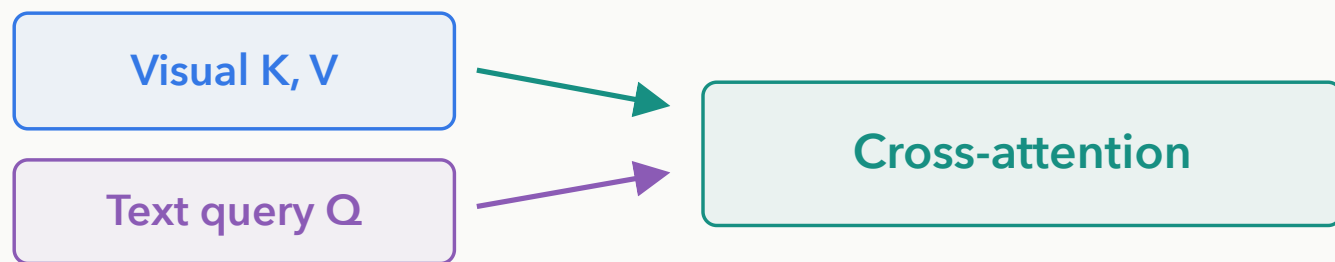


Same image. Same question: What animal is shown?

## A · VISUAL PREFIX



## B · SEPARATE MEMORY



In-sequence visual context or a separate visual memory.

# Idea 2, end to end in one function

**image**

One image

**question** = "What animal is shown?"**answer\_so\_far** = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

|                                                        |                    |
|--------------------------------------------------------|--------------------|
| <b>def</b> next_token(image, question, answer_so_far): |                    |
| memory = vision_encoder(image).features                | N × dV             |
| ids = tokenize(question, answer_so_far)                | T token IDs        |
| text = embed(ids)                                      | T × dLM            |
| <b>for</b> block <b>in</b> decoder_blocks:             |                    |
| text = block.causal_self_attn(text)                    | T × dLM            |
| text = block.cross_attn(text, memory)                  | T × dLM            |
| text = block.mlp(text)                                 | T × dLM            |
| <b>return</b> argmax(lm_head(text[-1]))                | Next text token ID |

**cross\_attn**: Q from text; K and V from image memory. The two sequences stay separate.

Sublayer helpers include residuals and norms. Append the returned token ID to the answer; repeat.

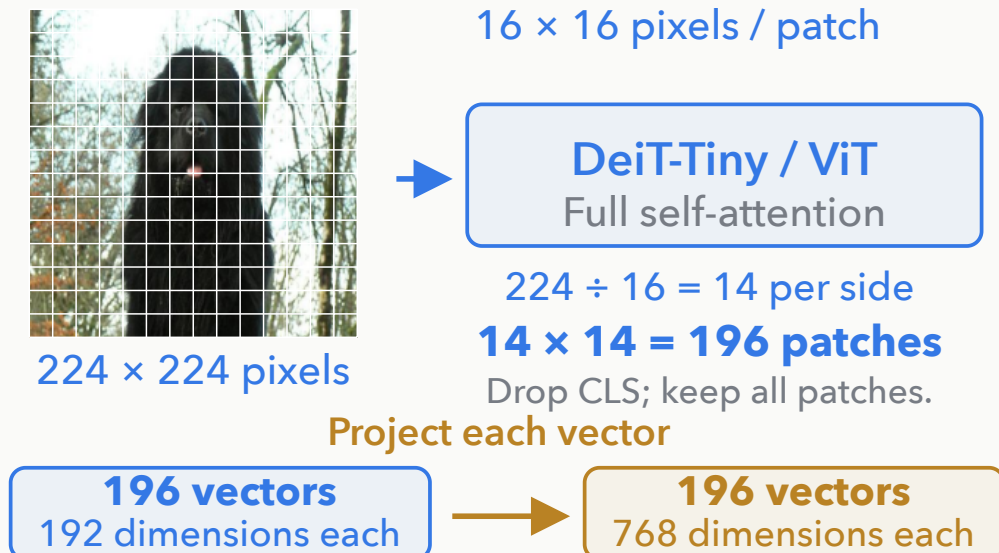
## SECTION 05

# Do we need every visual token?

Every image adds decoder positions. What do they cost, and can a shorter visual summary help?

# How long is this image + question + answer prefix?

## ONE IMAGE · AFTER RESIZE / CROP



## COMPLETE TEXT SUPPLIED TO THE MODEL

Question: What animal is shown?

Answer: It is a

DistilGPT2 tokenizer → these 13 pieces



· = leading space ↵ = one newline token

13 IDs → 13 text embeddings  
768 dimensions per vector

196 image vectors

13

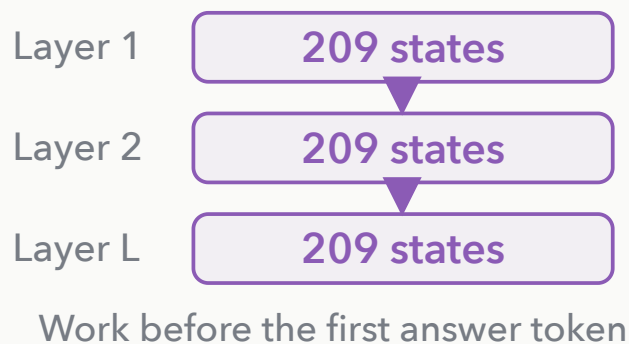
**196 visual + 13 text = 209 positions in the causal decoder**

Projection changes each vector: 192 → 768 dimensions. We still have 196 vectors.

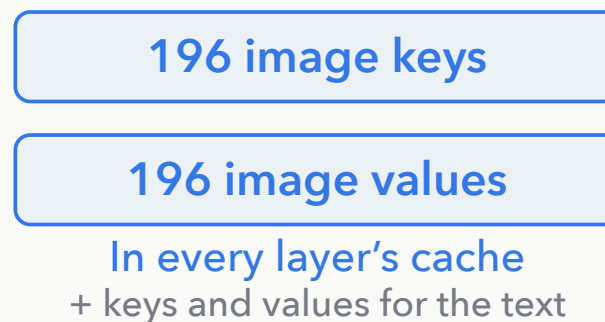
# 209 positions fit. What do they cost?



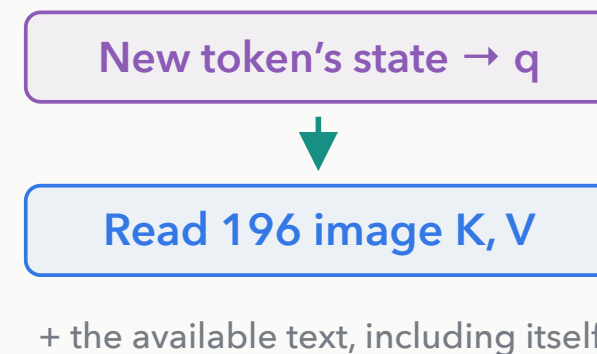
## 1 · PROCESS THE INPUT



## 2 · STORE THE IMAGE K, V



## 3 · READ THEM AGAIN



**Caching reuses K and V; it does not remove the image positions.**

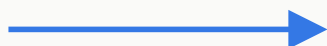
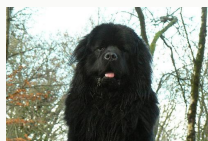
Every image position adds decoder work, cache storage and later attention reads.

# What happens when the prompt contains several images?

**Question: "Describe each image."**

Assume the same  $224 \times 224$  input and 196 patch vectors per image.

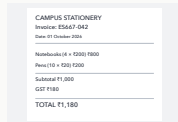
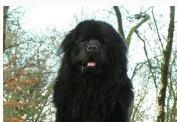
## ONE IMAGE



**196 image positions**

**196**

## FOUR IMAGES IN ONE PROMPT



**$4 \times 196 = 784$  image positions**

**196**

**196**

**196**

**196**

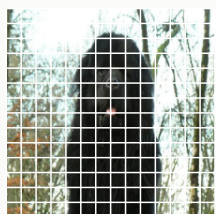
4× the visual K, V in each layer's cache

**The text and the growing answer also need decoder positions.**

**More visual positions consume more cache and leave less context room for the answer.**

# Keep the text; shorten the visual prefix

**SAME IMAGE + ViT**



**196**

patch rows

**Learned summarizer**  
Choose  $M = 8$  readers

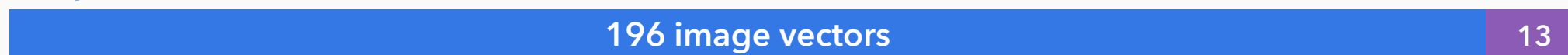
All 196 rows remain available.

**8 summary vectors**



Project 8 rows to width 768 → prepend them before the same 13 text embeddings

**Full prefix:  $196 + 13 = 209$**



**Short prefix:  $8 + 13 = 21$**



**188 fewer decoder positions to process**  
A smaller cache to store and read · same scale

**SAME TEXT**

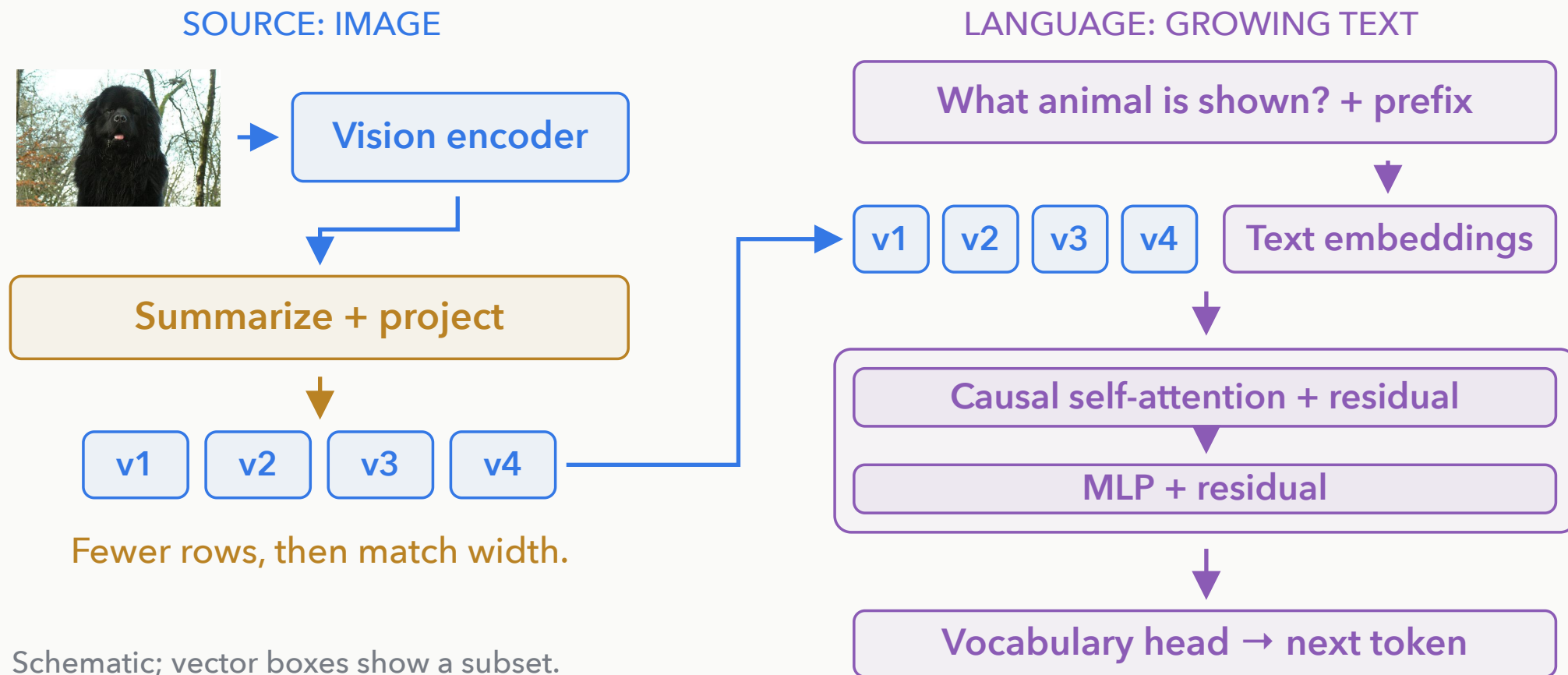
Question: What animal is shown?

Answer: It is a

**13 text embeddings**

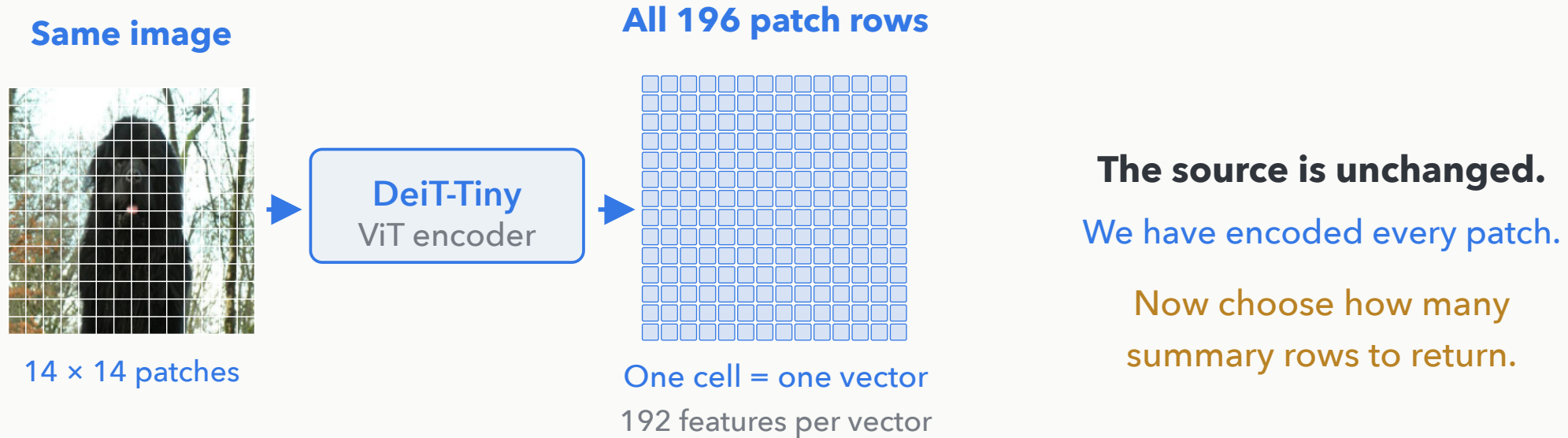
Summarize 196 image vectors into 8: fewer decoder positions, with evidence we must preserve.

# Idea 3 in detail · make the visual context shorter



Replace the row-wise projector with a learned summary followed by projection.

# Build the summary from all 196 image rows

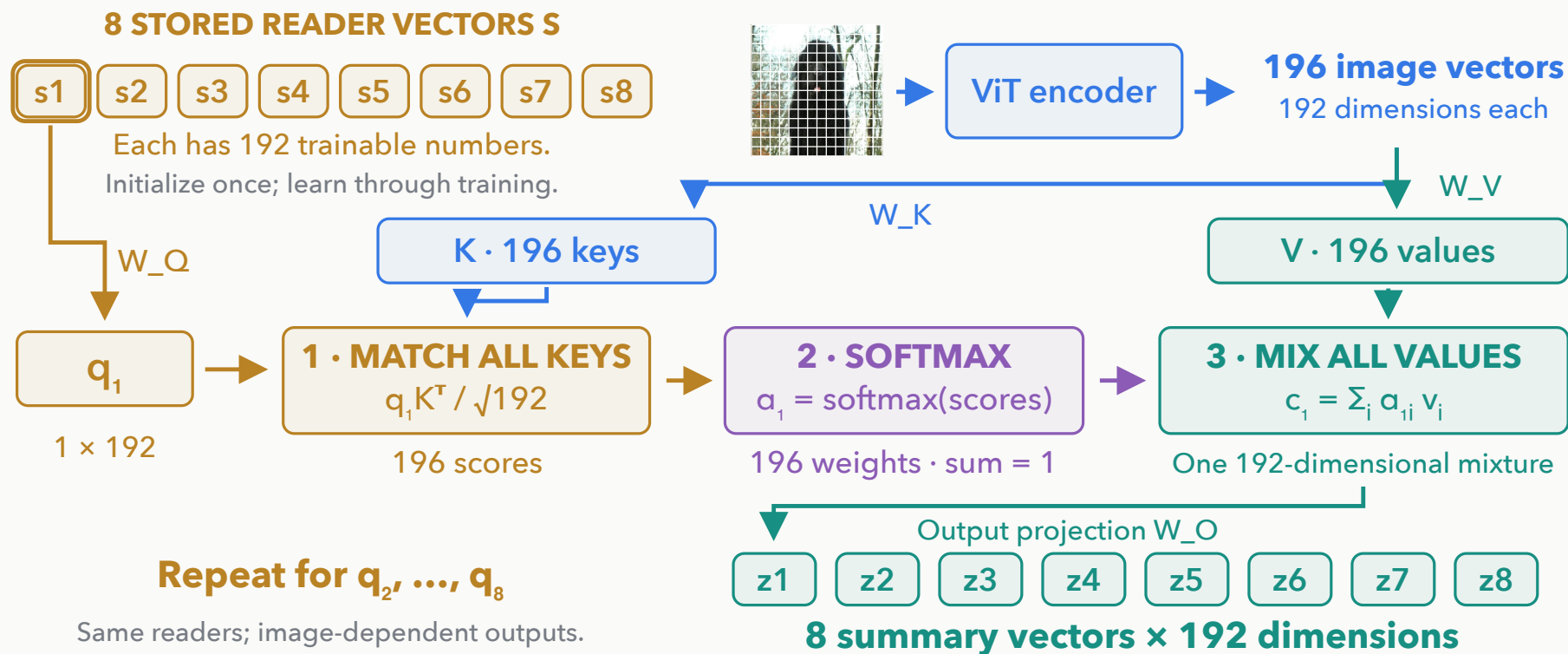


The grid tracks patch positions; the feature matrix is  $H_V \in \mathbb{R}^{196 \times 192}$ .

ViT full attention lets each patch row use information from the rest of the image.

| The source memory is the encoded image: one feature vector per patch position.

# One learned reader makes one visual summary

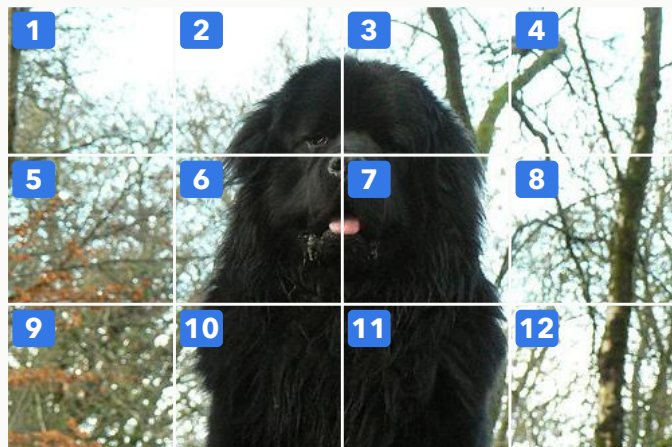


One-head diagram · the notebook uses 3 heads; its new reader is untrained.

Learn the readers; compute a new weighted mixture for each reader and each image.

# Try two numeric readers on the familiar image

## TOY: 12 PATCHES → 2 SUMMARIES



Assume two projected reader queries:

$$q_1 = [1, 0]$$

$$q_2 = [0, 3]$$

Hand-set values, not photo measurements

Q:  $2 \times 2$  K, V:  $12 \times 2 \rightarrow$  two output vectors, each with 2 features

## SAME IMAGE MEMORY AS BEFORE

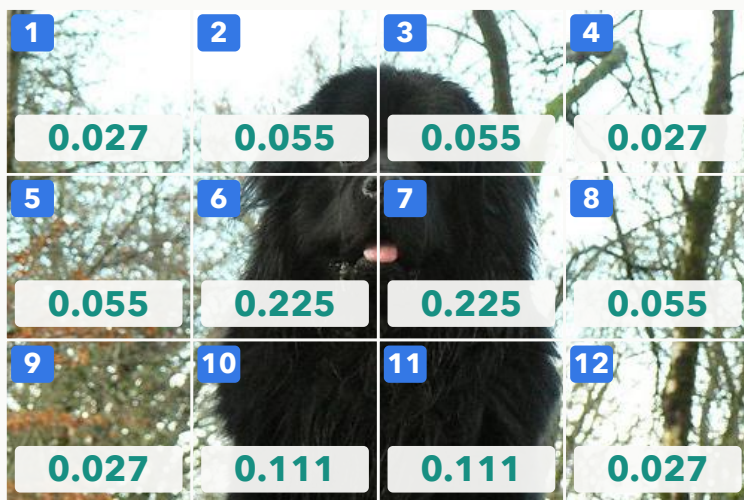
| Patch $i$ | Key $k_i$ | Value $v_i$ |
|-----------|-----------|-------------|
| 1         | [0, 3]    | [3, 0]      |
| 2         | [1, 2]    | [3, 2]      |
| 3         | [1, 2]    | [3, 2]      |
| 4         | [0, 3]    | [3, 0]      |
| 5         | [1, 1]    | [2, 2]      |
| 6         | [3, 0]    | [3, 6]      |
| 7         | [3, 1]    | [4, 6]      |
| 8         | [1, 2]    | [3, 2]      |
| 9         | [0, 1]    | [1, 0]      |
| 10        | [2, 0]    | [2, 4]      |
| 11        | [2, 0]    | [2, 4]      |
| 12        | [0, 2]    | [2, 0]      |

For this small calculation, choose two readers; give both all twelve memory rows.

# Two readers make two mixtures of the same image

## READER 1 · $q_1 = [1, 0]$

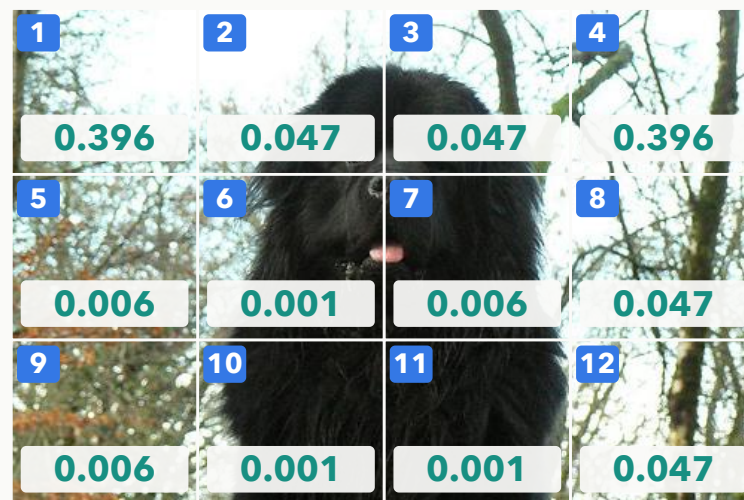
Match = 1 × the key's first coordinate



$$z_1 = \sum_i \alpha_{1i} v_i \approx [2.867, 4.031]$$

## READER 2 · $q_2 = [0, 3]$

Match = 3 × the key's second coordinate



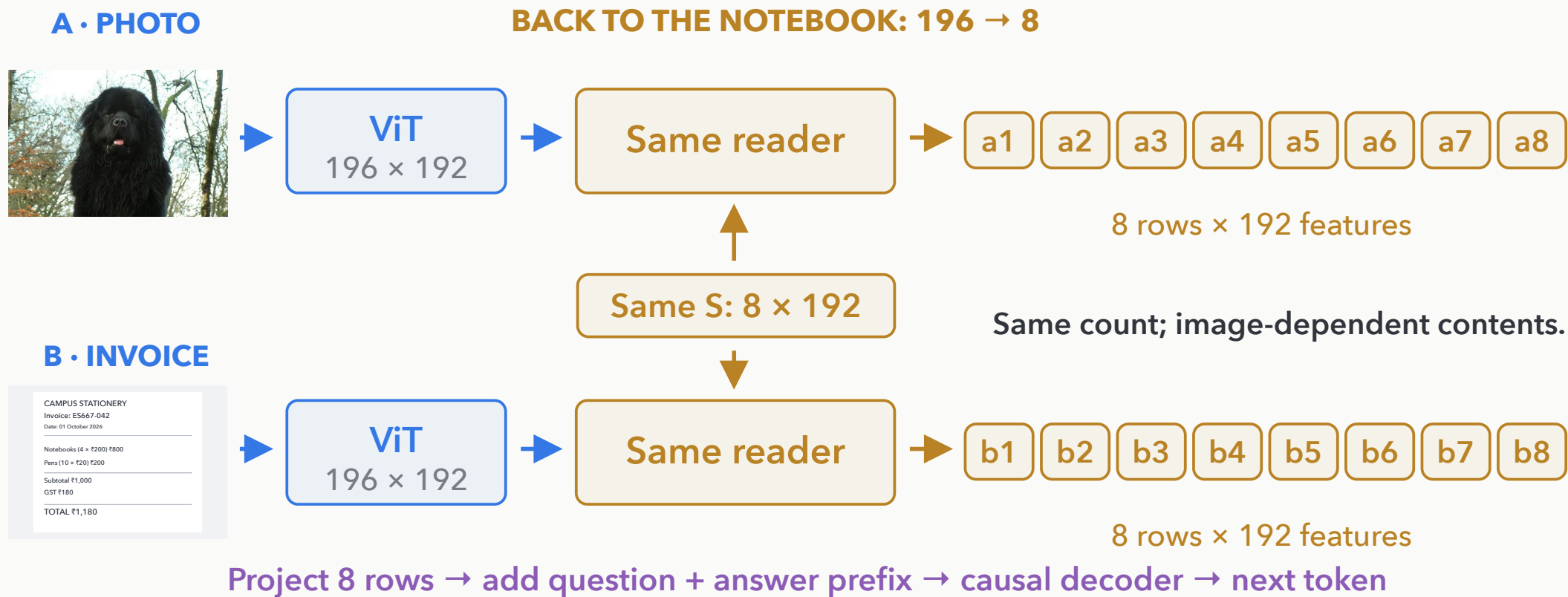
$$z_2 = \sum_i \alpha_{2i} v_i \approx [2.94, 0.34]$$

12 value rows → 2 weighted mixtures · each reader can use every patch

Toy head ·  $\alpha = \text{softmax}(qK^T / \sqrt{2})$  · output projection = I · not measured attention

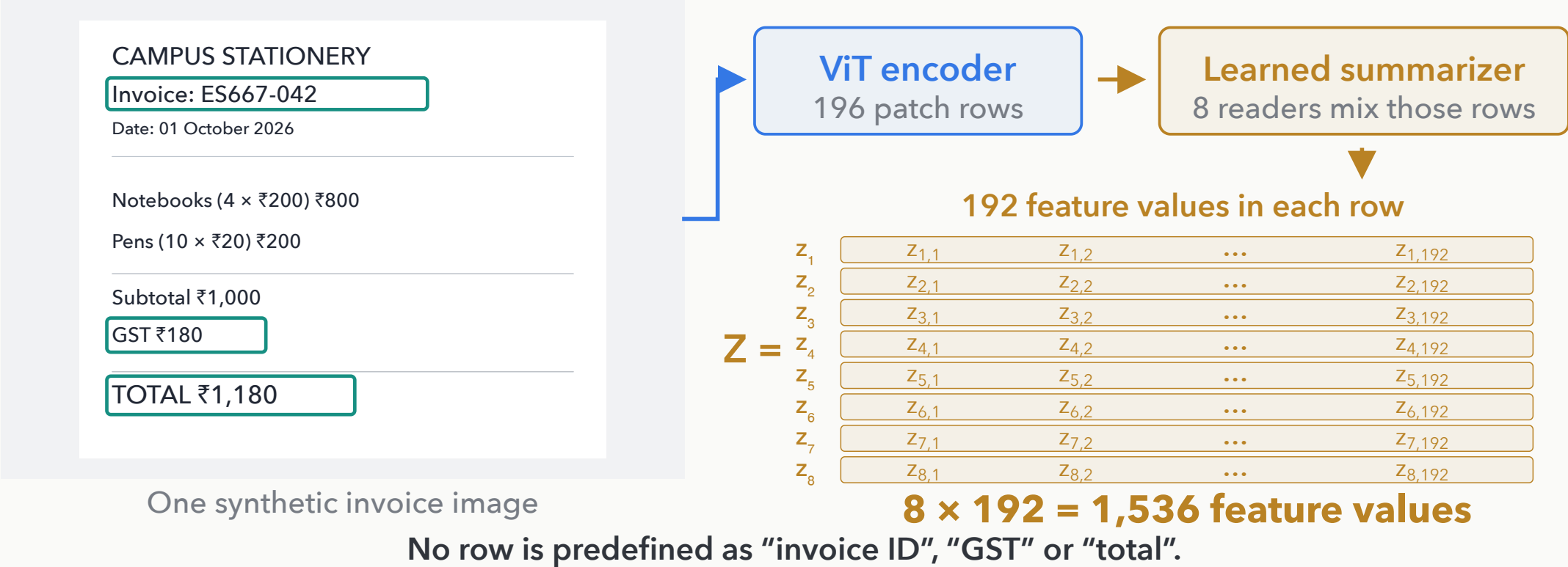
A fixed number of readers gives a fixed number of summary vectors.

# Same readers; a different image to summarize



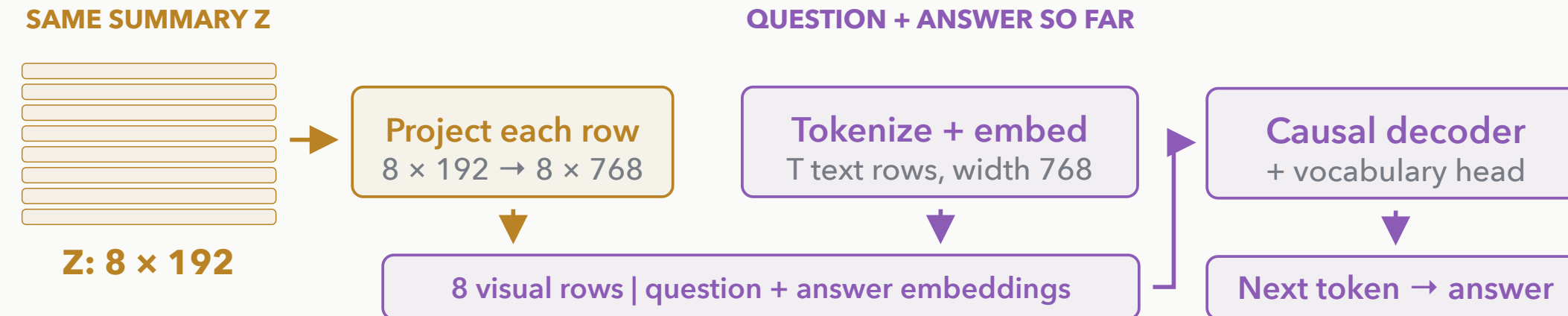
The summary length stays fixed; its feature values depend on the source image.

# Eight vectors can represent many image details



A detail can be represented across several rows and feature coordinates.

# Reuse the summary; change only the question



**REUSE Z · START A FRESH ANSWER FOR EACH QUESTION**

Reference answers we want after training

What is the invoice ID?

**ES667-042**

How much GST is charged?

**₹180**

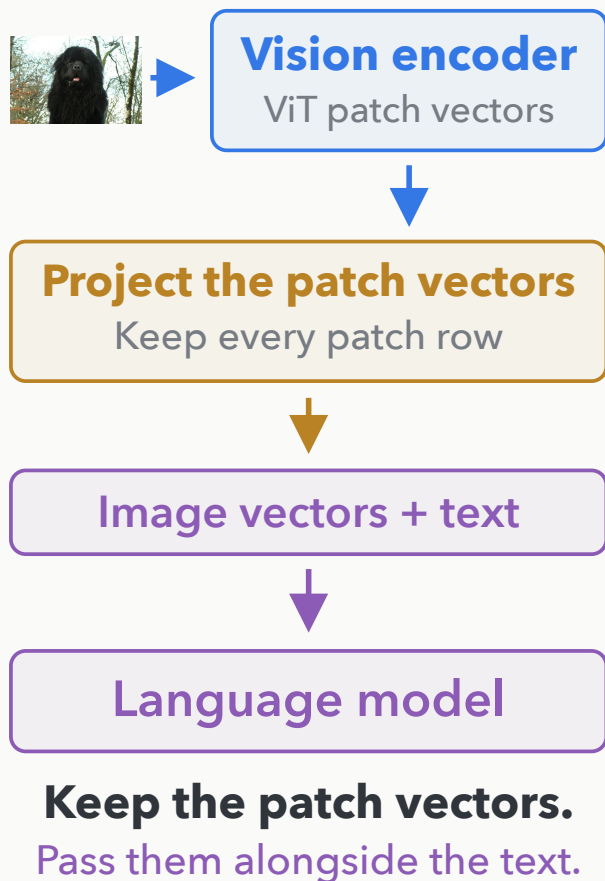
What is the total payable?

**₹1,180**

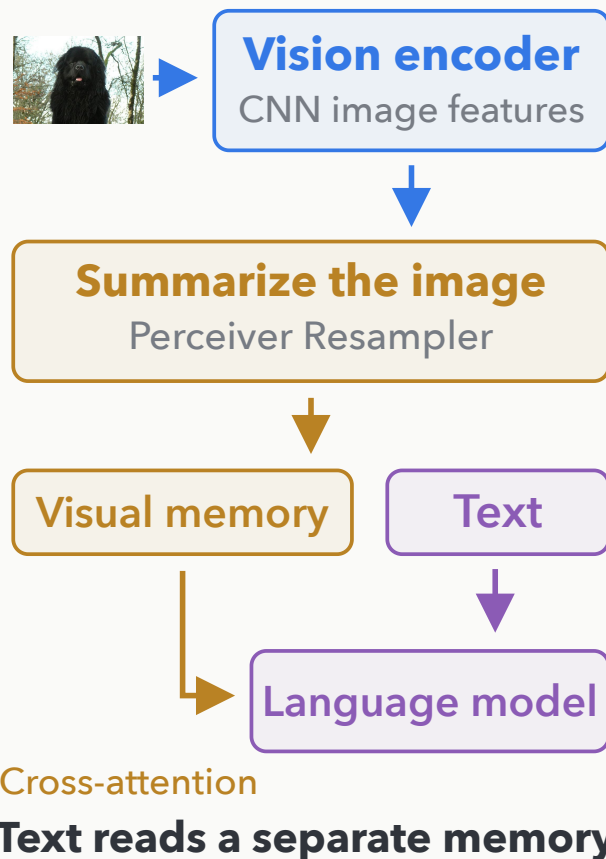
| The summary is useful when it preserves evidence for the questions we need to answer.

# How each model uses the image

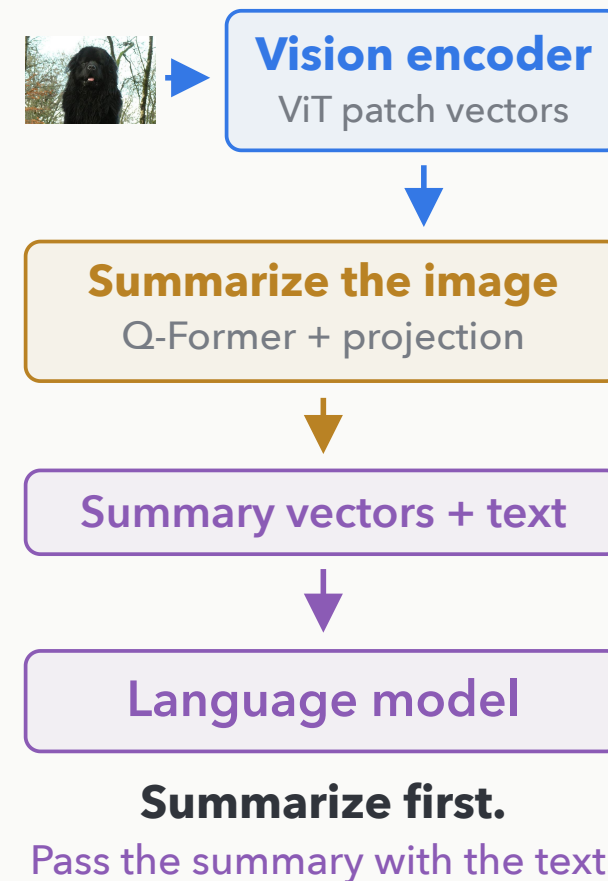
## LLaVA



## Flamingo



## BLIP-2



Text = question + answer so far. BLIP-2 shows its OPT language-model route.

# The learned readers stay; their image summaries change

## LEARNED / STORED

Vision weights

Connector weights

Language weights + token embeddings

Cross-attention weights, if present

Learned queries, if present

## COMPUTED FOR THIS INPUT

Image: visual states  $H$  and projected  $Z$

Prompt: language hidden states

Positions: queries, keys and values

Attention weights + vocabulary logits

Selected tokens appended at inference

Parameters persist across examples; activations depend on the input.

# Idea 3, end to end in one function

**image**

One image

**question** = "What animal is shown?"**answer\_so\_far** = "It is a" (empty at the start)

One next-token step · high-level pseudocode

Shapes · batch omitted

|                                                                 |                         |
|-----------------------------------------------------------------|-------------------------|
| <code>def next_token(image, question, answer_so_far):</code>    |                         |
| <code>    patches = vit(image).patch_states</code>              | $N \times d_V$          |
| <code>    summary = summarizer(queries, patches)</code>         | $M \times d_S; M < N$   |
| <code>    visual = projector(summary)</code>                    | $M \times d_{LM}$       |
| <code>    ids = tokenize(question, answer_so_far)</code>        | T token IDs             |
| <code>    text = embed(ids)</code>                              | $T \times d_{LM}$       |
| <code>    inputs = concat([visual, text], axis="tokens")</code> | $(M + T) \times d_{LM}$ |
| <code>    hidden = causal_decoder(inputs)</code>                | $(M + T) \times d_{LM}$ |
| <code>    return argmax(lm_head(hidden[-1]))</code>             | Next text token ID      |

**summarizer**: M learned queries read all N patch rows through cross-attention; outputs depend on the image. Shown: BLIP-2 / OPT-style prefix. For a Flamingo-style connection, use the summaries as Idea 2's memory.

## SECTION 06

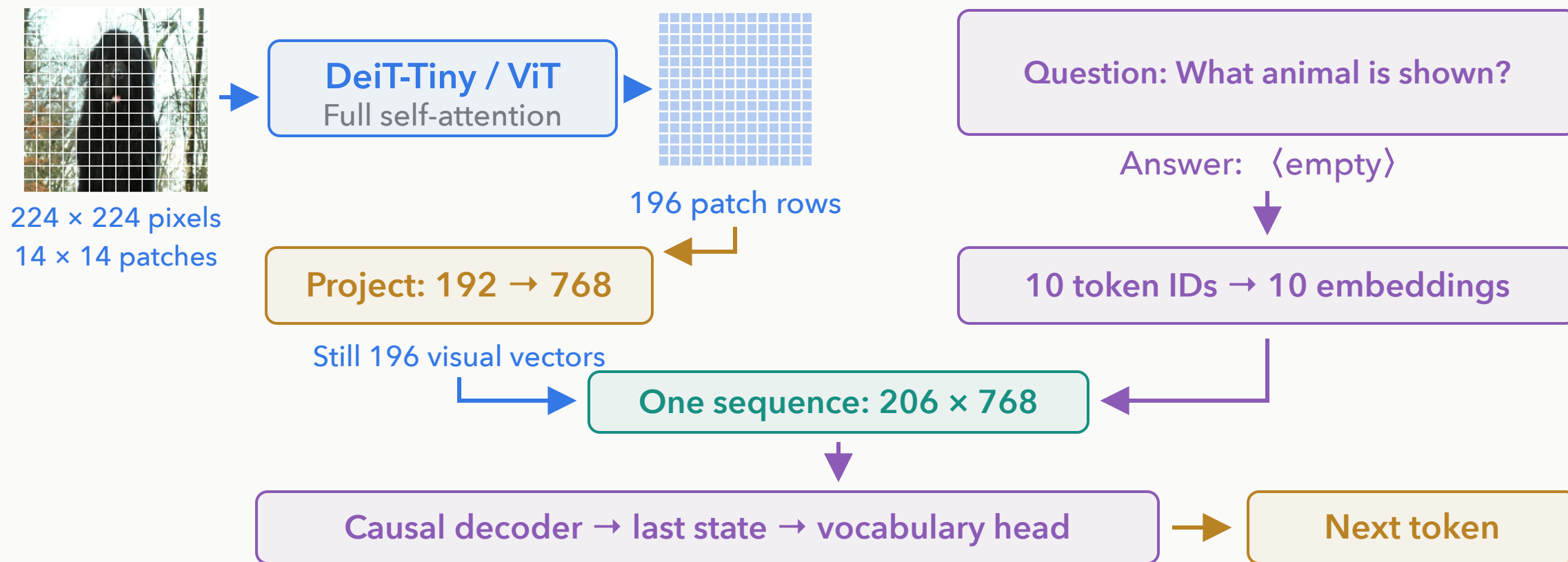
# Generate an answer with Idea 1

Keep every patch vector. Prepend them to the question. Follow one fixed image and prompt through the causal decoder.

# Use Idea 1: all patch vectors become earlier context

IDEA 1 • ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs • illustrative answer

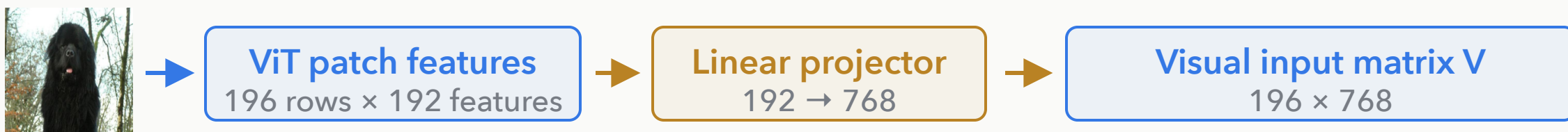


One chosen route: all image patches → projection → visual prefix → causal decoder.

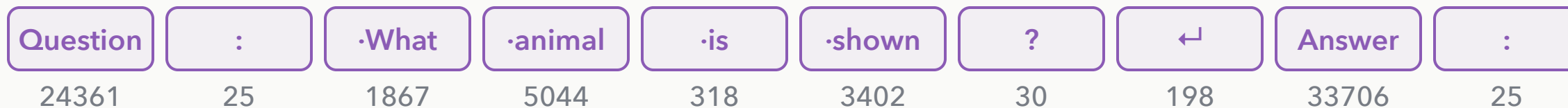
# Build the exact image-and-question input

## IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

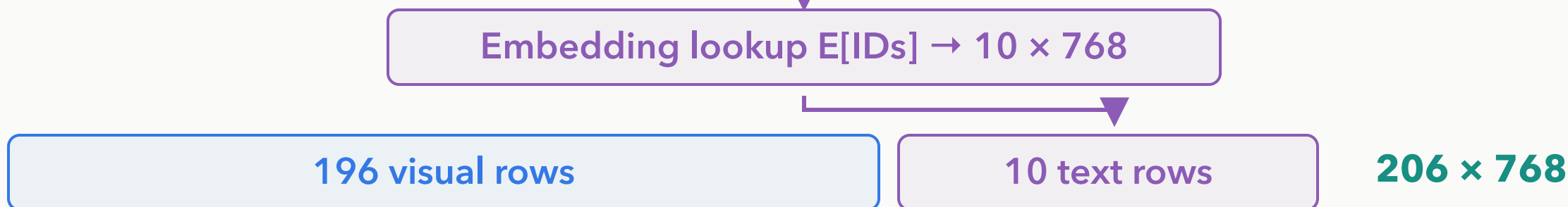
Real shapes / token IDs · illustrative answer



Question: What animal is shown? ↵ Answer:



10 real token IDs; · marks a leading space, ↵ marks the newline.

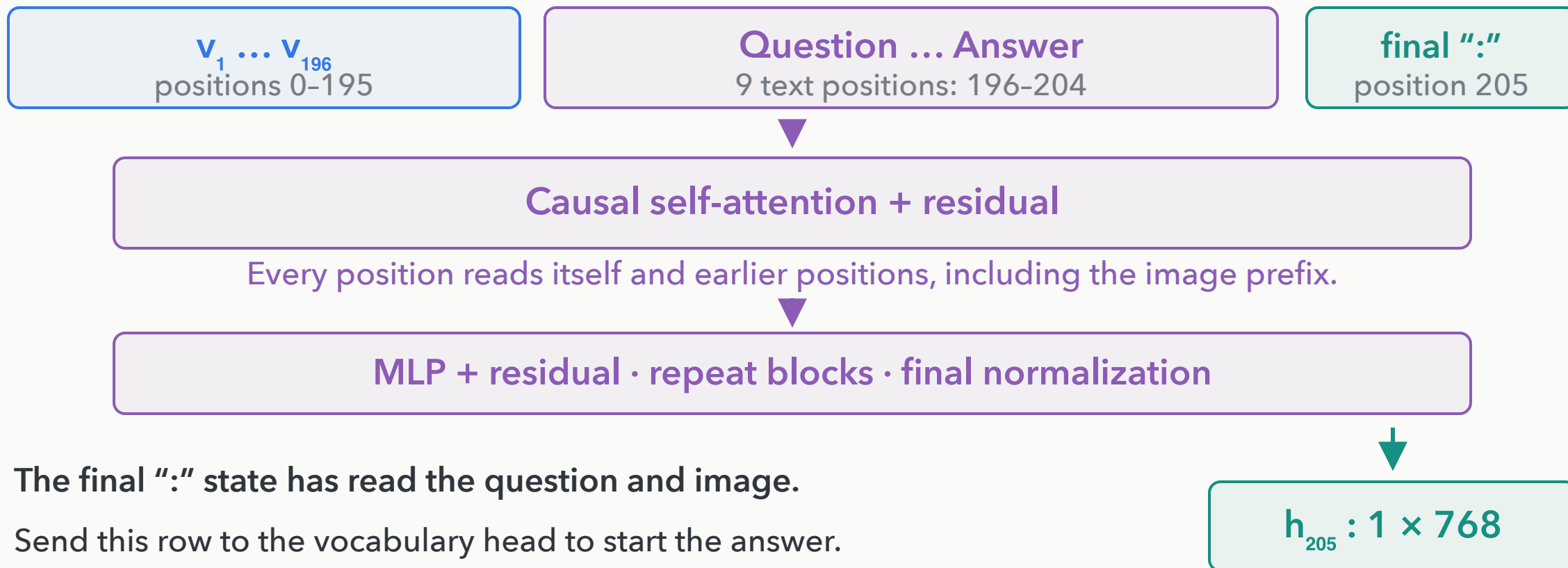


196 visual rows + 10 prompt embeddings = 206 decoder positions.

# Read the last position to start the answer

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs · illustrative answer



Update the supplied embeddings first; predict a token only at the vocabulary head.

# How does one state become vocabulary scores?

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Hand-set toy head · not model scores



Tiny worked head: choose  $h = [1, 2]$ , four vocabulary entries, and zero bias.



For ·A:  $1 \times 1 + 2 \times 1 = 3$

Each output column scores one vocabulary entry using the same final state.

# Turn the scores into one next-token choice

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Hand-set toy head · not model scores

softmax: exponentiate each score, then divide by their sum

| Token | Logit | exp(logit) | Probability |        |
|-------|-------|------------|-------------|--------|
| ·A    | 3     | 20.086     | <div></div> | 0.8310 |
| ·The  | 1     | 2.718      | <div></div> | 0.1125 |
| ·dog  | 0     | 1.000      | <div></div> | 0.0414 |
| EOS   | -1    | 0.368      | <div></div> | 0.0152 |

Sum = 24.172

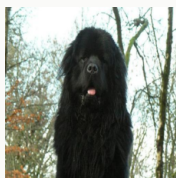
Sum = 1; displayed values rounded.

Greedy choice: " A" → token ID 317 → embedding lookup next

The four-entry calculation is a toy; the real vocabulary has 50,257 entries.

# Feed the selected token back as an embedding

## IDEA 1 • ALL PATCH ROWS → VISUAL PREFIX



What animal is shown?

Answer so far: empty

The head returns an ID.

The decoder receives its embedding.

Real shapes / token IDs • illustrative answer

Selected "A" • ID 317



E[317]: one 768-wide row



E[317]

new position 206

196 image vectors

10 prompt embeddings



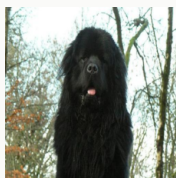
207 positions → same causal decoder → next token

Append E[317] for "A"; the image and question stay fixed.

# The “A” position now predicts “black”

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs · illustrative answer

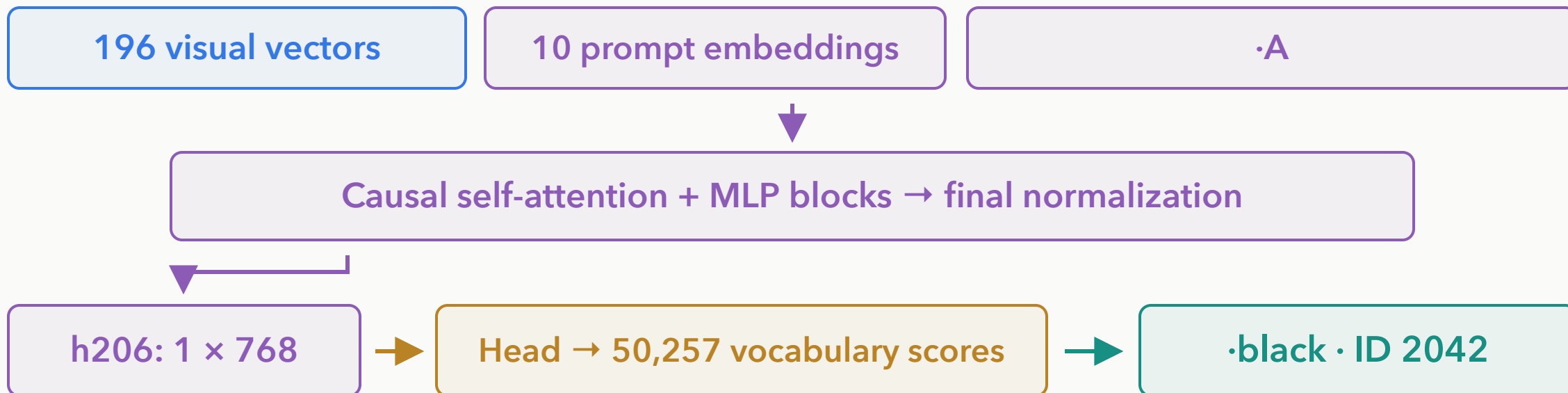


What animal is shown?

Answer so far: A

**196 + 10 + 1 = 207 positions**

Last input: ·A · position 206



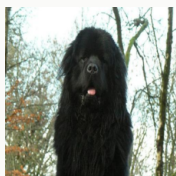
Next step: append ID 2042, look up its embedding, and repeat.

A selected token becomes part of the next step's input.

# The “black” position now predicts “dog”

IDEA 1 • ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs • illustrative answer

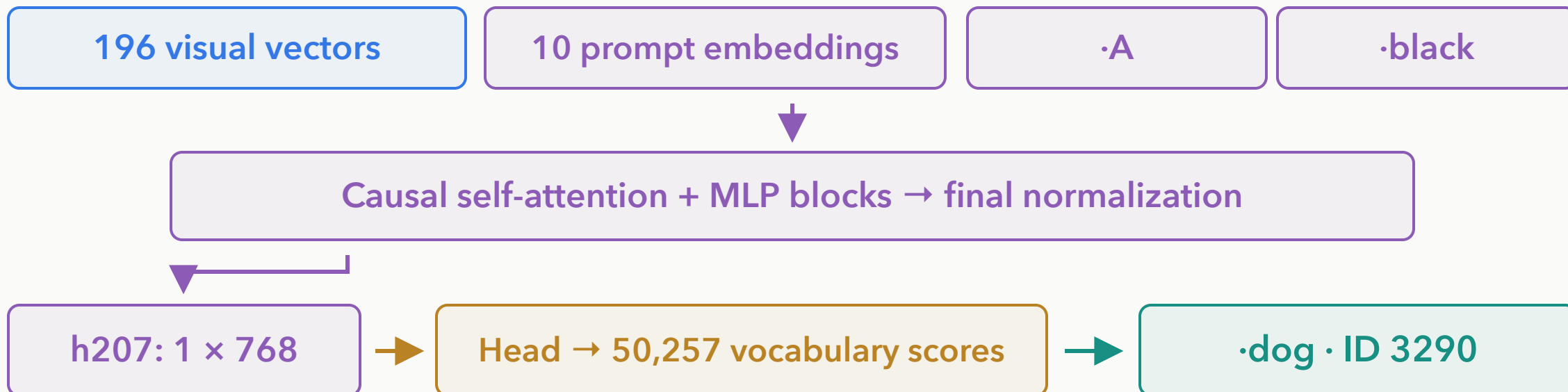


What animal is shown?

Answer so far: A black

**196 + 10 + 2 = 208 positions**

Last input: ·black · position 207



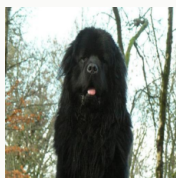
Next step: append ID 3290, look up its embedding, and repeat.

Read the newest position after the decoder; reuse the same vocabulary head.

# After " dog", predict the punctuation

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs · illustrative answer

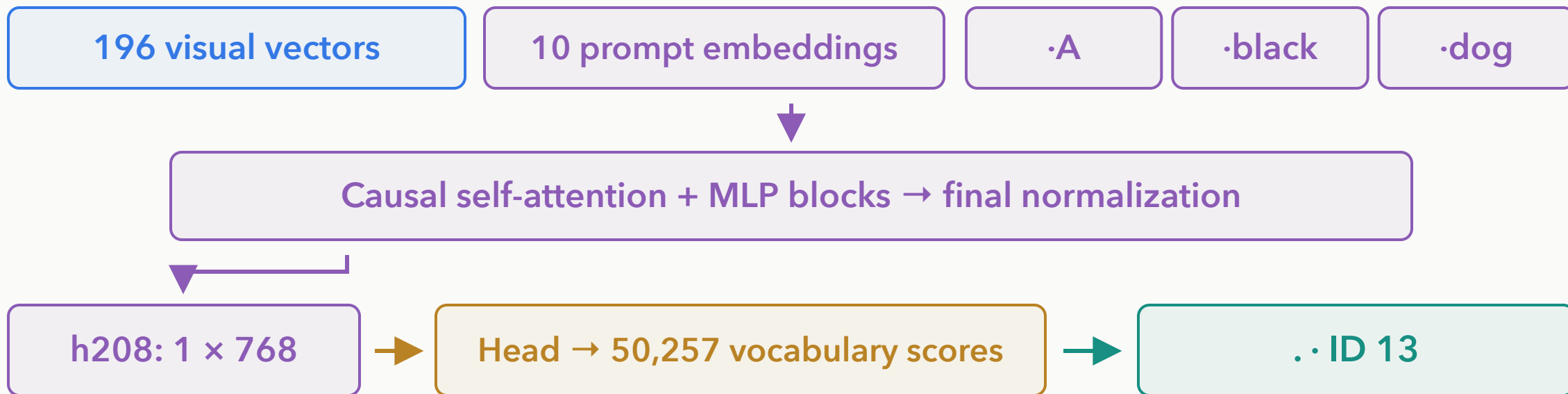


What animal is shown?

Answer so far: A black dog

**196 + 10 + 3 = 209 positions**

Last input: ·dog · position 208



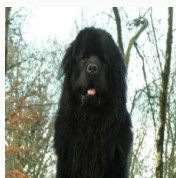
Next step: append ID 13, look up its embedding, and repeat.

The period is another vocabulary token, selected by the same computation.

# EOS ends the answer

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs · illustrative answer

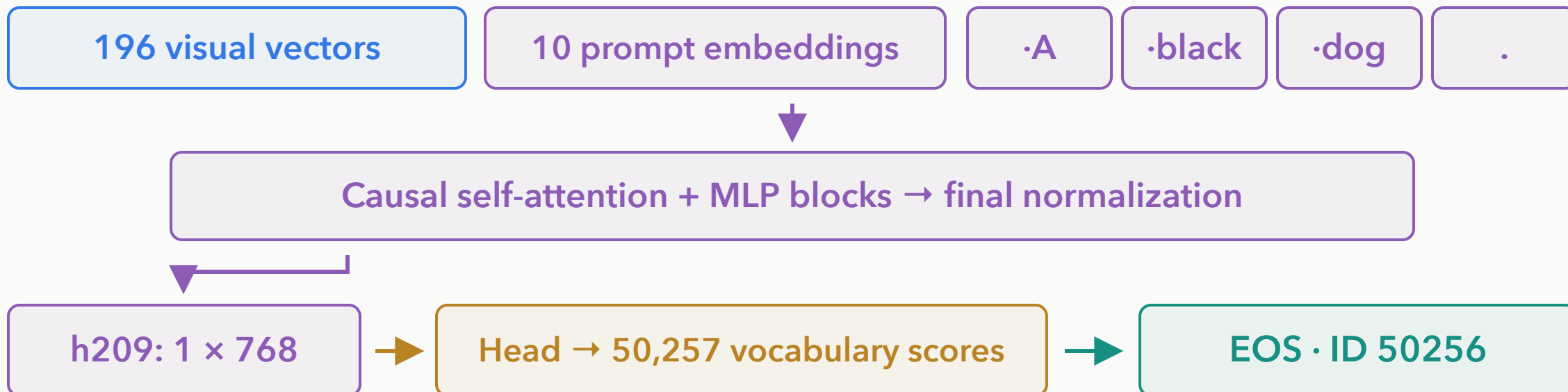


What animal is shown?

Answer so far: A black dog.

**196 + 10 + 4 = 210 positions**

Last input: . · position 209



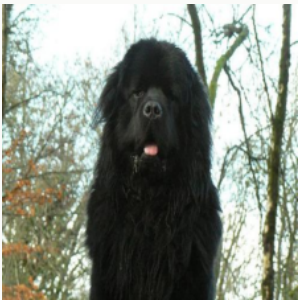
EOS: stop this reply; do not display the control token.

EOS is a control-token ID; stop generation and decode the answer text.

# Check the whole trace: one new token per step

IDEA 1 · ALL PATCH ROWS → VISUAL PREFIX

Real shapes / token IDs · illustrative answer



Fixed image  
196 visual rows

What animal is shown?  
10 prompt embeddings

Same decoder + head  
at every step.

| Answer so far | Input rows | Last position | Select |
|---------------|------------|---------------|--------|
| ⟨empty⟩       | 206        | 205           | ·A     |
| A             | 207        | 206           | ·black |
| A black       | 208        | 207           | ·dog   |
| A black dog   | 209        | 208           | .      |
| A black dog.  | 210        | 209           | EOS    |

Decoded illustrative answer: " A black dog."

Fixed image and question; a growing answer prefix; the same decoder and head.

## Measured check: change only the image.

What animal is shown?



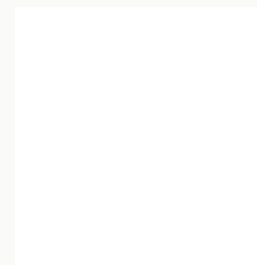
**Dog**

| There is a black dog in the foreground.



**Cat**

| A cat is visible.



**Blank**

| A squirrel.

$$p(y_t \mid y_{<t}, I, P)$$

| Recorded SmolVLM-256M answers: a separate trained model.

## SECTION 07

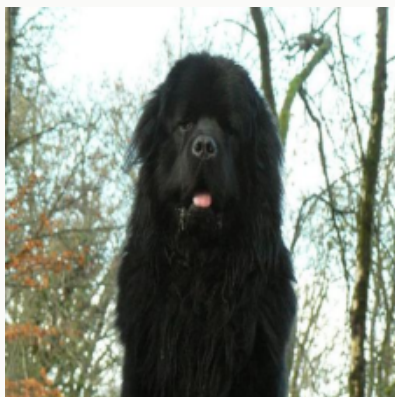
# How do we train the connection?

Stay with Idea 1 and the same dog image. Supply a correct answer, score its tokens, then update the visual connection.

# Training starts with a correct answer supplied by us

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

One authored reference answer · real token IDs



The input image

What animal is shown?

The input question

A black dog.

Written by the teacher

This is the reference answer.

### Training targets

·A

ID 317

·black

ID 2042

·dog

ID 3290

.

ID 13

EOS

ID 50256

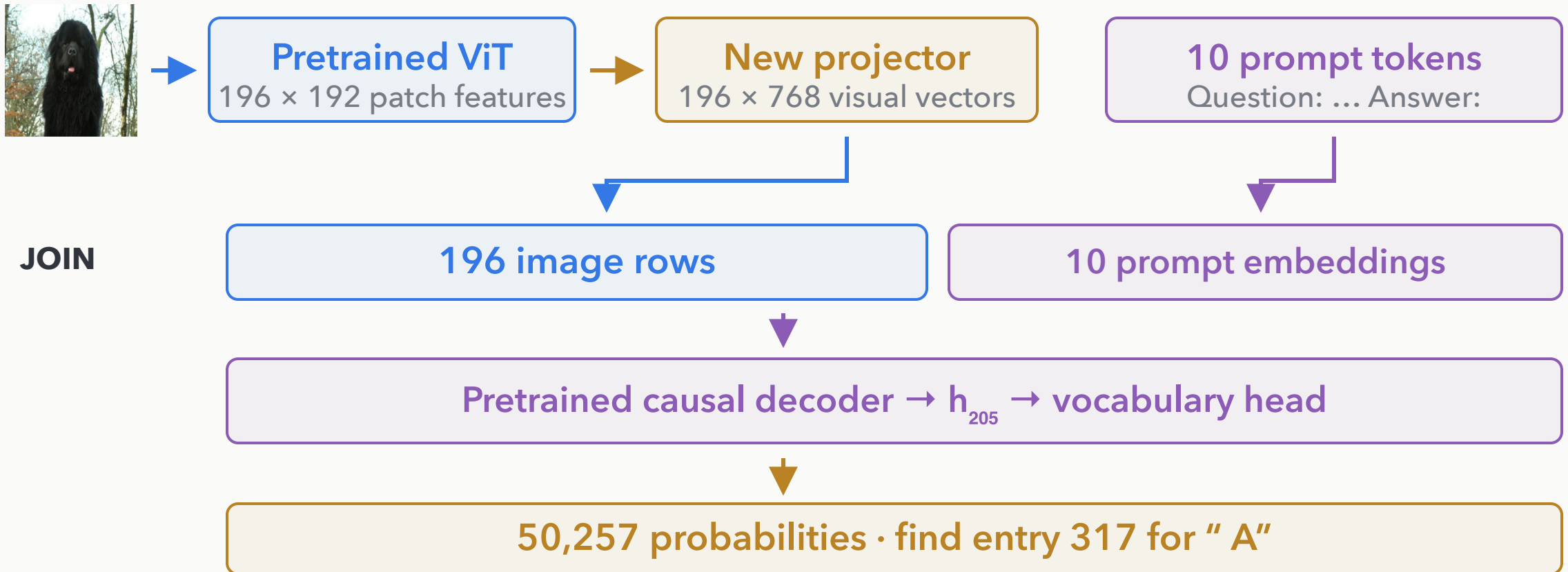
· marks a leading space. EOS says the answer is finished.

The dataset supplies the answer we want the model to learn.

# Run the same model; inspect the first answer prediction

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

One authored reference answer · real token IDs



The reference ID tells us which vocabulary probability to inspect.

# How much probability did the correct token receive?

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

Illustrative probabilities · not measured

Reference target: " A"

From the last "Answer:" state

softmax over all 50,257 vocabulary scores

Select  $p[317] = 0.20$

All entries sum to 1.

The target stays " A".



Loss for this target =  $-\ln(0.20) = 1.609$

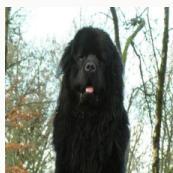
If  $p[317]$  were 0.80, its loss would be only  $-\ln(0.80) = 0.223$ .

Low probability on the correct token gives a large loss.

# Teach every next token using the correct prefix

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

Illustrative probabilities · not measured



Question: What animal is shown?

Reference: A black dog.

Input ends with

Answer:

·A

·black

·dog

.

Next target

·A

·black

·dog

.

EOS

To predict " dog", supply " A black" – even if the model would have guessed differently.

$p(\text{correct target})$

0.20

0.40

0.50

0.80

0.90

$-\ln p$

1.609

0.916

0.693

0.223

0.105

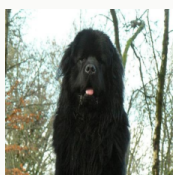
**Mean answer loss =  $(1.609 + 0.916 + 0.693 + 0.223 + 0.105) / 5 = 0.709$**

Known reference prefixes let us score all five targets in one causal pass.

# One input sequence; five answer targets to score

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

One authored reference answer · real token IDs



196 image rows + 10 prompt tokens + 5 reference tokens = 211 positions

Each decoder input row has 768 features. Batch size  $B = 1$ .

196 image vectors

10 prompt tokens

.A

.black

.dog

.

EOS

**Target labels at the same positions:**

–100 means “do not score this target”.

–100 at all 206 context positions

317

2042

3290

13

50256

**logits at positions 205-209 → labels at positions 206-210**

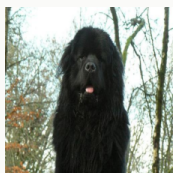
The final prompt “:” predicts the first “A”. The final EOS logit is unused.

**Ignore context labels; still use the image and question as context.**

# Knowing the answer does not let the model peek ahead

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

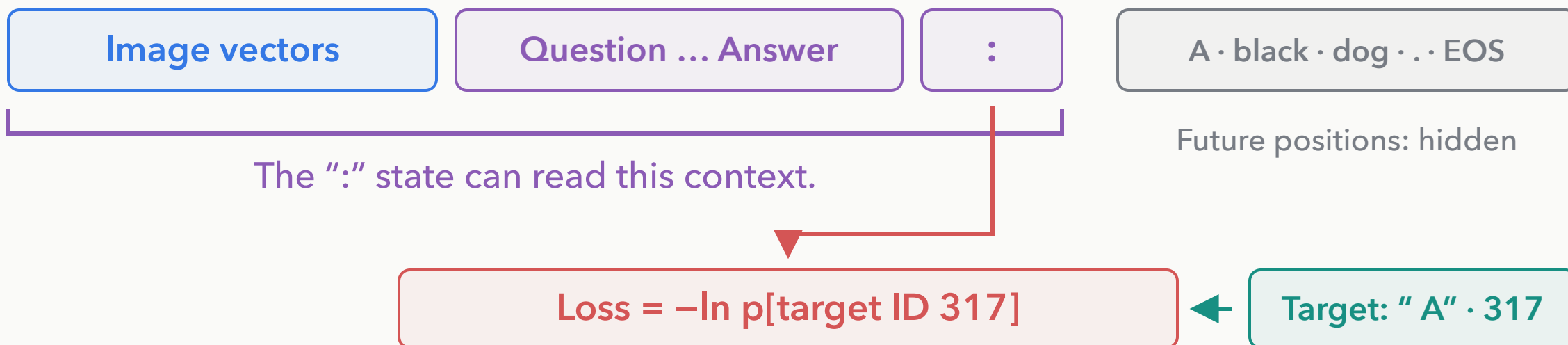
One authored reference answer · real token IDs



Question: What animal is shown?

Answer:  $\langle \text{empty} \rangle$

Predict "A" at the last prompt ":"



We grade answer targets only. The question remains visible to attention.

Attention controls what a state reads. Loss labels select what we grade.

# Use the answer loss to change the projector

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

One authored reference answer · real token IDs

### FORWARD



Also receives prompt + reference embeddings.

### BACKWARD



The LM passes the gradient back; only the projector changes.

$$\text{Projector weights} \leftarrow \text{weights} - \text{learning rate} \times \text{gradient}$$

Frozen LM weights still pass a gradient to the trainable visual connection.

# One training step: predict, score, update

## TRAIN IDEA 1 · THE SAME IMAGE AND QUESTION

One authored reference answer · real token IDs

Setup: frozen vit + lm; trainable projector; optimizer contains only projector weights.

One example; no padding. answer\_ids includes EOS. Dropout is off for this demonstration.

```

1  with torch.no_grad(): H = vit(pixel_values=pixels).last_hidden_state[:, 1:]
2  V = projector(H)
3  ids = torch.cat([prompt_ids, answer_ids], dim=1)
4  X = torch.cat([V, lm.get_input_embeddings()(ids)], dim=1)
5  labels = torch.full(X.shape[:2], -100, device=ids.device)
6  labels[:, V.shape[1] + prompt_ids.shape[1]:] = answer_ids
7  loss = lm(inputs_embeds=X, labels=labels).loss
8  optimizer.zero_grad()
9  loss.backward()
10 optimizer.step()

```

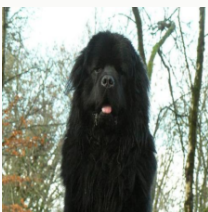
**Repeat with the next example. Shapes:  $X = [1, 211, 768]$ ; labels =  $[1, 211]$ .**

**Repeat this step on many labelled image-text examples.**

# What changes from caption training to instructions?

## IDEA 1 · TWO TRAINING STAGES IN LLaVA

Authored examples · original training recipe



### Same rule in both stages:

reference tokens → next-token loss → gradients → selected weight updates

#### 1 · Learn from image-caption pairs

Prompt: "Describe this image."

Reference: "A black dog outdoors."

#### Update the projector.

Keep the vision encoder and LM fixed.

#### 2 · Learn from image + question + answer

Question: "What colour is the dog?"

Reference: "Black."

#### Update the projector and LM.

Keep the vision encoder fixed.

**Original LLaVA recipe · other VLMs choose different modules to train.**

Captions and QA above are teaching examples, not quotations from its dataset.

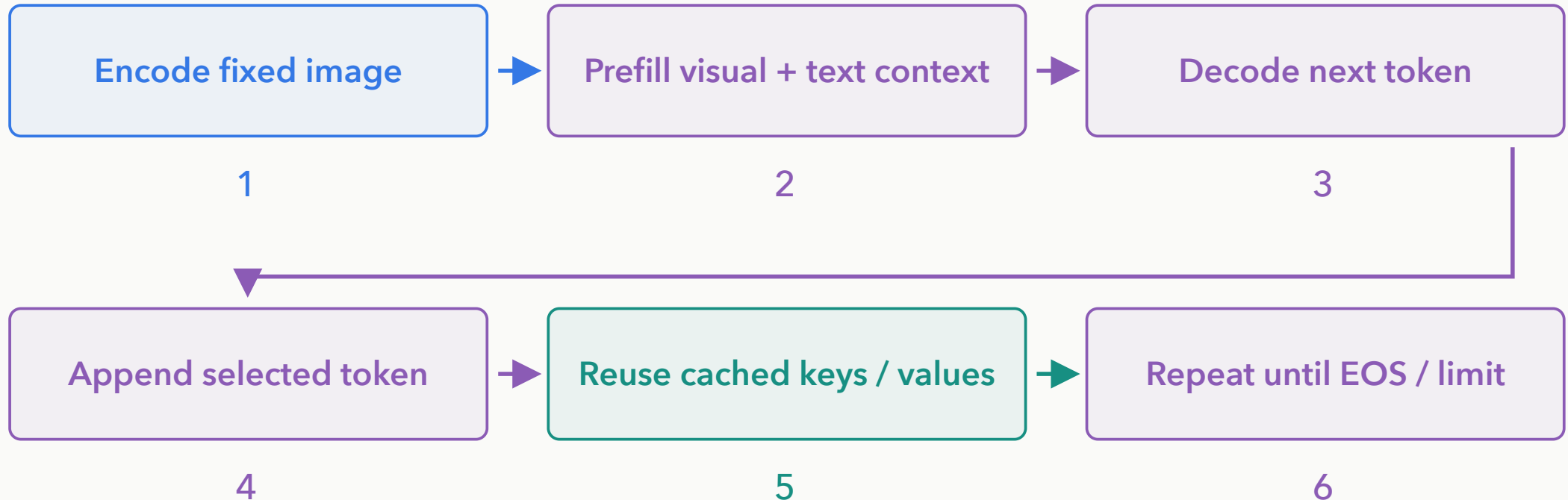
**The learning rule stays; the examples and trainable weights change.**

## SECTION 08

# What happens at inference?

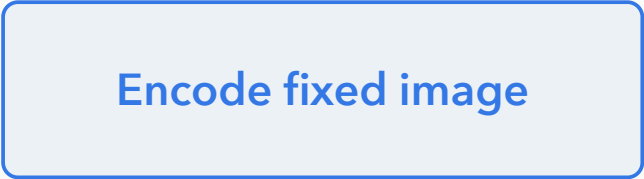
At inference: prepare the image once, then grow the answer.

# The whole generation loop before its individual steps



Prepare the fixed context once; repeatedly predict and append a token.

# Encode the fixed image once

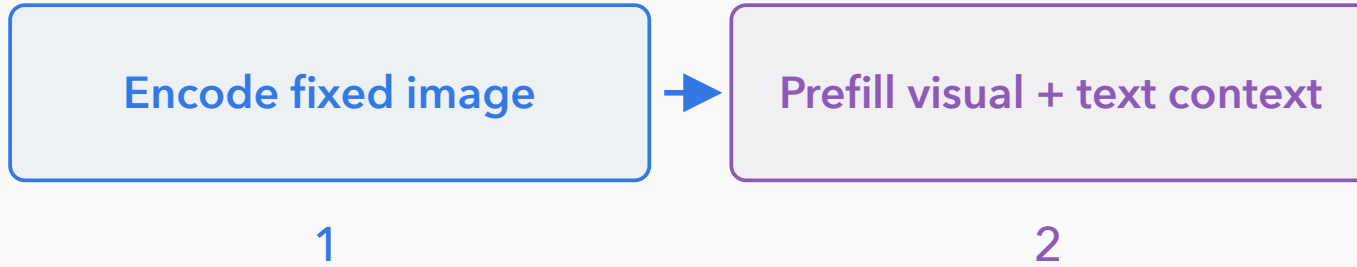


Encode fixed image

1

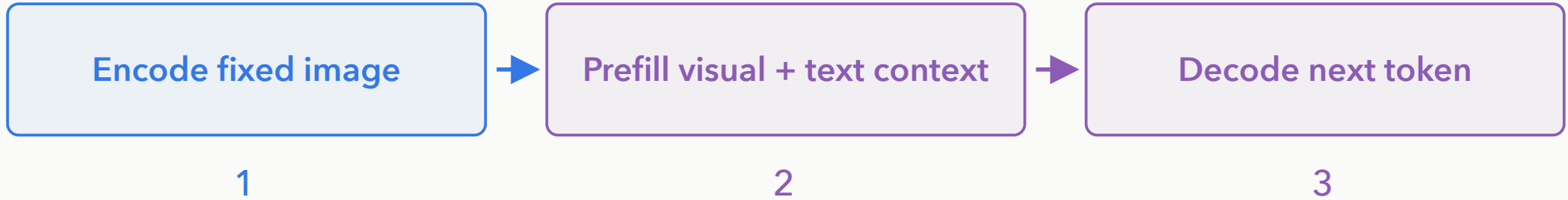
| Only the generated prefix changes at each decoding step.

# Prefill the image and question context



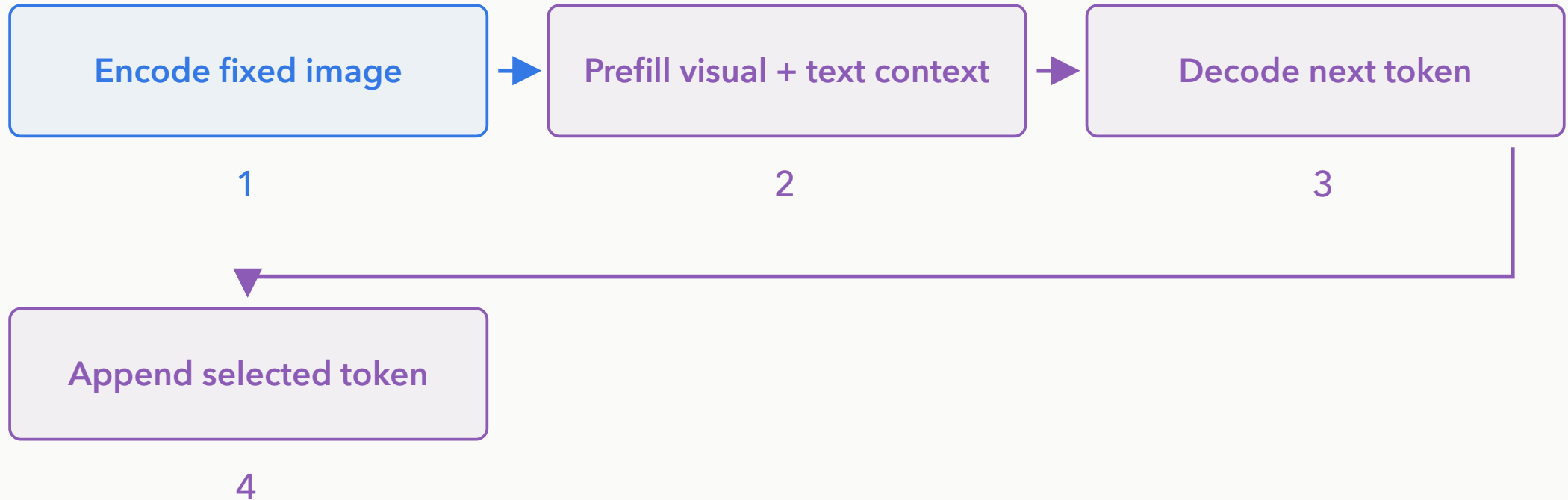
| Only the generated prefix changes at each decoding step.

# Decode one new token



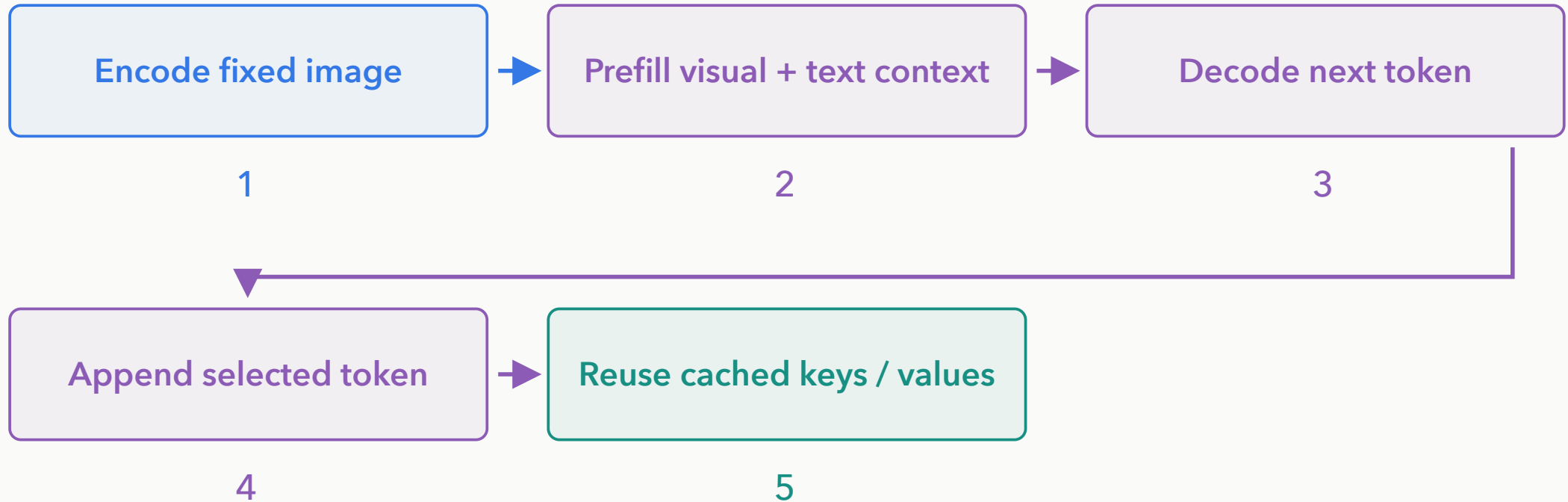
| Only the generated prefix changes at each decoding step.

# Append the selected token



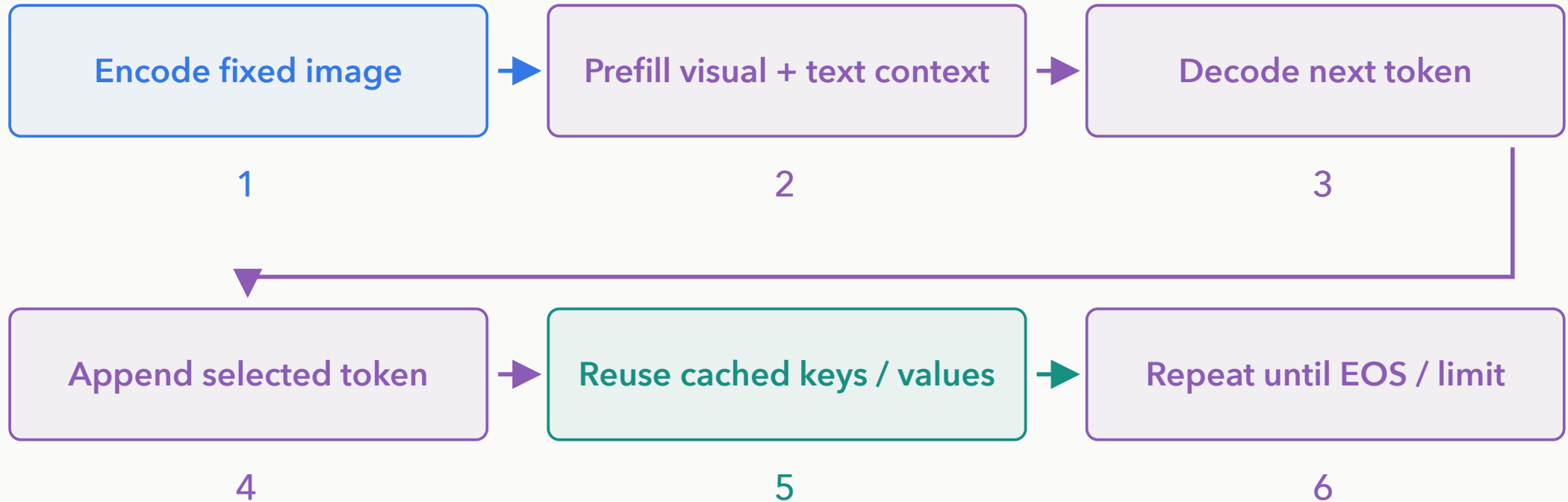
| Only the generated prefix changes at each decoding step.

# Reuse the cached keys and values



| Only the generated prefix changes at each decoding step.

# Repeat until EOS or the generation limit



| Only the generated prefix changes at each decoding step.

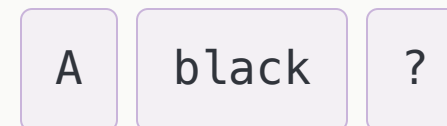
# Same conditional model, different available prefix

## Training



Reference prefix supplied.  
Target loss and gradients.

## Inference



Generated prefix grows.  
No reference answer or backward pass.

**| Known targets allow parallel teacher-forced positions; generation is sequential.**

## Reuse the image; keep the conversation context



User: What animal is this?

Assistant: A dog.

User: What colour is its fur?

Assistant: Black.

Illustrative conversation, not a captured model transcript.

| Visual features can persist. Decoder cache reuse also depends on the exact prefix.

## SECTION 09

# Can we trust the answer?

Check evidence use, not just fluent language.

# A fluent sentence can have no visual support



| Fluent ≠ grounded.

What color is the bicycle?

---

Reference: No bicycle is visible.

| Recorded baseline: "The bicycle is red."

# Keep a small diagnostic suite

| IMAGE   | FIXED QUESTION                       | REFERENCE                    |
|---------|--------------------------------------|------------------------------|
| Dog     | What animal is this?                 | Dog                          |
| Desk    | Is the laptop plugged in?            | Yes, visible cable connected |
| Shapes  | What is immediately left of the mug? | Red ball                     |
| Invoice | What is the invoice number?          | ES667-042                    |
| Chart   | Which month has the highest value?   | March                        |
| Coffee  | What color is the bicycle?           | No bicycle visible           |

Run / record the diagnostic suite ↗

[Offline fallback: recorded model outputs](#)

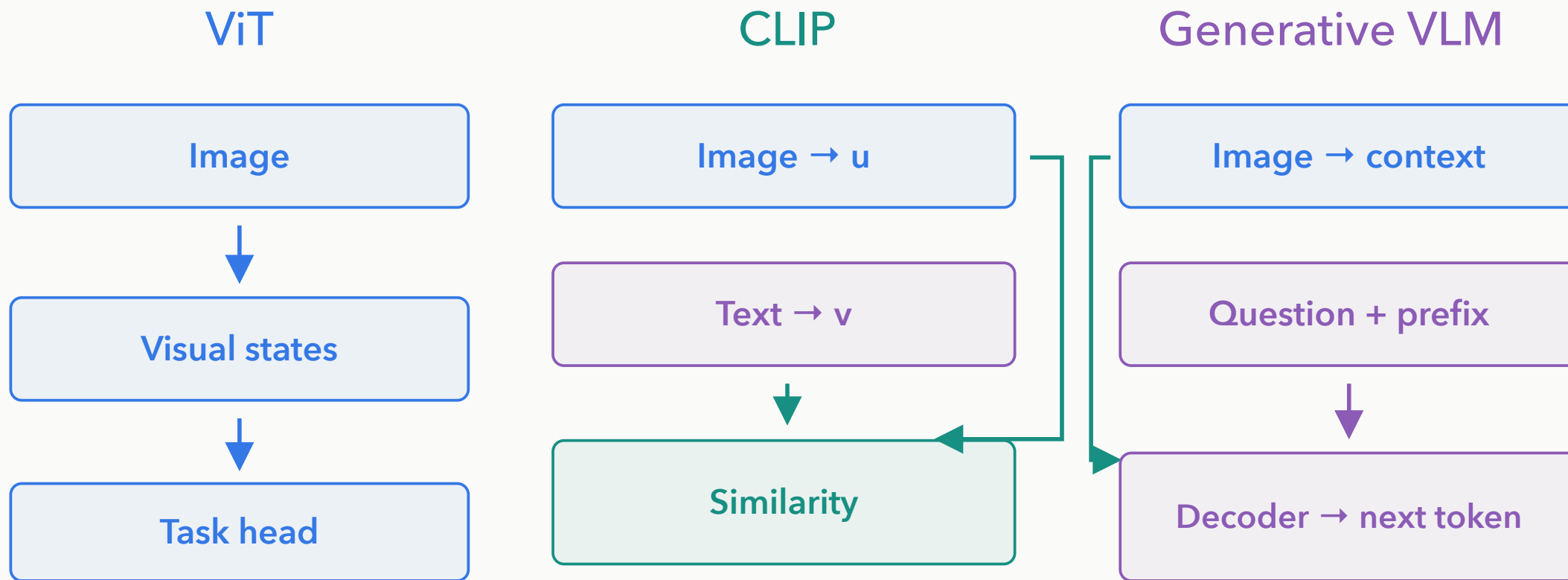
**Preserve the raw answer, reveal the reference, then judge support.**

## SECTION 10

# Summary

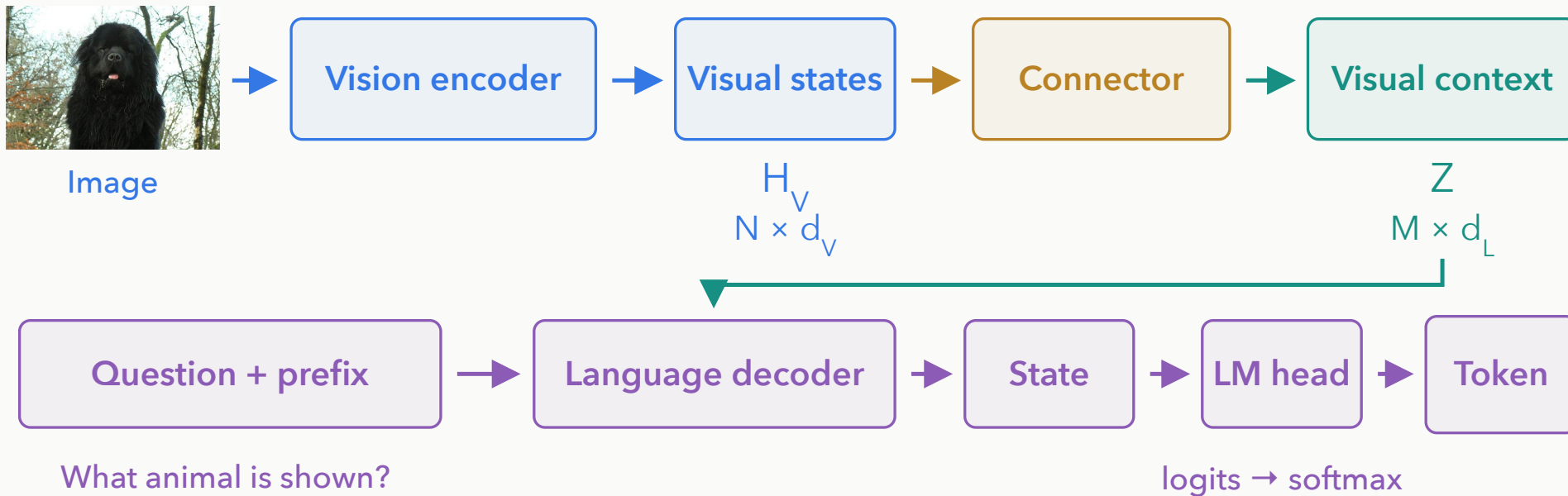
Represent the image, expose context, predict the next token.

# ViT → CLIP → generative VLM



Visual representation → cross-modal matching → conditioned generation.

# One computation to remember



$$p(y_t \mid y_{<t}, I, P)$$

Visual information becomes context for next-token prediction.

# Flamingo: a Visual Language Model for Few-Shot Learning

Jean-Baptiste Alayrac · Jeff Donahue · Pauline Luc · et al.

[Read the paper ↗](#)

## 6 · Read the colors

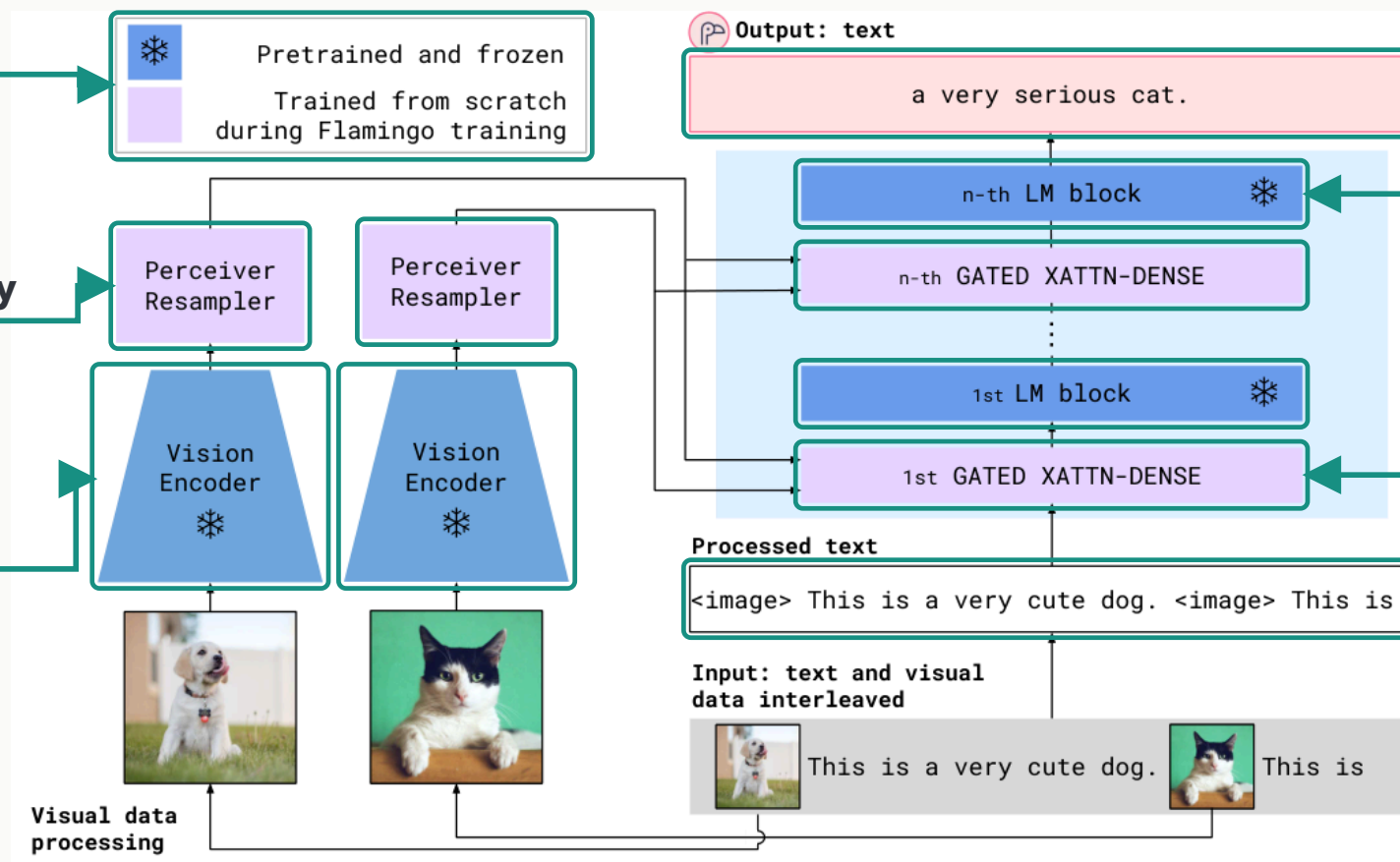
Blue + snowflake: pretrained and frozen.  
Purple: trained here.

## 2 · Make a summary

The resampler returns 64 vectors per image.

## 1 · Vision encoder

Each image becomes a grid of features.



## 5 · Language model

Update text states; predict the next token.

## 4 · Visual lookup

Text queries read the visual summaries.

## 3 · Supply the text

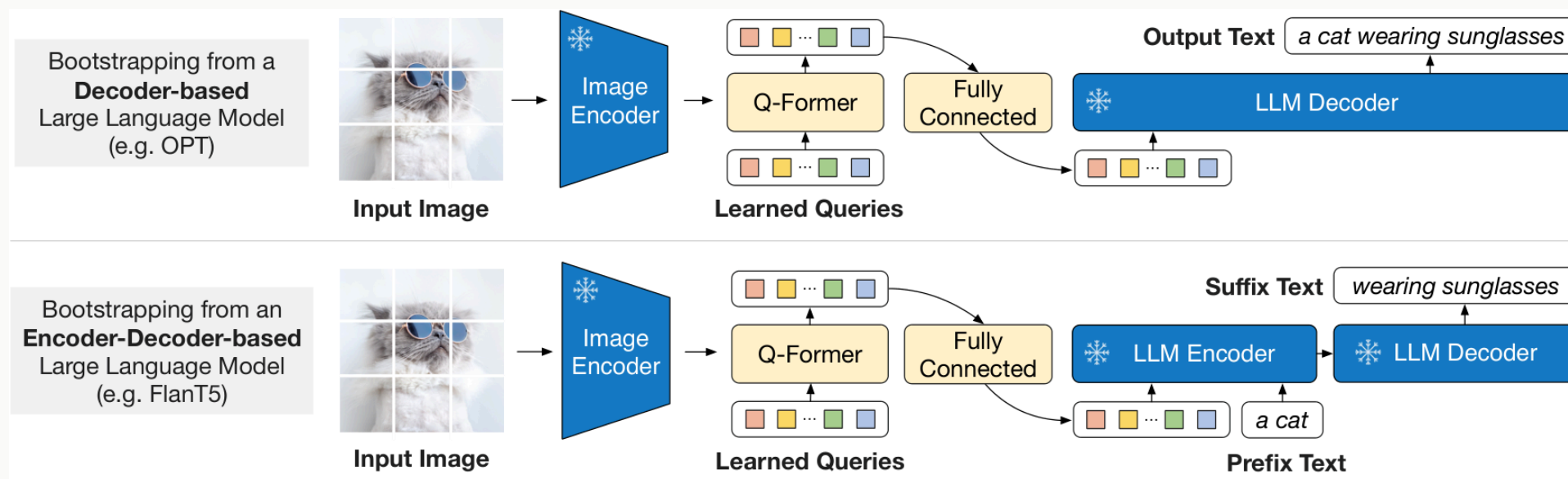
Tokenize, then embed words + image markers.

Original Figure 3 + teaching callouts · follow the arrows from the images and text upwards

# BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models

Junnan Li · Dongxu Li · Silvio Savarese · Steven Hoi

[Read the paper ↗](#)



Original Figure 3 · Top: decoder-only LLM. Bottom: encoder-decoder LLM.

## 1 · Idea 3 · learned-query summaries

32 learned queries produce 32 image-dependent output vectors.

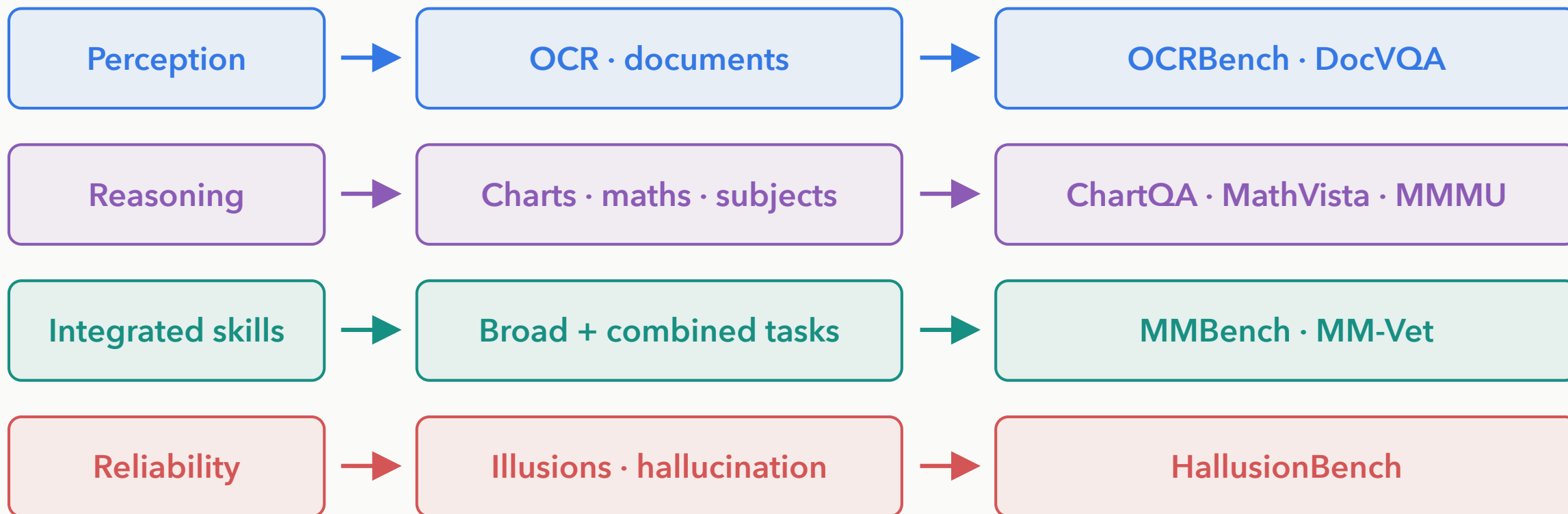
## 2 · Idea 1 · prefix for OPT (top)

Project the summaries; prepend them to text embeddings.

## 3 · Bottom: encoder-decoder

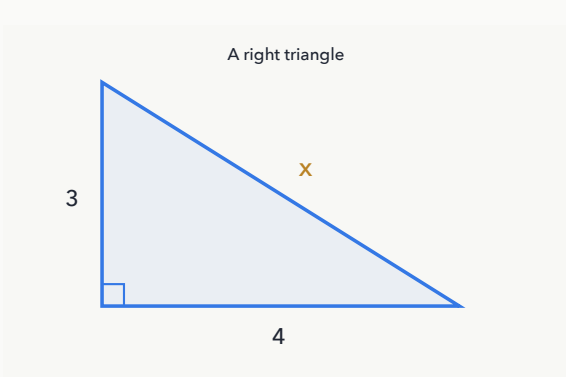
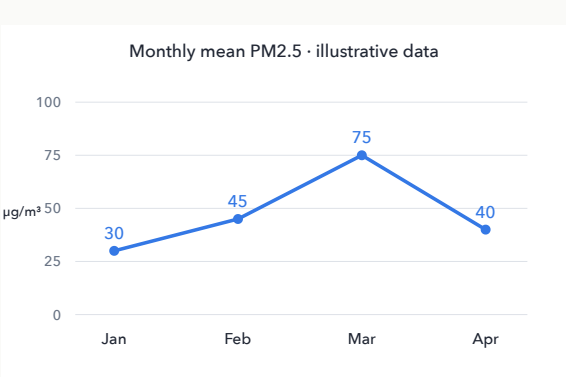
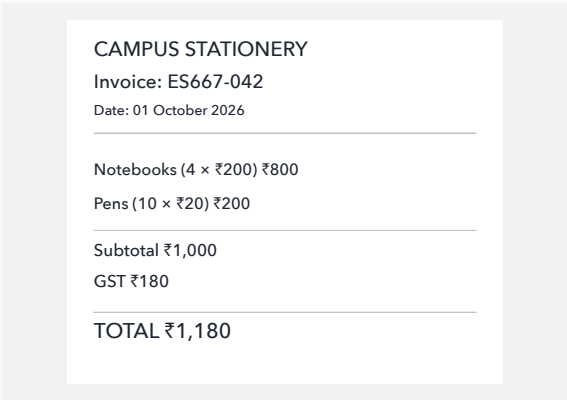
Visual vectors + text enter the encoder; its decoder generates the answer.

# Different benchmarks probe different capabilities



| Choose the evaluation from the capability you need.

# Inspect the questions before interpreting a score



Read: What is the invoice ID?

Calculate: March minus January?

Reason: What is x?

Reject: What colour is the bicycle?

Authored classroom examples, not copied benchmark items.

**A wrong answer may come from perception, reasoning, or accepting a false premise.**

# Use a metric that matches the answer format

| OUTPUT / USE          | MEASURE                               | WHAT STILL NEEDS INSPECTION          |
|-----------------------|---------------------------------------|--------------------------------------|
| Fixed answer          | Accuracy                              | Ambiguity and answer normalization   |
| Document text         | Exact / normalized string match; ANLS | OCR errors and layout dependence     |
| Open explanation      | Human or specified model rubric       | Judge bias and unsupported details   |
| Grounding coordinates | IoU / pointing accuracy               | Object identity and spatial accuracy |
| Deployment            | Latency, memory, output tokens/s      | Hardware and decoding settings       |

**Report the evaluation protocol along with the number.**

# Further training can change how the model answers

## Behaviour

Instruction following  
Preference optimisation  
Safer responses

## Evidence use

Better grounding data  
Hard negative examples  
Missing-evidence responses

| The basic inference target remains image-conditioned next-token prediction.

# Sources • architectures and browser implementation

[CLIP • Radford et al., 2021](#)

<https://arxiv.org/abs/2103.00020>

[LLaVA • Liu et al., 2023](#)

<https://arxiv.org/abs/2304.08485>

[Flamingo • Alayrac et al., 2022](#)

<https://arxiv.org/abs/2204.14198>

[BLIP-2 • Li et al., 2023](#)

<https://arxiv.org/abs/2301.12597>

[SmolVLM-256M • model card](#)

<https://huggingface.co/HuggingFaceTB/SmolVLM-256M-Instruct>

[Official Transformers.js browser example](#)

<https://github.com/huggingface/transformers.js-examples/tree/main/smolvlm-webgpu>

[Oxford-IIIT Pet • CC BY-SA 4.0](#)

<https://www.robots.ox.ac.uk/~vgg/data/pets/>

**Primary sources; no leaderboard scores are used in this lecture.**

# Sources • evaluation

[OCRBench](#)

<https://arxiv.org/abs/2305.07895>

[DocVQA](#)

<https://arxiv.org/abs/2007.00398>

[ChartQA](#)

<https://arxiv.org/abs/2203.10244>

[MathVista](#)

<https://arxiv.org/abs/2310.02255>

[MMMU](#)

<https://arxiv.org/abs/2311.16502>

[MMBench](#)

<https://arxiv.org/abs/2307.06281>

[MM-Vet](#)

<https://arxiv.org/abs/2308.02490>

[HallusionBench](#)

<https://arxiv.org/abs/2310.14566>

**| The local six-image suite is an instructional diagnostic, not one of these benchmarks.**